

سیستم های عامل

CompYaar

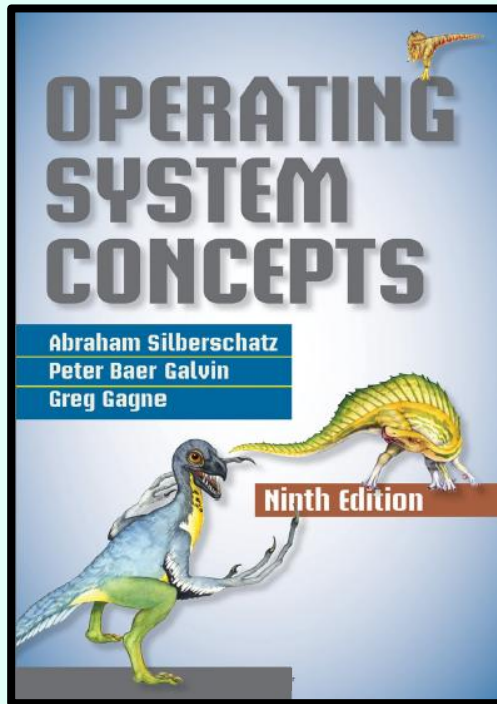
منبع

کتاب Operating System Concepts

Abraham Silberschatz -

Peter Baer Galvin -

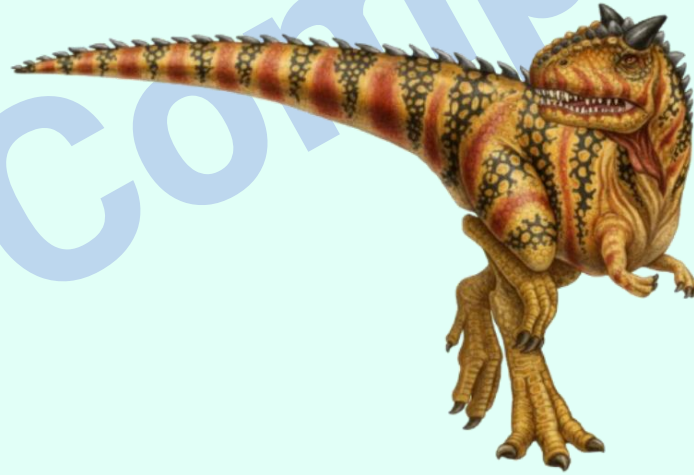
Greg Gagne -



مباحث

آشنایی (Introduction)
ساختارهای سیستم عامل (Operating System Structures)
فرآیندها (Processes)
نخها (Threads)
همگام سازی فرآیندها (Process Synchronization)
زمان بندی پردازنده (CPU Scheduling)
بن بست ها (Deadlocks)
حافظه اصلی (Main Memory)
حافظه مجازی (Virtual Memory)
ساختارهای حافظه انبوه (Mass-Storage Structure)

Chapter 1: Introduction



سیستم عامل چیست؟

■ برنامه‌ای که به عنوان واسطه بین کاربر کامپیوتر و سخت افزار عمل می‌کند

اهداف سیستم عامل:

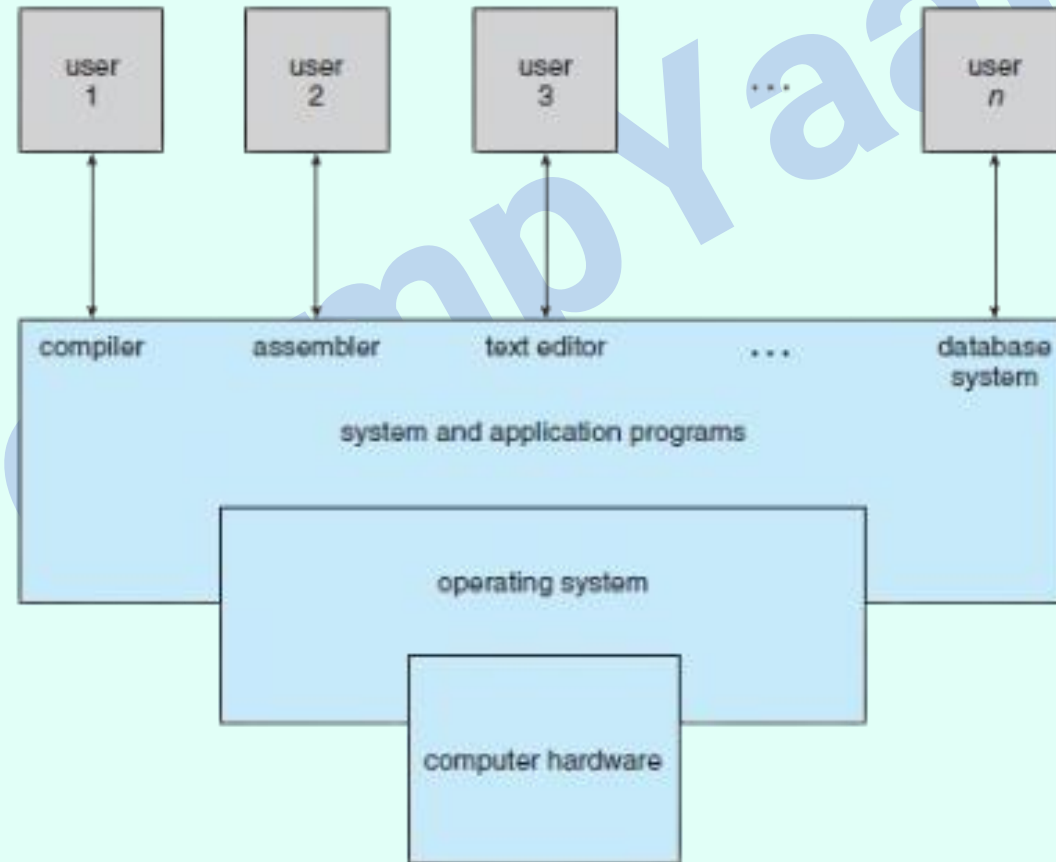
- اجرای برنامه‌های کاربر و ساده‌سازی حل مشکلات کاربر
- ساده‌سازی استفاده از کامپیوتر
- استفاده بهینه از سخت افزار کامپیوتر

ساختار سیستم کامپیوتری

یک سیستم کامپیوتری را می‌توان به چهار قسمت تقسیم کرد:

1. سخت افزار (Hardware): تأمین منابع پردازشی
(مثال) CPU، حافظه، دستگاه‌های I/O
2. سیستم عامل (Operating System): کنترل استفاده از سخت افزار میان برنامه‌ها و کاربرهای مختلف
3. برنامه‌های کاربردی (Application Programs): برنامه‌هایی که کاربر از آن‌ها استفاده می‌کند
(مثال) جستجوگرهای وب، بازی‌های ویدیویی و ...
4. کاربران (Users): انسان، کامپیوترهای دیگر و ...

ساختار سیستم کامپیوتری



سیستم عامل چیست؟

■ سیستم عامل یک تخصیص دهنده منابع است

منابع را مدیریت می کند و به طور بهینه تخصیص می کند

■ سیستم عامل یک برنامه کنترلی است

اجرای برنامه ها را کنترل می کند تا از خطاها و استفاده نامناسب کامپیوتر جلوگیری کند

Kernel

Kernel: برنامه‌ای که در تمام لحظات در حال اجراست

■ در واقع Kernel همان سیستم عامل است

CompYaar

شروع کار کامپیوتر

Bootstarp program: برنامه‌ای که هنگام روشن کردن یا ریست کامپیوتر بارگذاری می‌شود

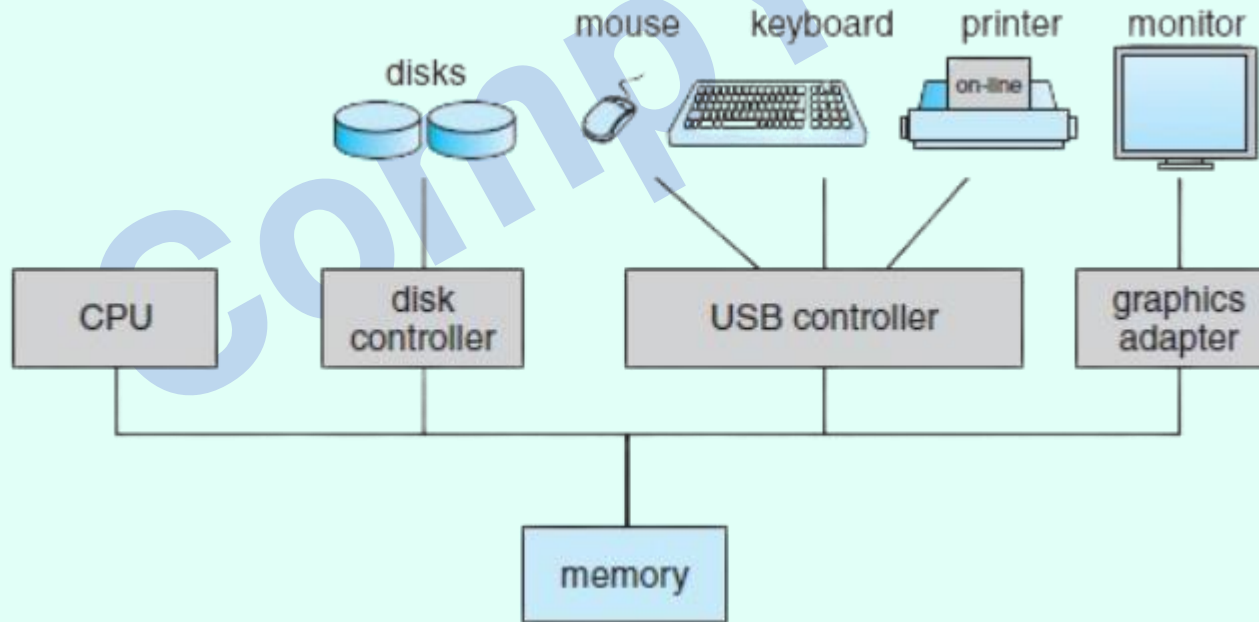
■ در ROM یا EPROM ذخیره می‌شود و معمولاً تحت عنوان firmware شناخته می‌شود

■ تمام ابعاد سیستم را آماده می‌کند

■ سیستم عامل (Kernel) را بارگذاری و اجرا را شروع می‌کند.

ساماندهی سیستم کامپیوتری

یک یا چند CPU و دستگاه‌های کنترلی از طریق یک bus به حافظه مشترک متصل می‌شوند



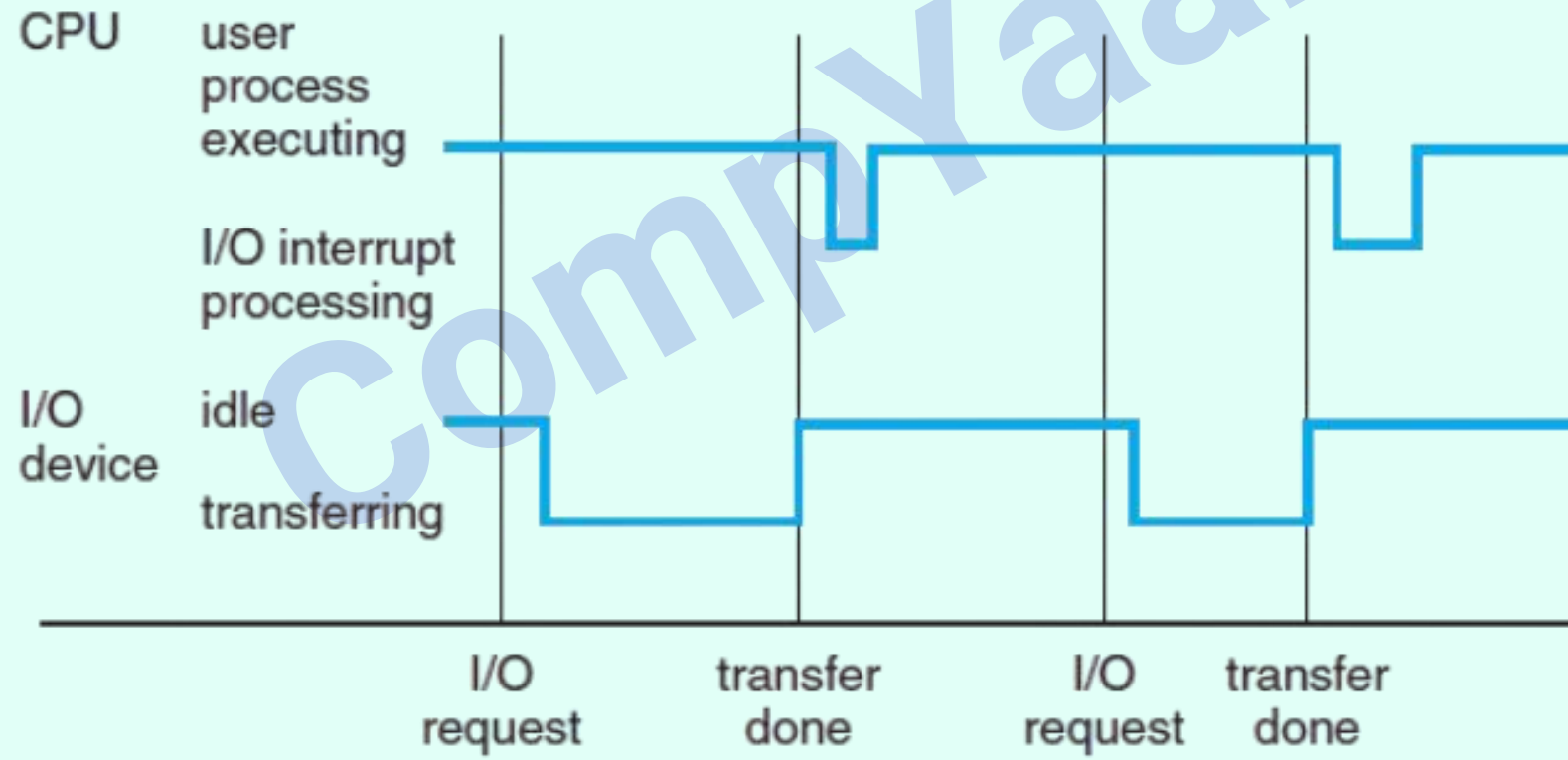
وقفه (Interrupt)

■ سیگنالی است که به CPU فرستاده می‌شود تا اجرای برنامه فعلی خود را متوقف کند و بلافاصله رویدادی مهم را بررسی کند.

■ وقوع یک رویداد توسط یک وقفه (Interrupt) نشان داده می‌شود که یا توسط سخت افزار است یا نرم افزار:
مثال سخت افزار: فشار دادن دکمه کیبورد، کلیک موس و ...
مثال نرم افزار: System Call و ...

■ سخت افزار یک وقفه را با ارسال سیگنالی به CPU فعال می‌کند و نرم افزار با اجرای عملیاتی خاص به نام System Call (Monitor Call) وقفه را فعال می‌کند.

وقفه (Interrupt)



Interrupt Handling

- در هنگام وقوع وقفه، سیستم عامل وضعیت CPU را با ذخیره کردن رجیسترها و Program Counter حفظ می‌کند
- دو روش کلی برای هندل کردن وقفه وجود دارد:

Polling

Vectored

Polling

■ CPU در هنگام وقوع وقفه، تمام دستگاه‌ها را چک می‌کند تا بفهمد کدام وقفه را ایجاد کرده است

■ غیر بهینه است، زیرا زمان زیادی صرف چک کردن تمام دستگاه‌ها می‌کند

■ ساده است

Vector Interrupts

■ وقفه باید کنترل را به interrupt service routine مناسب منتقل کند

■ در یک جدول اشاره‌گر به interrupt routine ها را نگه می‌داریم

interrupt vector: آرایه‌ای از آدرس service routine ها

■ استثنا (exception) یا تله (trap) یک وقفه است که به واسطه error یا درخواست کاربر توسط نرم‌افزار ایجاد می‌شود

ساختار حافظه

- کامپیوترها اکثر برنامه‌های خود را از یک حافظه قابل نوشتن اجرا می‌کنند (RAM)
- حافظه اصلی توسط یک فناوری نیمه هادی پیاده‌سازی شده است (DRAM)
- برنامه‌های ایستا (Static) مانند bootstrap در حافظه‌های غیرقابل تغییر (ROM) ذخیره می‌شوند

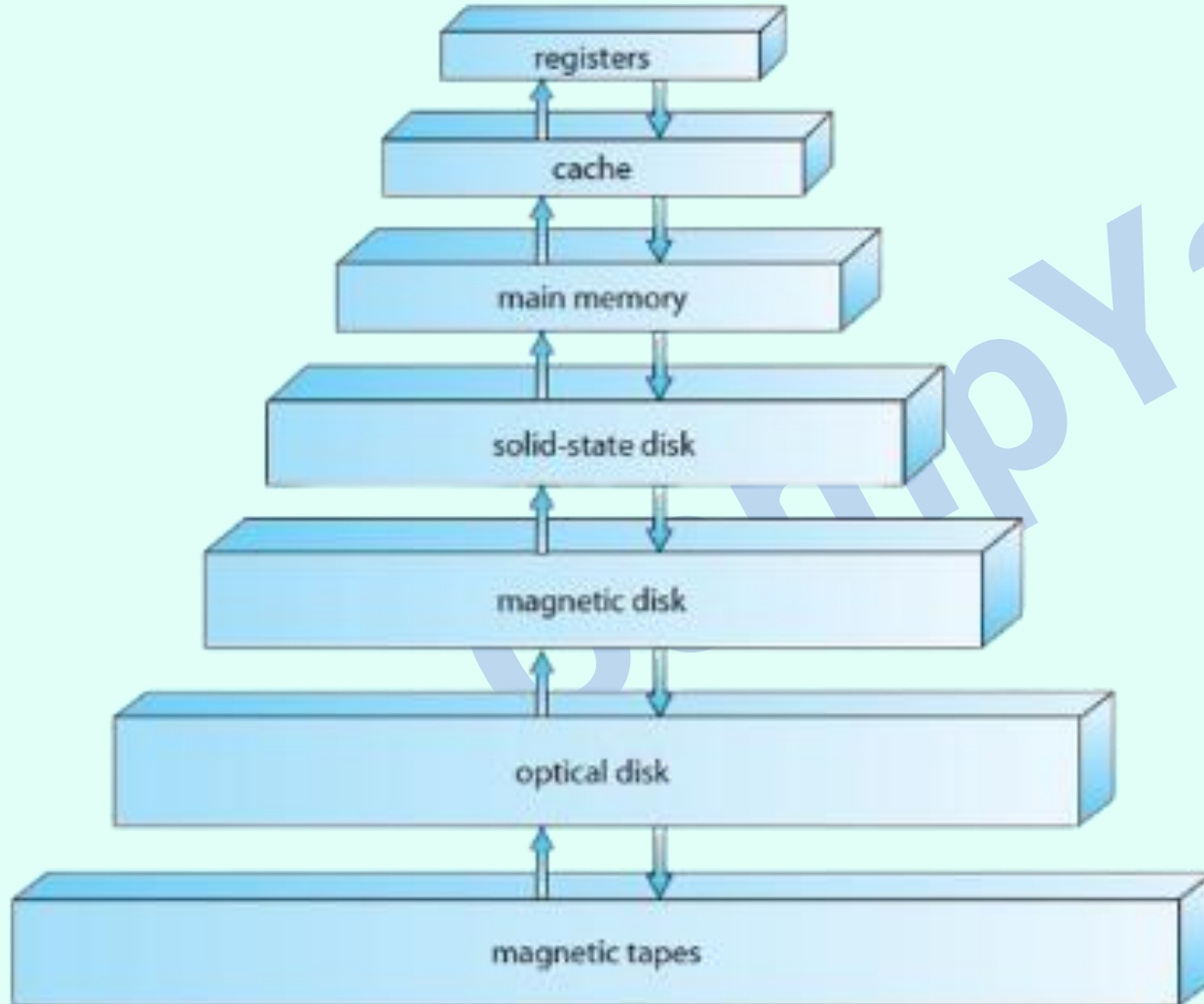
ساختار حافظه

- تمام حافظه‌ها آرایه‌ای از بایت‌ها هستند که هر بایت آدرسی دارد
- با دستورات load و store می‌توان بایت‌ها را بین حافظه اصلی و رجیسترها جابه‌جا کرد
- حافظه اصلی معمولا به قدری کوچک است که توانایی ذخیره‌سازی تمام برنامه‌ها را ندارد
- حافظه اصلی volatile است، یعنی در صورت خاموش شدن دستگاه، اطلاعات از دست می‌رود

ساختار حافظه

- به یک حافظه ثانویه جهت ذخیره‌سازی دائمی اطلاعات نیاز داریم
- معمول‌ترین نوع حافظه ثانویه دیسک مغناطیسی است
- دیسک‌های جامد (Solid-State Disk) نوع دیگری از حافظه هستند که از دیسک‌های مغناطیسی سریع‌تر هستند و non-volatile هم هستند

ساختار حافظه



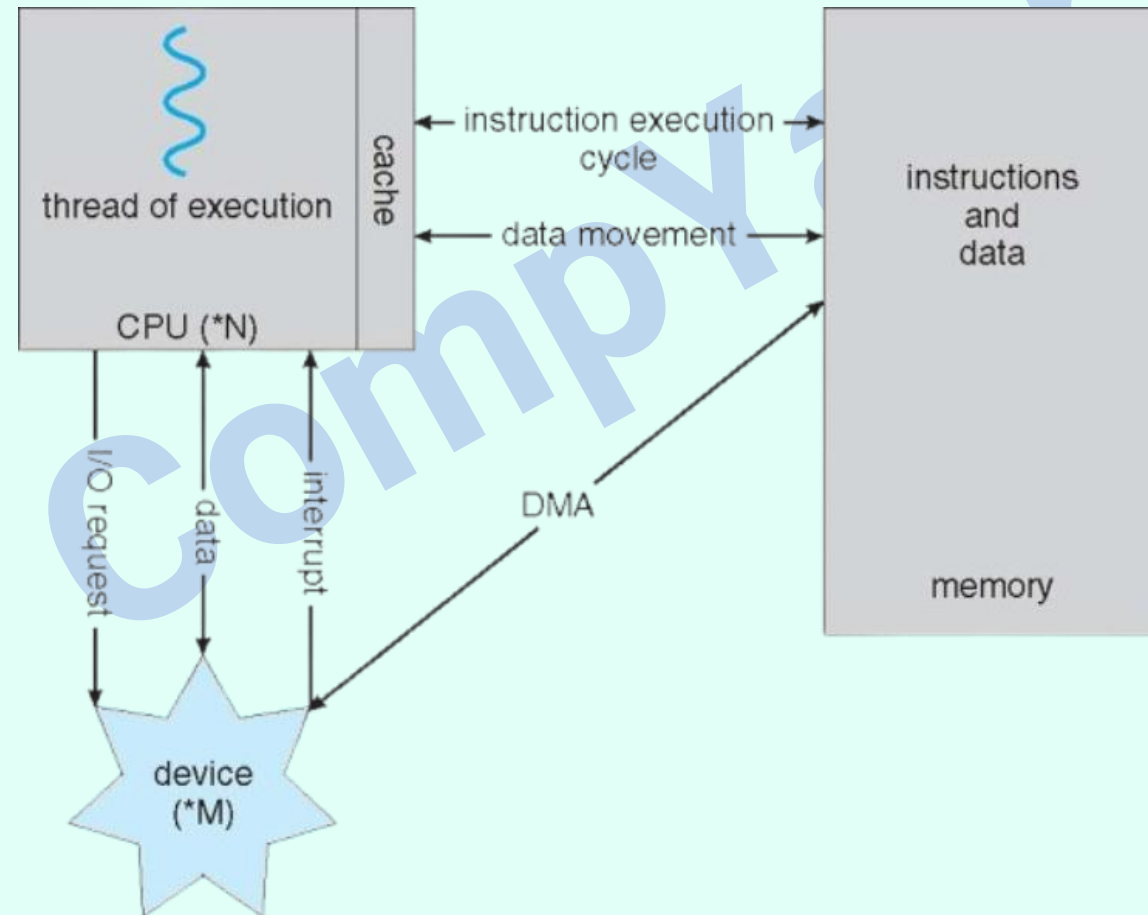
Caching

■ اصلی مهم که در لایه‌های مختلف کامپیوتر استفاده می‌شود

■ اطلاعات مورد استفاده از حافظه کند به حافظه سریع منتقل می‌شود

■ اگر اطلاعات مورد نیاز در cache بود از آن استفاده می‌کنیم، وگرنه آن را از حافظه کند به Cache منتقل می‌کنیم
و از آن استفاده می‌کنیم

How Modern Computer Works



معماری سیستم کامپیوتری

Multiprocessor: استفاده از چند CPU (Parallel)

افزایش throughput: در زمان کمتری کار بیشتری انجام می‌شود

کاهش هزینه: یک سیستم multiprocessor نسبت به چند سیستم single-processor هزینه کمتری دارد

افزایش reliability: با تقسیم کارها بین پردازنده‌ها، با از کارافتادن یک پردازنده، کل سیستم متوقف نمی‌شود

معماری سیستم کامپیوتری

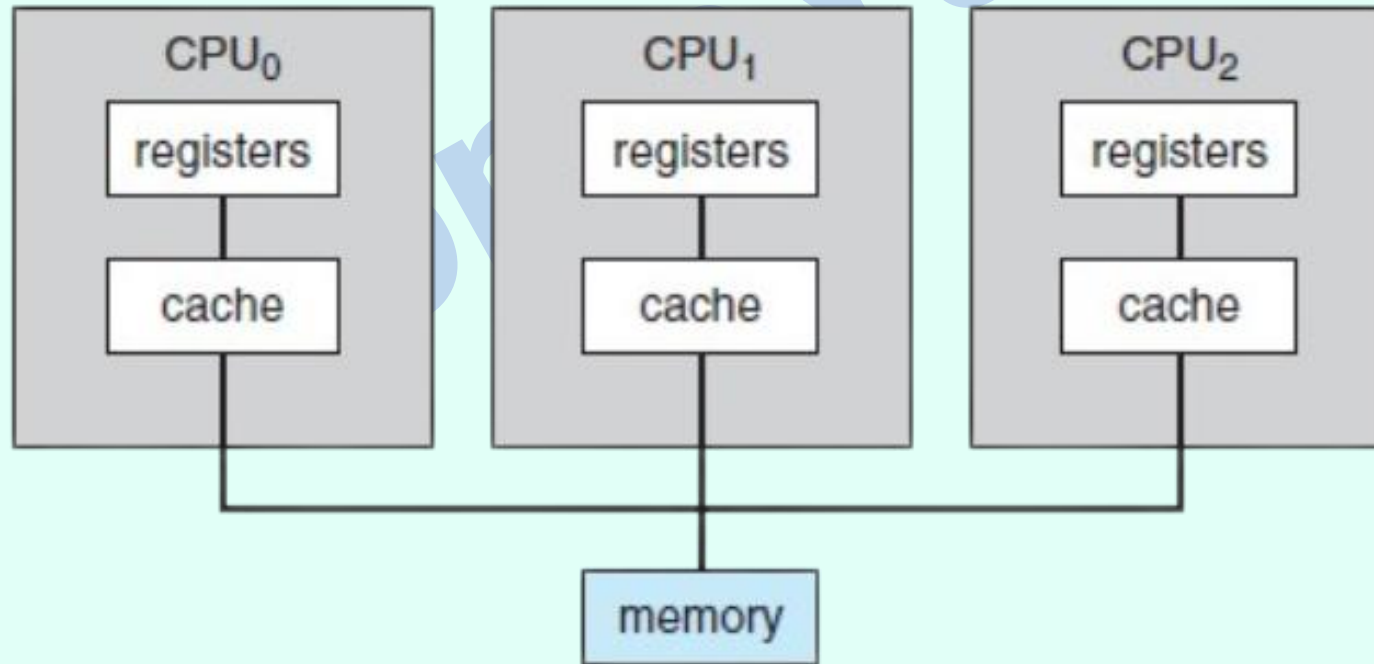
Multiprocessor: استفاده از چند CPU (Parallel)

Asymmetric multiprocessing: یک پردازنده master می‌شود و بقیه slave. master بین slave ها task های مختلف را تقسیم می‌کند

Symmetric multiprocessing: همه پردازنده ها منابعشان را به اشتراک می‌گذارند و task ای جدا انجام می‌دهند

معماری سیستم کامپیوتری

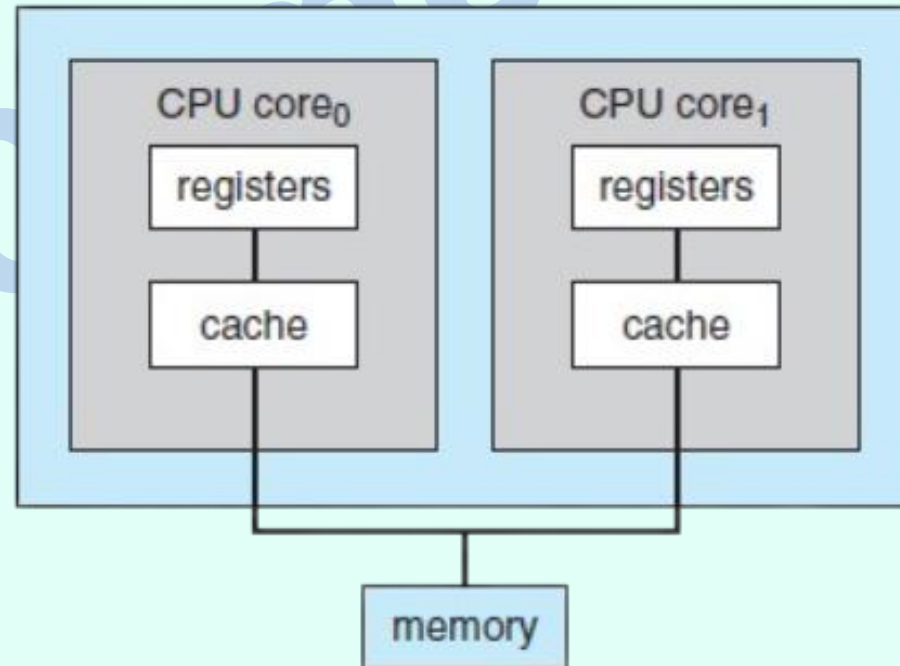
نمونه‌ای از SMA:



معماری سیستم کامپیوتری

Multicore: قرار دادن چندین core بر روی یک chip

کارایی بهتر، زیرا ارتباط بین چند core بر روی یک chip سریعتر است



Multiprogramming

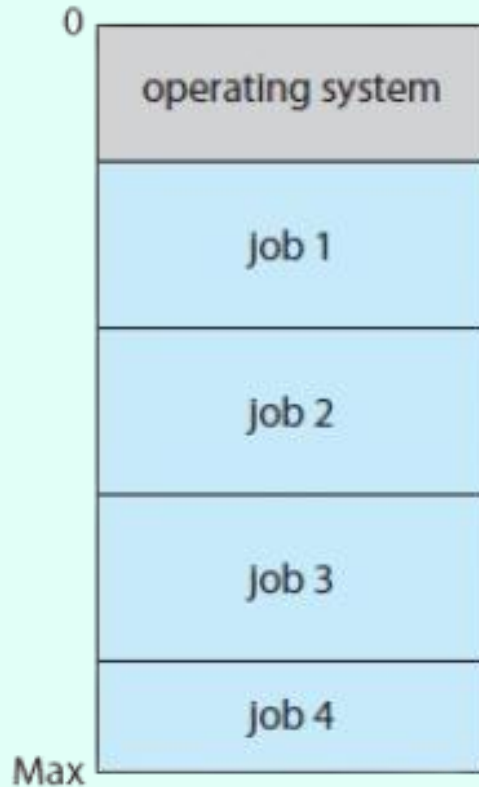
■ یک برنامه نمی‌تواند دائماً CPU و دستگاه‌های I/O را در اختیار داشته باشد. کاربر ممکن است همزمان چندین برنامه را اجرا کند

■ یک وکیل را در نظر بگیرید که چندین پرونده دارد. اگر یک پرونده به مرحله‌ای برسد که مثلاً نیاز باشد برای دادگاه صبر کنند، وکیل در این زمان به کارهای پرونده دیگری رسیدگی می‌کند

■ در یک سیستم multiprogramming، jobهای مختلفی از job pool در حافظه قرار دارند و CPU این jobها را انجام می‌دهد و اگر jobای مثلاً نیاز به زمان داشته باشد، CPU سراغ job دیگری می‌رود تا بیشترین بهره‌وری حاصل شود

Multiprogramming

حافظه در یک سیستم multiprogramming:



Timesharing (Multitasking)

■ CPU به سرعت بین jobهای مختلف تعویض انجام می‌دهد که کاربر می‌تواند حین اجرا هم با برنامه تعامل داشته باشد

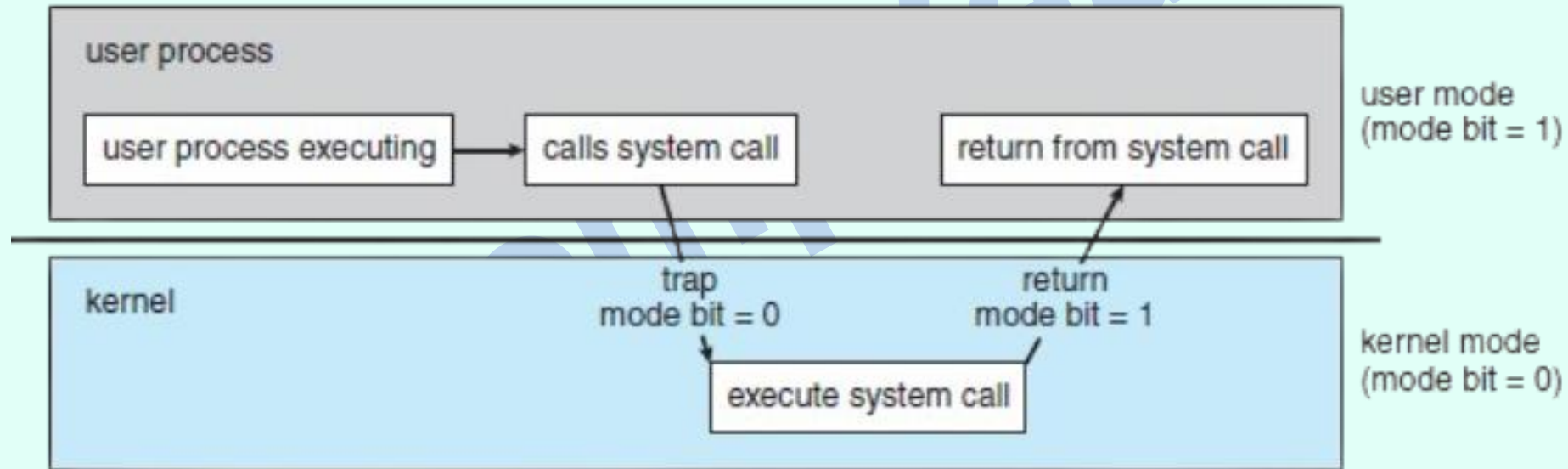
■ در این حالت response time باید مقدار کمی (مانند زیر 1 ثانیه) باشد

■ اگر چندین job آماده اجرا باشند، باید از طریق scheduling بین‌شان انتخاب کنیم

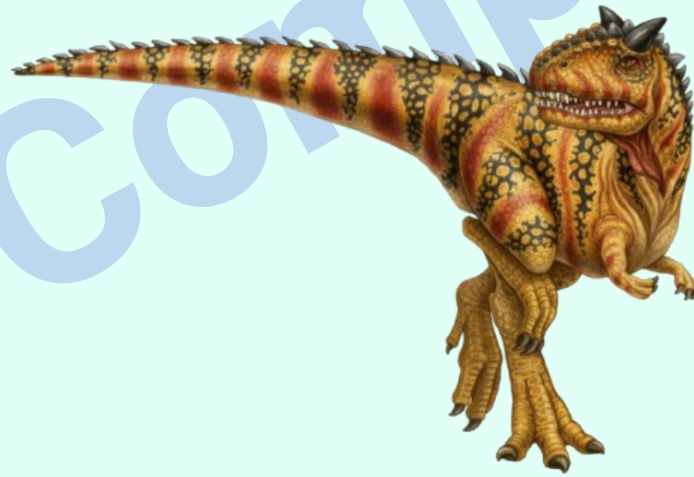
Dual Mode

- سیستم عامل در دو mode کار می‌کند، یکی برای اجرای کد سیستم عامل و دیگری اجرای کد کاربر
- به این دو حالت kernel mode و user mode گوئیم
- از bit mode برای تشخیص mode استفاده می‌کنیم

Dual Mode



Chapter 2: Operating-System Structures



سرویس‌های سیستم عامل (برای کاربر)

- 1. **رابط کاربری (User Interface):** می‌تواند به صورت command line (CLI) یا رابط گرافیکی (GUI) باشد
- 2. **اجرای برنامه (Program Execution):** سیستم باید بتواند برنامه را در حافظه لود و اجرا کند. همچنین باید بتواند اجرا را خاتمه دهد.
- 3. **عملیات‌های ورودی/خروجی (I/O Operations):** یک برنامه احتمالاً به I/O نیاز پیدا کند، که شامل یک فایل یا یک دستگاه I/O می‌شود
- 4. **کنترل فایل‌ها توسط برنامه (File-System Manipulation):** برنامه‌ها نیاز دارند تا بتوانند فایل‌ها را بخوانند یا در آن‌ها بنویسند، فایل ایجاد یا حذف کنند. در آن‌ها جستجو کنند و ...
- 5. **ارتباط بین فرآیندها (Communications):** فرآیندها به جهت تبادل داده، نیاز به ارتباط دارند. این ارتباط یا به صورت message passing است یا shared memory.
- 6. **تشخیص خطا (Error Detection):** سیستم عامل باید دائماً در جریان خطاهای سیستم باشد. خطا می‌تواند در CPU، حافظه، دستگاه‌های I/O یا برنامه کاربر باشد. برای هر نوع خطا، سیستم عامل باید اقدام مناسب انجام دهد.

سرویس‌های سیستم عامل (برای بهیئگی)

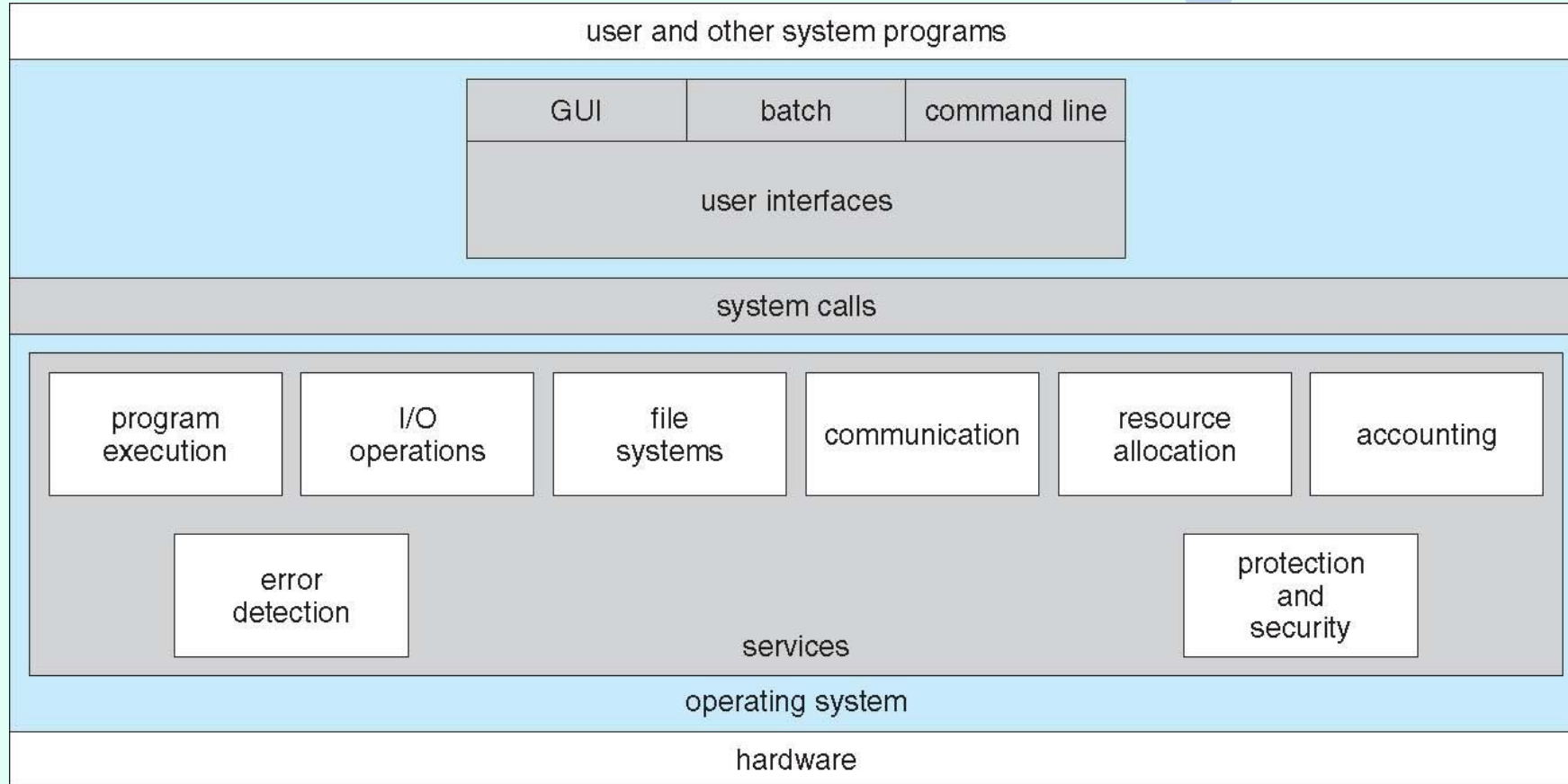
تخصیص منابع (Resource Allocation): سیستم عامل باید بتواند به طور بهینه منابع را بین کاربران و jobهای موازی تخصیص دهد. منظور از منابع چرخه‌های CPU، حافظه اصلی، دستگاه‌های I/O و ... است.

حسابرسی (Accounting): ردگیری اینکه هر کاربر چه مقدار و چه نوع منابعی را استفاده می‌کند

حفاظت (Protection): دسترسی به منابع سیستم تحت کنترل باشد.

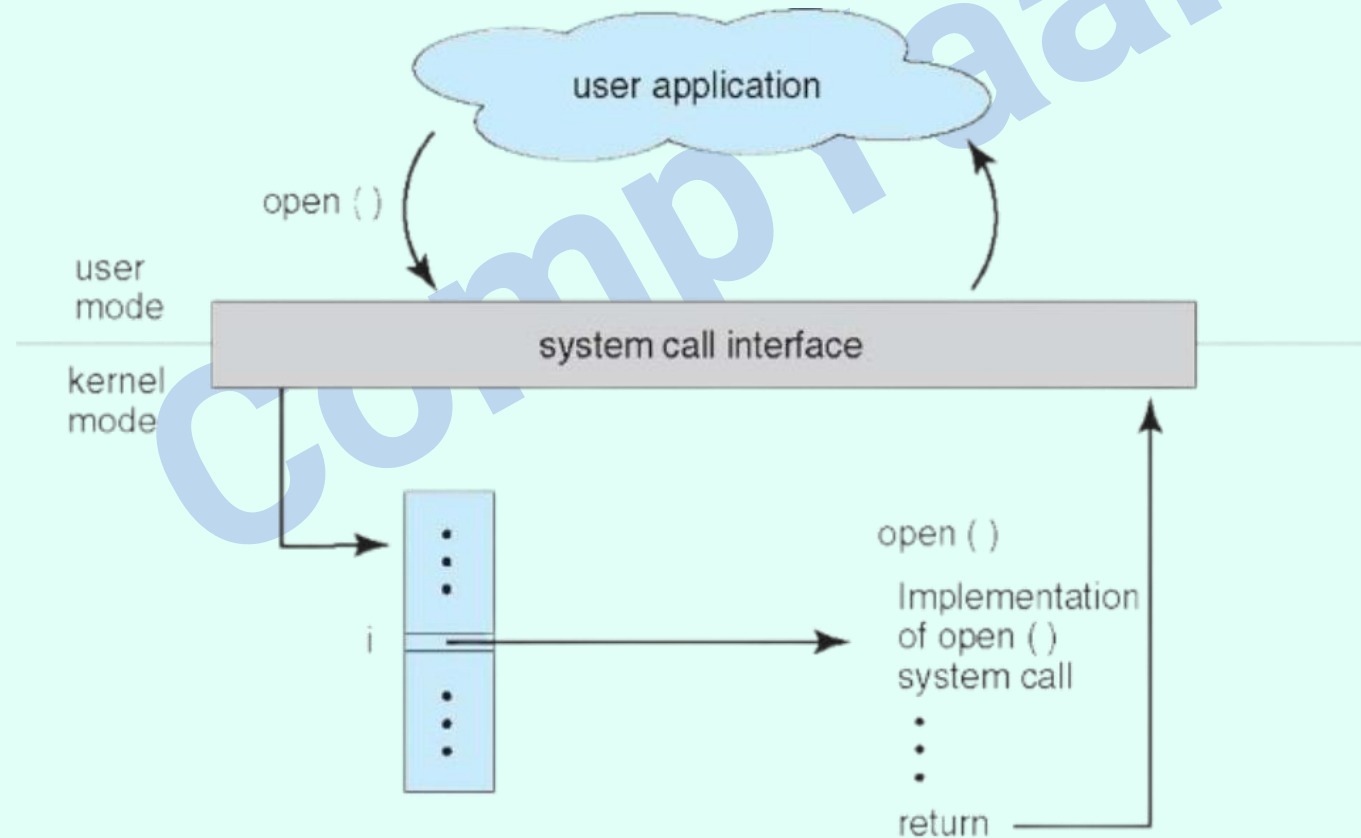
امنیت (Security): دسترسی به سیستم نیاز به احراز هویت (Authentication) داشته باشد.

سرویس‌های سیستم عامل



System Call

رابطی بین برنامه کاربر و Kernel



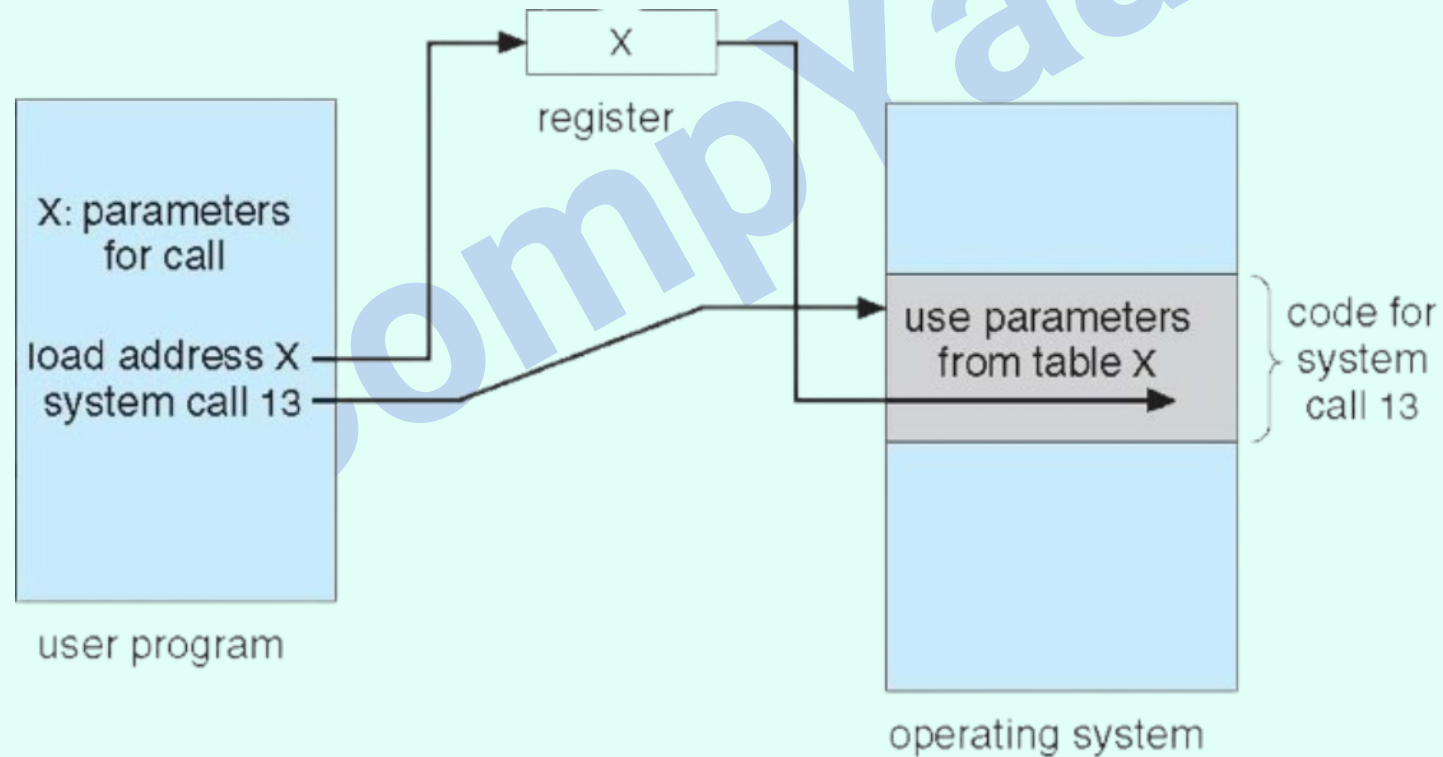
System Call: Parameter Passing

علاوه بر خود system call، به اطلاعات دیگری نیاز داریم، که این اطلاعات را در قالب پارامتر، به سیستم کال پاس می‌دهیم. سه روش parameter passing وجود دارد:

1. قرار دادن پارامترها در رجیستر و پاس دادن رجیستر. مشکل: ممکن است تعداد پارامترها از تعداد رجیسترهای سیستم بیشتر شود.
2. پارامترها را در یک بلوک، جدول یا حافظه قرار می‌دهیم، و آدرس آن‌ها را پاس می‌دهیم
3. پارامترها توسط برنامه در پشته push می‌شوند، و توسط سیستم عامل pop می‌شوند

System Call: Parameter Passing

Parameter passing با استفاده از جدول:



انواع System Call ها

■ Process Control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate memory, free memory
- dump memory if error
- debugger
- locks

انواع System Call ها

■ File Management

- create file, terminate file
- open file, close file
- read, write, reposition
- get file attributes, set file attributes

■ Device Management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

انواع System Call ها

■ Information Maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file or device attribute

■ Communications

- create communication connection, delete communication connection
- send message, receive message
- shared memory model create
- transfer status information
- attach and detach remote devices

انواع System Call ها

■ Protection

- control access to resources
- get permissions, set permissions
- allow user access, deny user access

Compyaar

طراحی و پیاده‌سازی سیستم عامل

■ ساختار داخلی سیستم عامل‌های متفاوت می‌تواند بسیار فرق داشته باشد.

■ شروع طراحی باید با تعریف اهداف همراه باشد.

■ نوع سخت افزار و سیستم، بر این اهداف تاثیر می‌گذارد.

■ اهداف کاربر و سیستم، متفاوت از هم هستند:

اهداف کاربر: سیستم عامل به سادگی قابل استفاده باشد، قابل اطمینان باشد، ایمن باشد، سریع باشد و ...

اهداف سیستم: به سادگی قابل طراحی و پیاده‌سازی باشد، منعطف باشد، خطا نداشته باشد، بهینه باشد و ...

طراحی و پیاده‌سازی سیستم عامل

■ توجه به دو اصل مهم سیاست و مکانیزم:

سیاست (Policy): چه کاری انجام خواهد شد؟

مکانیزم (Mechanism): چگونه انجام خواهد شد؟

■ تفاوت قائل شدن برای دو اصل بالا، باعث افزایش انعطاف (Flexibility) می‌شود

ساختارهای مختلف سیستم عامل

ساده (Simple)

پیچیده (Complex)

لایه‌ای (Layered)

ریزه‌سته (Microkernel)

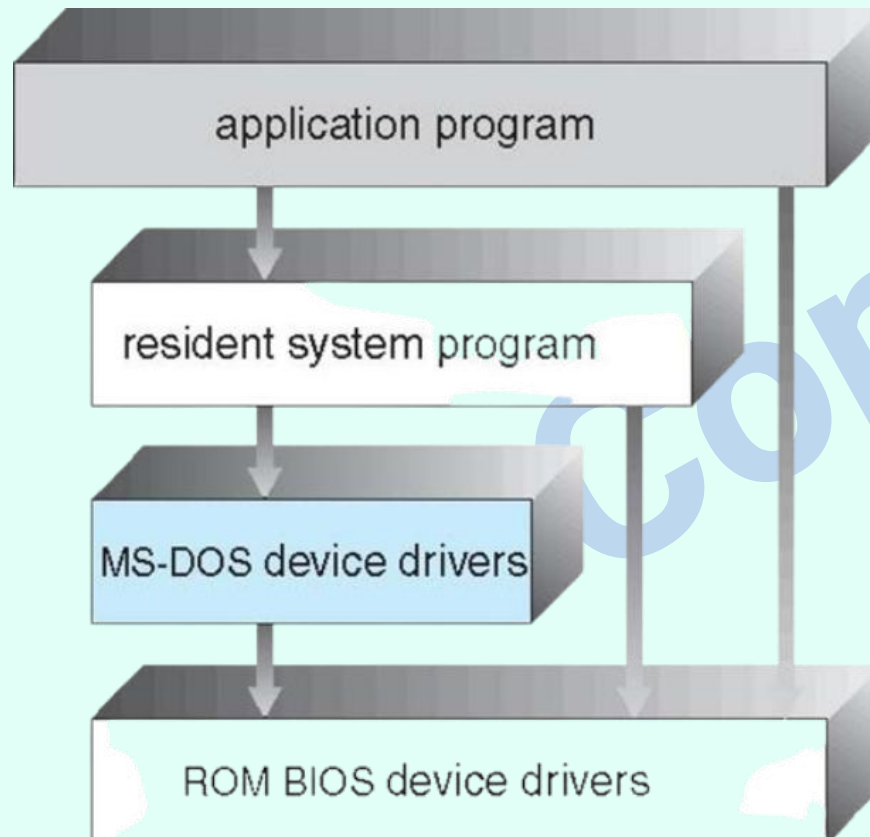
CompYaar

ساختار ساده: MS-DOS --

هدف: ارائه بیشترین کارکرد (functionality) در کمترین فضا

■ ماژولار نیست.

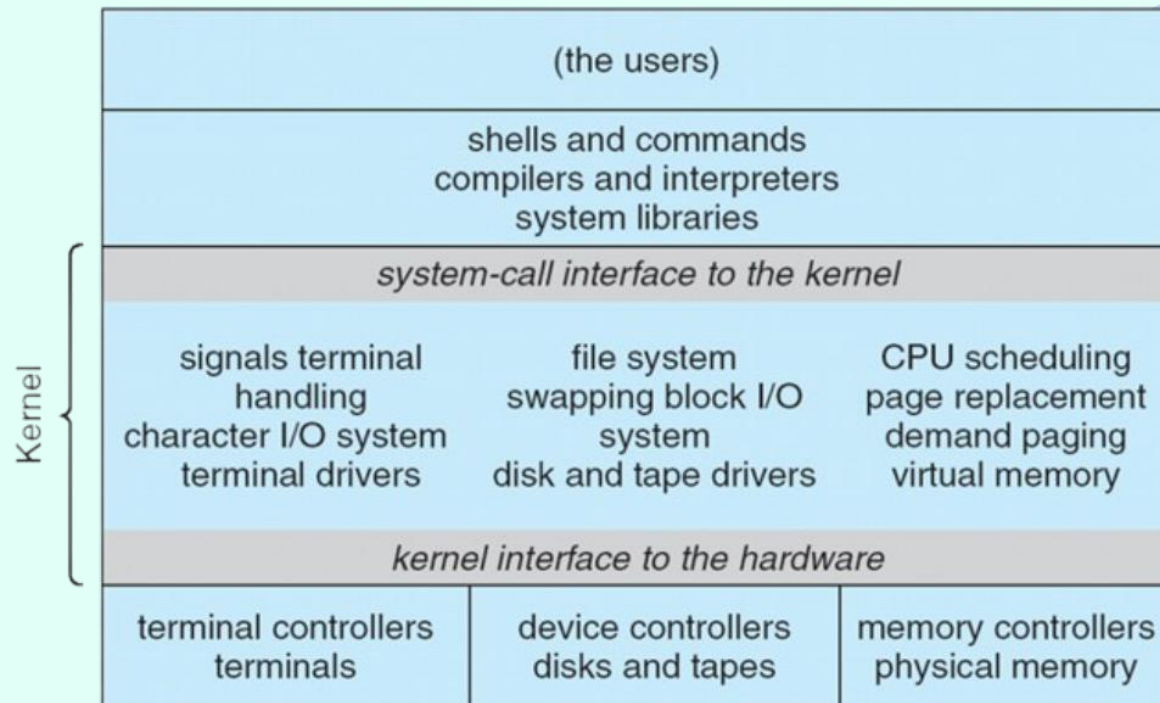
■ رابطها و سطوح کارکرد آن به خوبی جدا نشده‌اند.



ساختار پیچیده: UNIX--

■ به دلیل محدودیت‌های سخت افزاری، ساختاری نسبتاً ساده داشت، ولی پیچیده‌تر از MS-DOS.

■ شامل دو قسمت برنامه‌های سیستمی و Kernel است.

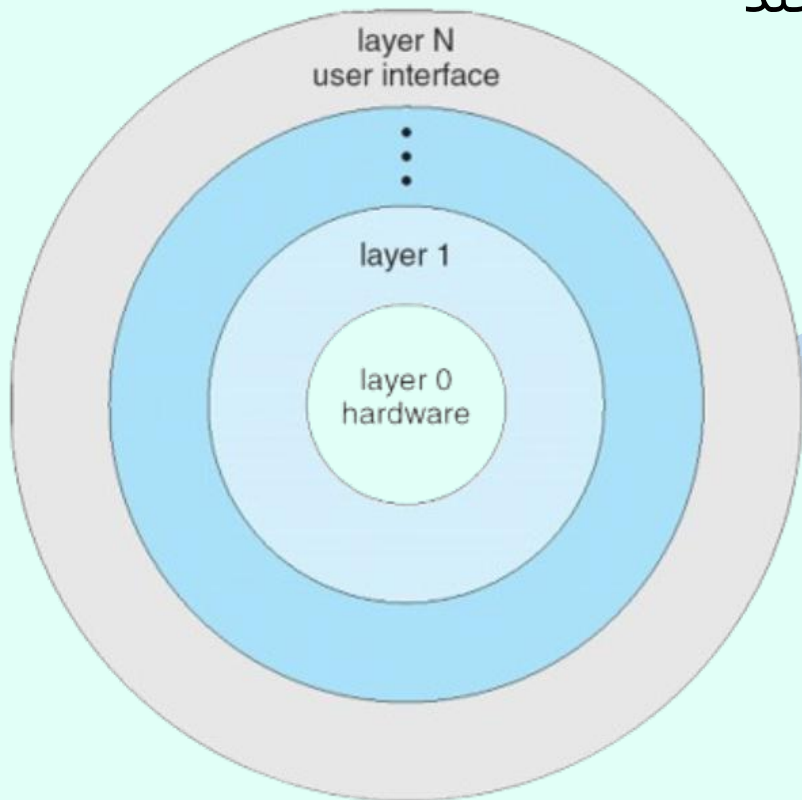


وظایف Kernel: مدیریت فایل، زمان‌بندی CPU،
مدیریت حافظه، مدیریت فرآیندها و ...

ساختار لایه‌ای

■ لایه صفر سخت افزار است و آخرین لایه، رابط کاربری است

■ هر لایه می‌تواند از توابع و سرویس‌های لایه‌های پایین‌تر از خود استفاده کند



ساختار ریزهسته: Mach

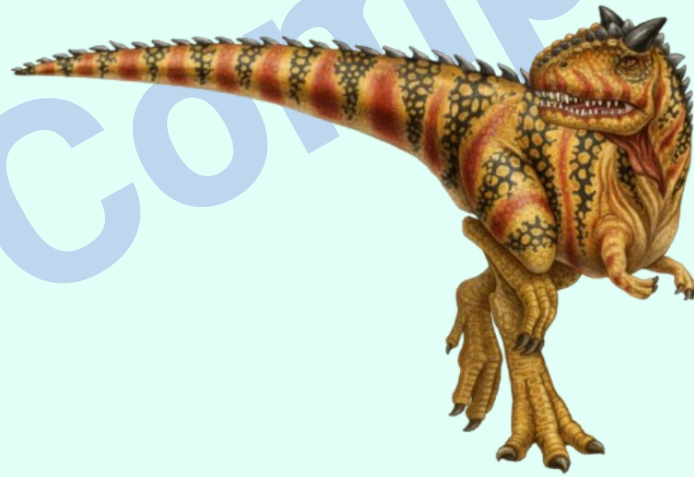
■ تلاش می‌شود کمترین مقدار کد در سطح kernel اجرا شود، و مابقی به فضای کاربر منتقل شود.

■ ارتباط بین فرآیندها در این ساختار، از طریق message passing است

مزایا: گسترش آن راحت است، انتقال به معماری جدید ساده است، مورد اطمینان است، ایمن است و ...

معایب: overhead ناشی از ارتباط بین فضای کاربر و فضای kernel بالا است

Chapter 3: Processes



فرآیند (Process)

فرآیند (Process): برنامه در حال اجرای در حافظه

قسمت‌های مختلف یک فرآیند:

قسمت متنی (text section): کد برنامه

program counter: نگهدارنده حالت فعلی برنامه

پشته (stack): شامل داده‌های موقت (متغیرهای محلی، پارامترهای توابع و ...)

قسمت داده (data section): شامل متغیرهای جهانی

هرم (heap): شامل حافظه پویا تخصیص داده شده حین اجرا

فرآیند (Process)

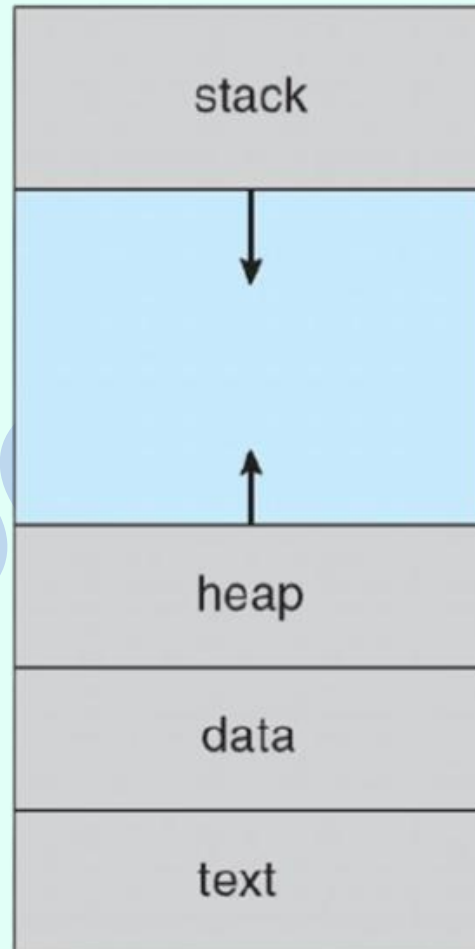
■ تفاوت برنامه (Program) و فرآیند (Process):
برنامه **passive** است (به صورت فایل اجرایی در دیسک ذخیره شده است)، فرآیند **active** است (همان برنامه، در حال اجرا)

■ یک برنامه تبدیل به فرآیند می‌شود، هرگاه فایل اجرایی آن در حافظه لود شود

■ یک برنامه می‌تواند چند فرآیند باشد.
(مثال) چند کاربر، یک برنامه را اجرا کنند.

فرآیند (Process)

■ فرآیند در حافظه:



حالت فرآیند (Process State)

■ یک فرآیند می‌تواند حالات مختلفی داشته باشد:

جدید (new): فرآیند ایجاد می‌شود

در حال اجرا (running): دستورات در حال اجرا شدن هستند

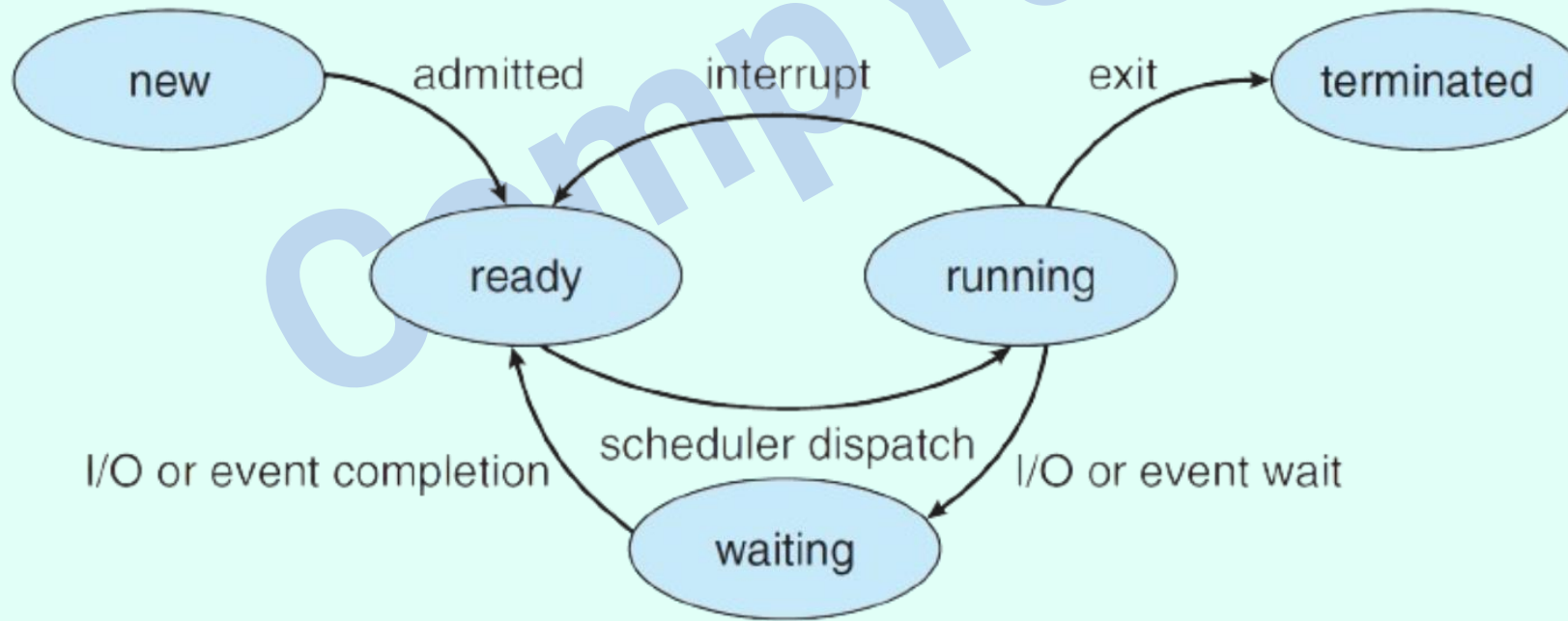
صبر کردن (waiting): فرآیند برای وقوع رویدادی صبر می‌کند

آماده (ready): فرآیند صبر می‌کند تا به یک پردازنده منتسب شود

پایان یافته (terminated): اجرای فرآیند تمام می‌شود

حالت فرآیند (Process State)

■ دیاگرام زیر برای حالات مختلف فرآیند است:



بلوک کنترل فرآیند (PCB)

بلوک کنترل فرآیند (Process Control Block): اطلاعات هر فرآیند در آن قرار دارد

■ اطلاعاتی که در PCB نگهداری می‌شود:

وضعیت فرآیند (Process State): در کدام یکی از وضعیت‌های اسلاید قبل قرار دارد

Program Counter: آدرس دستور بعدی جهت اجرا

رجیسترهای CPU: داده‌های درون رجیسترها هنگام اجرای فرآیند

اطلاعات زمان‌بندی CPU: اولویت‌ها، اشاره‌گرهای صف زمان‌بندی و ...

اطلاعات مدیریت حافظه: حافظه اختصاص داده شده به هر حافظه

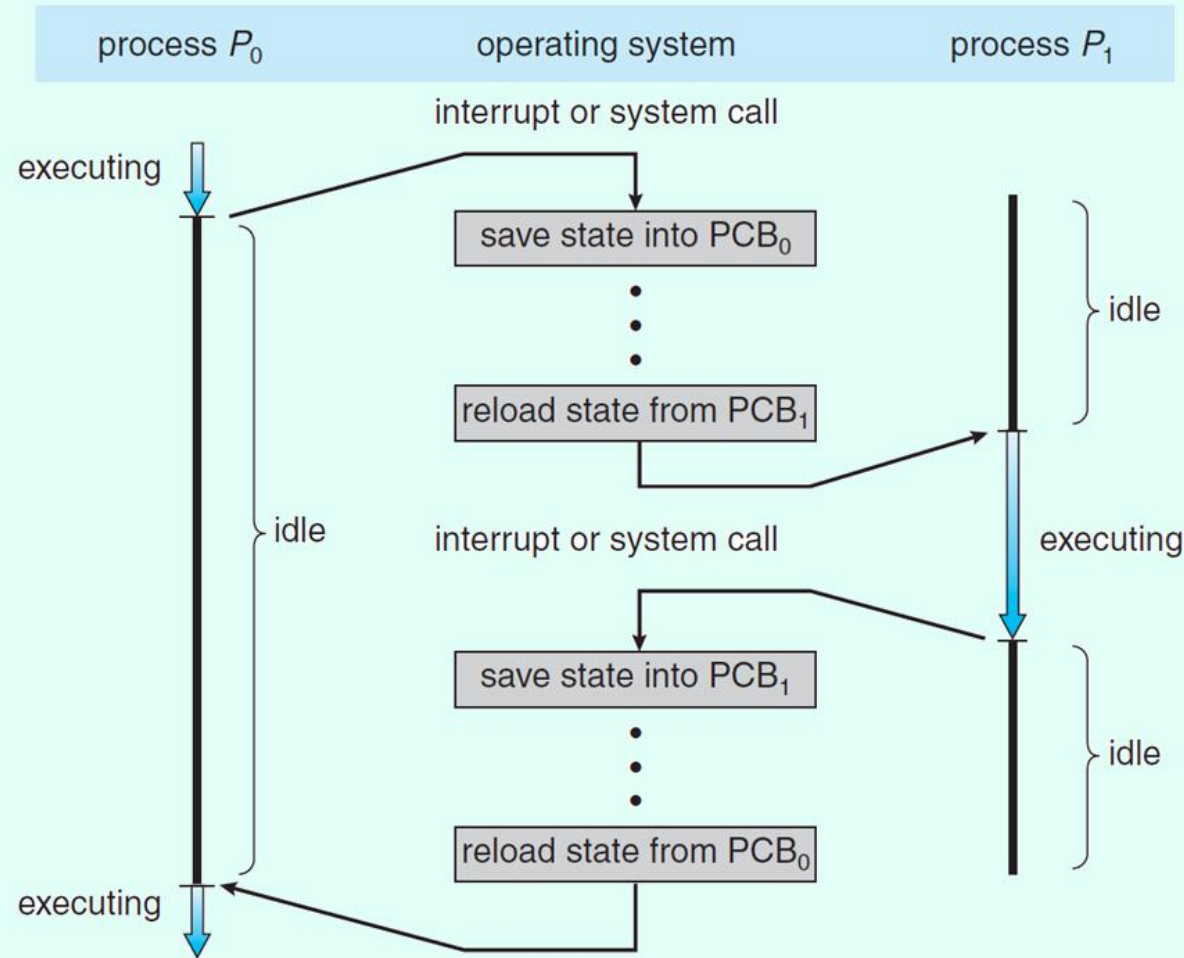
اطلاعات حسابرسی: میزان استفاده از CPU، اطلاعات مربوط به کلاک CPU و ...

اطلاعات وضعیت I/O: دستگاه‌های I/O اختصاص داده شده به هر فرآیند و ...

بلوک کنترول فرآیند (PCB)



جابه‌جایی بین چند فرآیند



نخ (Thread)

- فرآیندهایی که در این فصل بررسی می‌کنیم، فقط یک نخ دارند.
- اگر یک فرآیند چند program counter داشته باشد، در زمان چند دستور می‌توانند اجرا شوند ← چندنخی
- اطلاعات نخ‌ها باید در PCB قرار بگیرند
- در فصل بعدی به طور مفصل راجب نخ‌ها صحبت خواهیم کرد

زمان‌بندی فرآیندها (Process Scheduling)

زمان‌بند (Scheduler): از بین فرآیندهای موجود، یکی را جهت اجرا در CPU انتخاب می‌کند

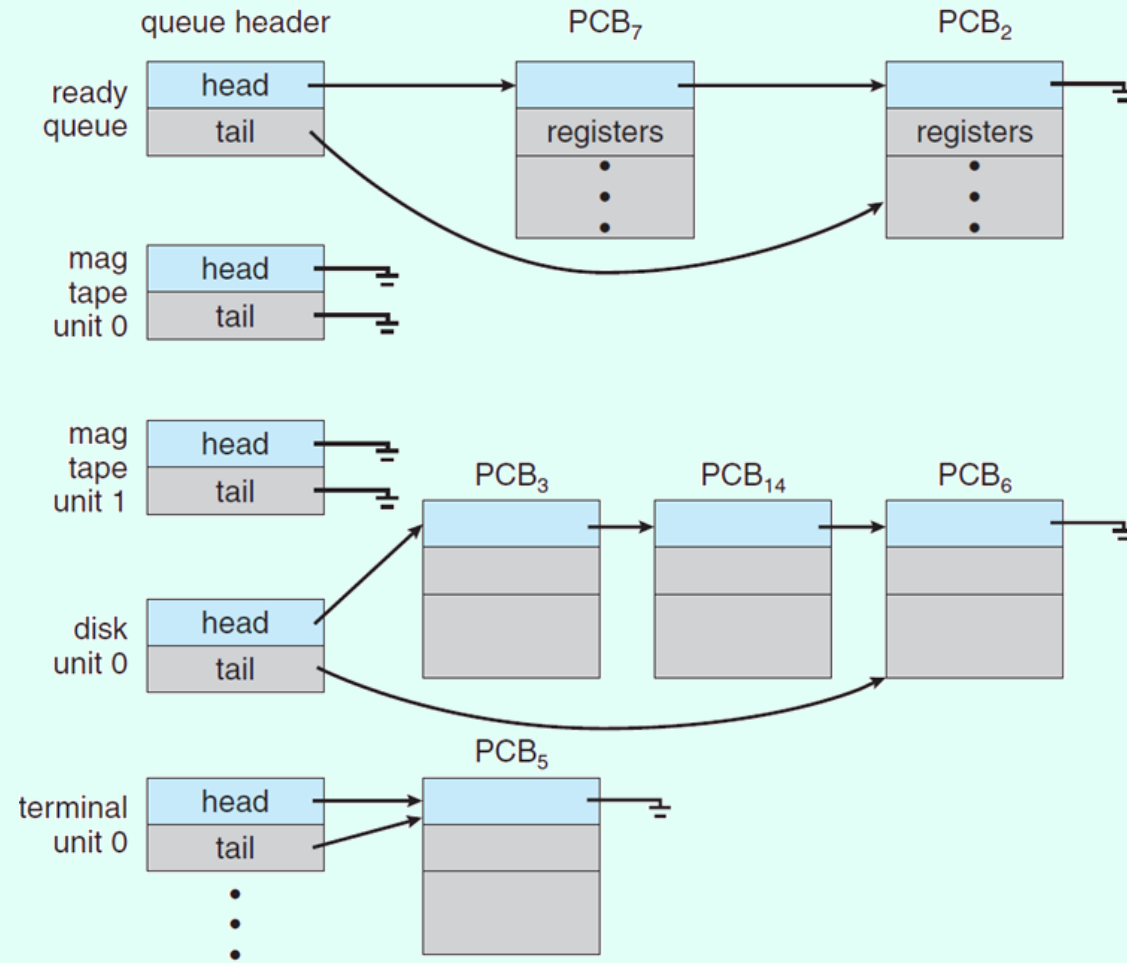
■ به جهت زمان‌بندی، چندین صف (queue) تشکیل می‌دهیم:

Job queue: مجموعه تمام فرآیندها در سیستم

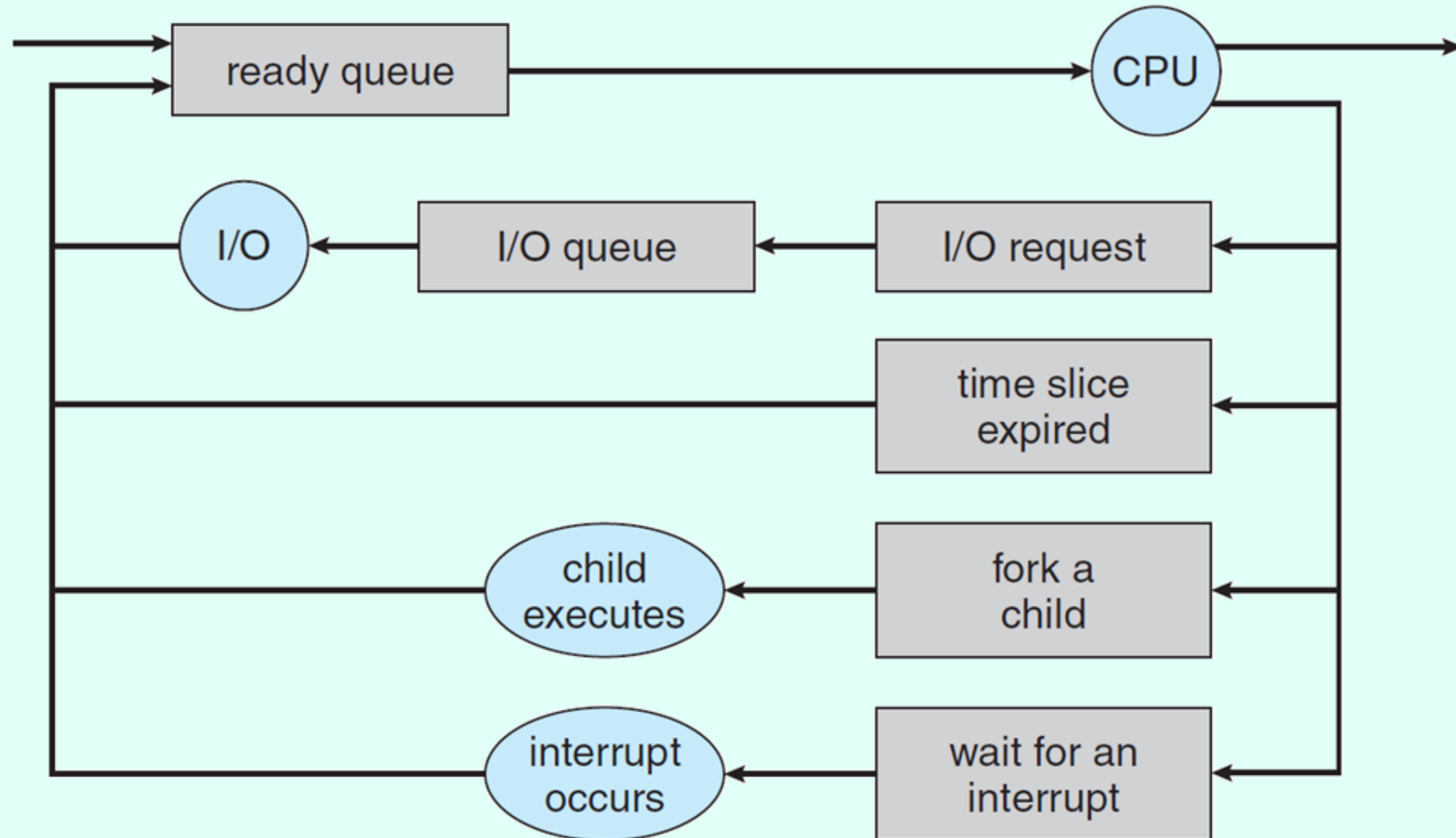
Ready queue: مجموعه تمام فرآیندهای درون حافظه، آماده و منتظر اجرا

Device queue: مجموعه تمام فرآیندهای منتظر دستگاه I/O

زمان بندی فرآیندها (Process Scheduling)



دیاگرام صفہا



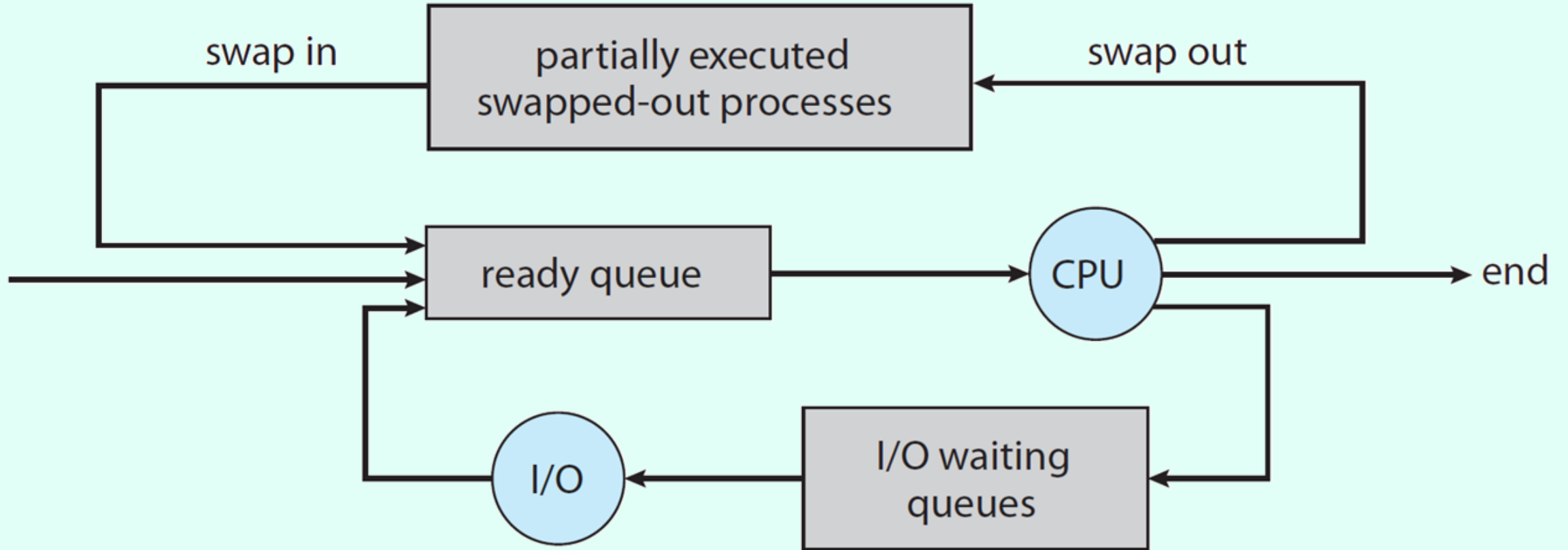
انواع زمان‌بندها

زمان‌بند کوتاه‌مدت (Short-term Scheduler): تعیین می‌کند کدام فرآیند باید اجرا شود و CPU به آن اختصاص داده شود. به آن **زمان‌بند پردازنده (CPU Scheduler)** هم گفته می‌شود. این زمان‌بند زیاد فعال می‌شود (هر چند میلی ثانیه یکبار، به همین جهت باید سریع باشد).

زمان‌بند بلندمدت (Long-term Scheduler): تعیین می‌کند کدام فرآیند باید به ready queue منتقل شود. این زمان‌بند کم فعال می‌شود (هر چند ثانیه یا هر چند دقیقه، به همین جهت ممکن است کند باشد). این زمان‌بند درجه multiprogramming را کنترل می‌کند.

زمان‌بند میان‌مدت (Medium-term Scheduler): اگر نیاز به کاهش درجه multiprogramming بود، از این زمان‌بند استفاده می‌شود. فرآیند را از حافظه به دیسک می‌برد، و بعد از مدتی از دیسک مجدداً به حافظه برگرداند.

زمان بند میان مدت



انواع زمان‌بندها

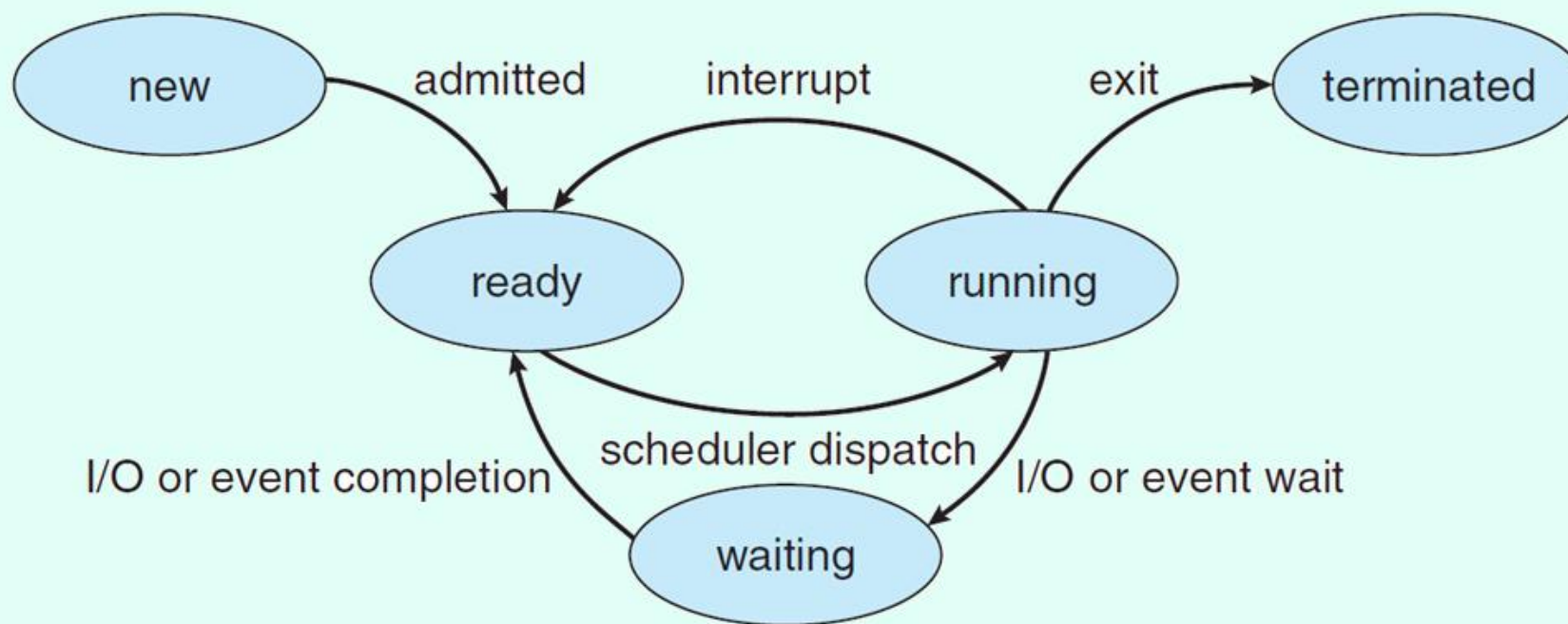
■ می‌توان فرآیندها را به دو گروه تقسیم کرد:

I/O-bound: زمان بیشتری صرف عملیات‌های I/O می‌کند تا CPU ← دوره‌های زیاد کوتاه مدت در CPU

CPU-bound: زمان بیشتری صرف محاسبات در CPU می‌کند ← دوره‌های کم بلند مدت در CPU

انواع زمان‌بندها

مثال) هر کدام از زمان‌بندهای کوتاه‌مدت، بلندمدت و میان‌مدت در چه انتقال‌هایی در دیاگرام زیر فعال می‌شوند؟



عملیات‌های فرآیندها

■ سیستم باید مکانیزمی برای عملیات‌های زیر برای فرآیندها فراهم کند:

ایجاد فرآیند (Process Creation)

پایان فرآیند (Process Termination)

ایجاد فرآیند (Process Creation)

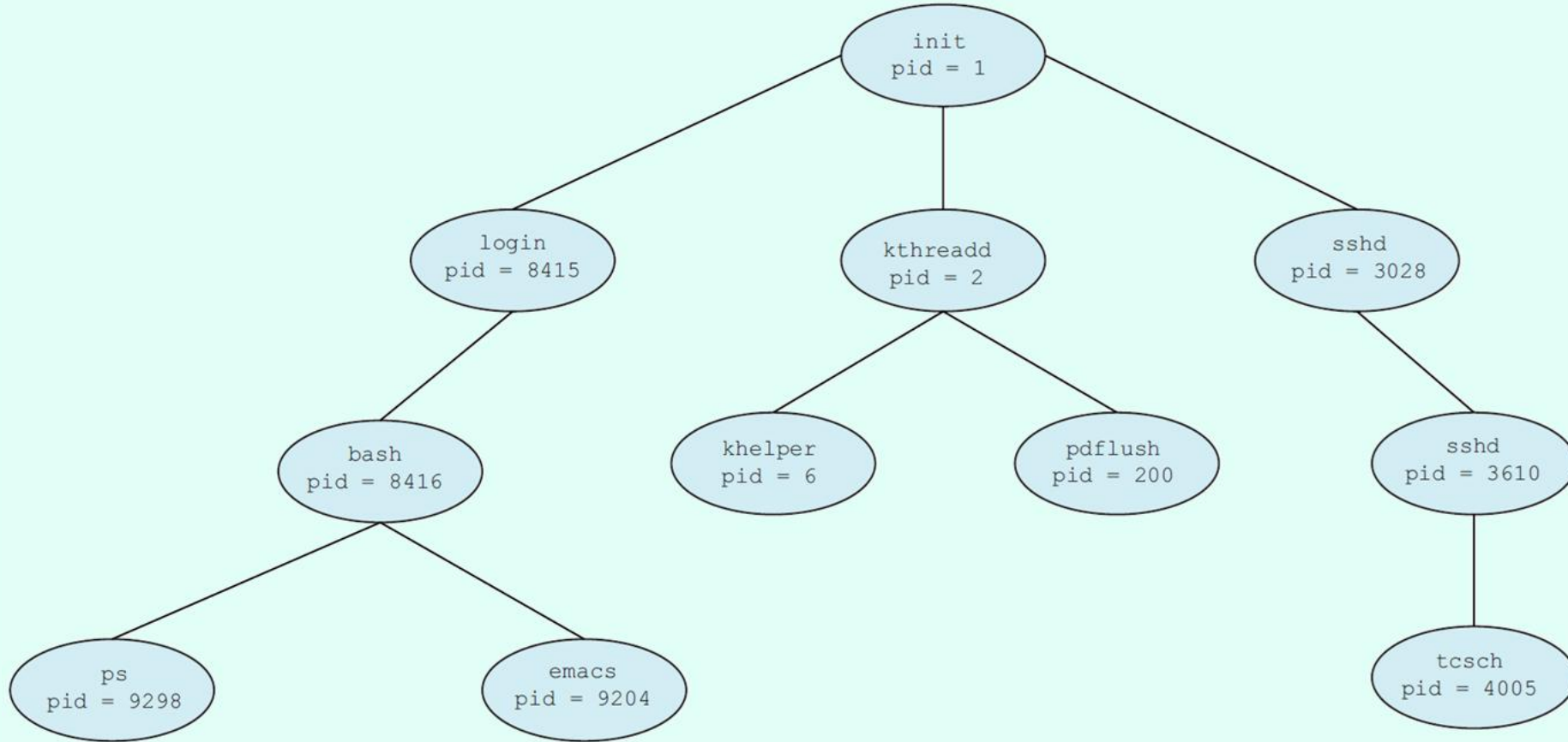
■ فرآیند پدر (Parent) می‌تواند فرآیندهای فرزند (Child) را ایجاد کند، که در نهایت باعث ایجاد درخت فرآیندها (Process Tree) می‌شود

■ فرآیندها توسط شناسه فرآیند (process identifier = pid) شناسایی و مدیریت می‌شوند.

■ پدر می‌تواند تمام منابع خود را در اختیار فرزندان بگذارد، یا قسمتی را به اشتراک بگذارد و یا هیچ منبعی را در اختیار فرزندان خود قرار ندهد

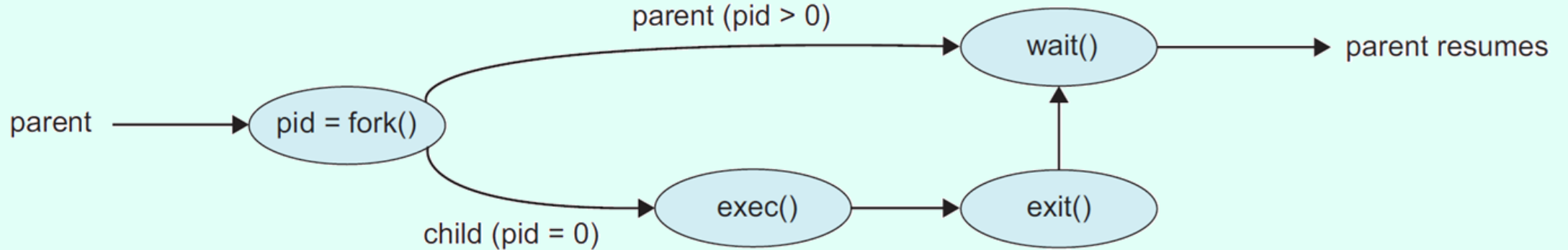
■ پدر و فرزندان می‌توانند به صورت concurrent اجرا شوند یا پدر تا زمان پایان اجرای فرزندان صبر کند

ایجاد فرآیند (Process Creation)



ایجاد فرآیند (Process Creation)

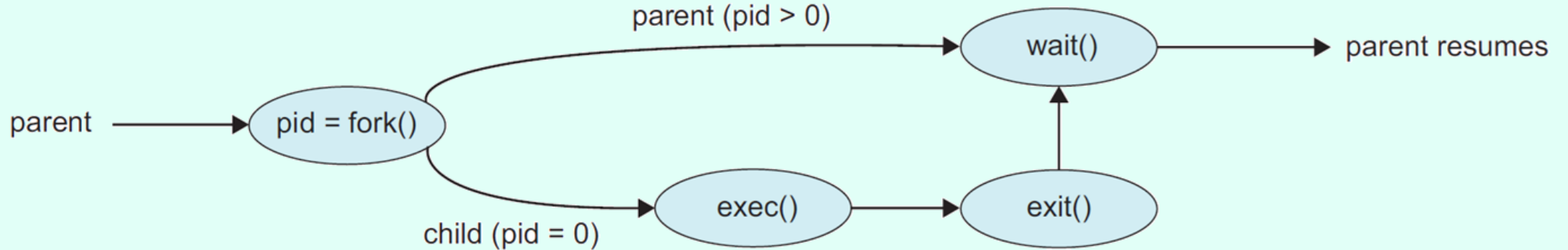
■ ایجاد فرآیند در یک سیستم UNIX:



* در فرآیند والد، فرزند یک pid دارد که توسط سیستم تعیین می‌شود، ولی در فرآیند فرزند، pid برابر صفر است.

ایجاد فرآیند (Process Creation)

■ ایجاد فرآیند در یک سیستم UNIX:



* در فرآیند والد، فرزند یک pid دارد که توسط سیستم تعیین می‌شود، ولی در فرآیند فرزند، pid برابر صفر است.

پایان فرآیند (Process Termination)

■ فرآیند بعد از پایان اجرای خود، از سیستم عامل درخواست حذف شدن می‌کند. این عملیات با سیستم کال `exit()` انجام می‌شود.

■ داده‌های وضعیت فرزند از طریق `wait()` به پدر منتقل می‌شود، و منابع اختصاص داده شده به فرآیند فرزند آزاد می‌شود.

■ پدر می‌تواند با سیستم کال `abort()` اجرای فرزند خود را پایان دهد. این کار به دلایل زیر ممکن است اتفاق بیفتد:

- فرزند از منابع اختصاص داده شده به خود فراتر رود
- وظیفه منتسب به فرزند دیگر مورد نیاز نباشد
- کار پدر تمام شده و سیستم عامل نمی‌خواهد فرآیندی بدون پدرش در سیستم باشد

پایان فرآیند (Process Termination)

■ همانطور که اشاره شد برخی سیستم عامل‌ها اجازه نمی‌دهند فرآیندی بدون حضور پدرش ادامه داشته باشد. به همین جهت از مکانیزمی استفاده می‌کنند که در صورت پایان فرآیند پدر، تمام فرزندان آن نیز کارشان تمام شود. به این مکانیزم **cascading termination** گوییم.

■ ممکن است فرآیند پدر تا زمان پایان اجرای فرزندان خود صبر کند:

```
pid = wait(&status);
```

zombie: فرآیندی که اجرای آن تمام شده، ولی پدرش `wait()` را برای او فراخوانی نکرده و اطلاعاتش همچنان در PCB قرار دارد.

Orphan: فرآیندی که پدرش بدون فراخوانی `wait()` کارش تمام شده است.

پایان فرآیند (Process Termination)

مثال) قطعه کد زیر را در یک سیستم UNIX based در نظر بگیرید. سیستم به صورت انحصاری است و تا زمان sleep() فرآیندها، در حال اجرا روی پردازنده باقی می‌مانند. pid شناسه فرآیند و ppid شناسه پدر فرآیند است. در حالت اولیه یک فرآیند با pid=8001 در سیستم وجود دارد. با فرض استفاده از cascading termination، خروجی کد چه خواهد بود؟ (pid فرآیند جدید، یک واحد از pid پدر خود بیشتر است)

```
int main(){
    pid_t = fork();
    if(pid==0){
        printf("ppid: %d\n", getppid());
        sleep(2);
        printf("ppid: %d\n", getppid());
    } else{
        printf("pid: %d\n", pid);
        sleep(1);
    }
    return 0;
}
```

ارتباط فرآیندها (Interprocess Communication)

■ یک فرآیند یا مستقل (independent) است، یا همکار (cooperating).

■ فرآیند همکار یا بر فرآیندهای دیگر تاثیر می‌گذارد، یا از فرآیندهای دیگر تاثیر می‌پذیرد.

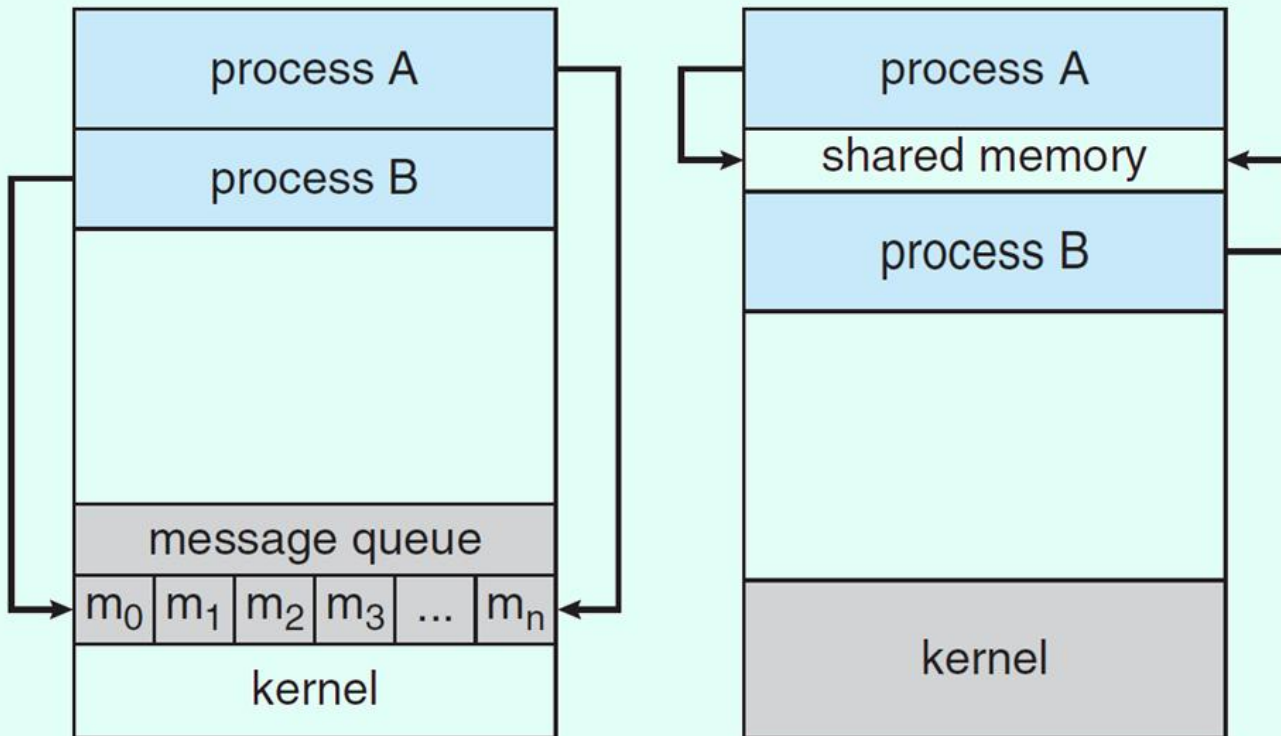
■ دلایل همکاری فرآیندها:

- به اشتراک گذاری داده
- افزایش سرعت محاسبات
- ماژولاریتی (modularity)
- سادگی (convenience)

ارتباط فرآیندها (Interprocess Communication)

■ برای همکاری فرآیندها، دو روش کلی وجود دارد:

حافظه مشترک (Shared Memory)
ارسال پیام (Message Passing)



حافظه مشترک (Shared Memory)

■ برای توضیح ارتباط فرآیندها از طریق حافظه مشترک، از مسئله‌های کلاسیک استفاده می‌شود، که ساده‌ترین آن‌ها، مسئله تولیدکننده-مصرف‌کننده (Producer-Consumer) است.

■ در این مسئله، یک بافر داریم که می‌تواند محدود یا نامحدود باشد. یک فرآیند نقش تولیدکننده را دارد که داده تولید و در بافر قرار می‌دهد و یک فرآیند نقش مصرف‌کننده را دارد و داده‌های درون بافر را مصرف می‌کند.

مسئله تولیدکننده-مصرفکننده

فرض می‌کنیم بافری خطی و محدود با 10 عنصر در اختیار داریم:

```
#define BUFFER_SIZE 10
typedef struct{
...
}item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

مسئله تولیدکننده-مصرفکننده

مصرفکننده و تولیدکننده به صورت زیر عمل می کنند:

```
item next_produced;
while(true){
    while((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

```
item next_consumed;
while(true){
    while(in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
}
```

* همگام سازی (synchronization) در این نوع مسائل بسیار مهم است که در فصل 5 با این مبحث آشنا می شویم.

ارسال پیام (Message Passing)

■ در این روش دیگر دو فرآیند حافظه‌ای را به اشتراک نمی‌گذارند، بلکه سیستم با ارائه دو عملیات زیر

`send(message)`

`receive(message)`

شرایط رد و بدل شدن پیام را بین دو فرآیند برقرار می‌کند.

■ برای این ارتباط، نیاز به یک لینک ارتباطی داریم که می‌تواند فیزیکی یا منطقی باشد. نمونه فیزیکی حافظه مشترک، شبکه و ... است. نمونه منطقی آن مستقیم یا غیرمستقیم، سنکرون یا آسنکرون و خودکار یا بافرینگ صریح است.

ارتباط مستقیم

■ در این روش فرآیندها اسم همدگیر را به صورت صریح استفاده می‌کنند:
send(P, message) : پیام message را به فرآیند P ارسال کن
receive(Q, message) : پیام message را از فرآیند Q دریافت کن

- ویژگی‌های لینک ارتباطی در این روش:
- لینک‌ها به صورت خودکار ایجاد می‌شوند
 - هر لینک دقیقاً بین یک جفت فرآیند برقرار می‌شود
 - بین هر جفت فرآیند دقیقاً یک لینک وجود دارد
 - لینک می‌تواند یکطرفه یا دوطرفه باشد، و معمولاً دوطرفه است

ارتباط غیرمستقیم

- در این روش، پیام‌ها به mailbox ارسال و از آن دریافت می‌شوند. هر mailbox یک شناسه منحصر به فرد دارد. فرآیندها برای ارتباط باید یک mailbox با یکدیگر به اشتراک بگذارند.
- ویژگی‌های لینک ارتباطی در این روش:
 - لینک ارتباطی زمانی ایجاد می‌شود که فرآیندها یک mailbox به اشتراک بگذارند
 - هر لینک می‌تواند چند فرآیند را به هم متصل کند
 - بین هر جفت فرآیند چندین لینک ممکن است برقرار شود
 - لینک می‌تواند یکطرفه یا دوطرفه باشد
- عملیات‌ها در این روش:
 - ساخت mailbox
 - ارسال پیام و دریافت پیام از طریق mailbox
 - نابود کردن mailbox

ارتباط غیرمستقیم

■ عملیات‌های زیر برای ارسال و دریافت پیام در روش ارتباط غیرمستقیم است:

send(A, message) : پیام message را به mailbox با نام A ارسال کن

receive(A, message) : پیام message را از mailbox با نام A دریافت کن

■ فرض کنید سه فرآیند P1، P2 و P3 یک mailbox به اشتراک می‌گذارند و فرآیند P1 یک پیام ارسال می‌کند و جفت فرآیندهای P2 و P3 عملیات receive را انجام می‌دهند. کدام فرآیند باید این پیام را دریافت کند؟

برای حل این مشکل، می‌توان اقدامات زیر را انجام داد:

- سیستم فقط اجازه ایجاد لینک بین دو فرآیند را دهد
- سیستم فقط اجازه فراخوانی receive توسط یک فرآیند در یک زمان را دهد
- سیستم اجازه انتخاب دریافت کننده را بدهد و فرستنده مطلع شود کدام فرآیند پیام را دریافت می‌کند

ارتباط سنکرون و آسنکرون

■ ارتباط از طریق ارسال پیام می‌تواند blocking و سنکرون (synchronous) باشد، یا non-blocking و آسنکرون (asynchronous) باشد.

Blocking (Synchronous): فرستنده تا زمانی که پیامش دریافت نشود بلاک می‌شود و گیرنده تا زمانی که پیامی در دسترس نباشد بلاک می‌شود.

Non-Blocking (Asynchronous): محدودیت عملی برای فرستنده و گیرنده وجود ندارد.

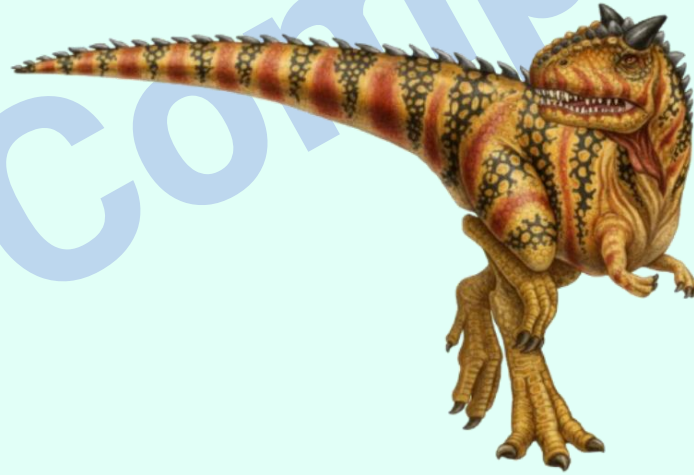
■ می‌توان ترکیبات مختلف ارسال و دریافت را با توجه به blocking و non-blocking بودن داشت. اگر هم ارسال و هم دریافت blocking باشند، یک **قرار ملاقات (rendezvous)** بین فرستنده و گیرنده داریم.

ارتباط سنکرون و آسنکرون

■ صف پیام‌ها که متصل به لینک است را می‌توان به یکی از سه روش زیر طراحی کرد:

1. **حجم صفر:** هیچ پیامی در صف روی لینک قرار نمی‌گیرد. فرستنده باید منتظر گیرنده باشد (rendezvous)
2. **حجم محدود:** طول محدودی مانند n برای صف در نظر گرفته می‌شود و در صورت پر شدن آن، فرستنده باید برای گیرنده صبر کند.
3. **حجم نامحدود:** هیچ محدودیتی برای تعداد ارسال پیام وجود ندارد و فرستنده هیچگاه نیاز به صبر ندارد.

Chapter 4: Threads



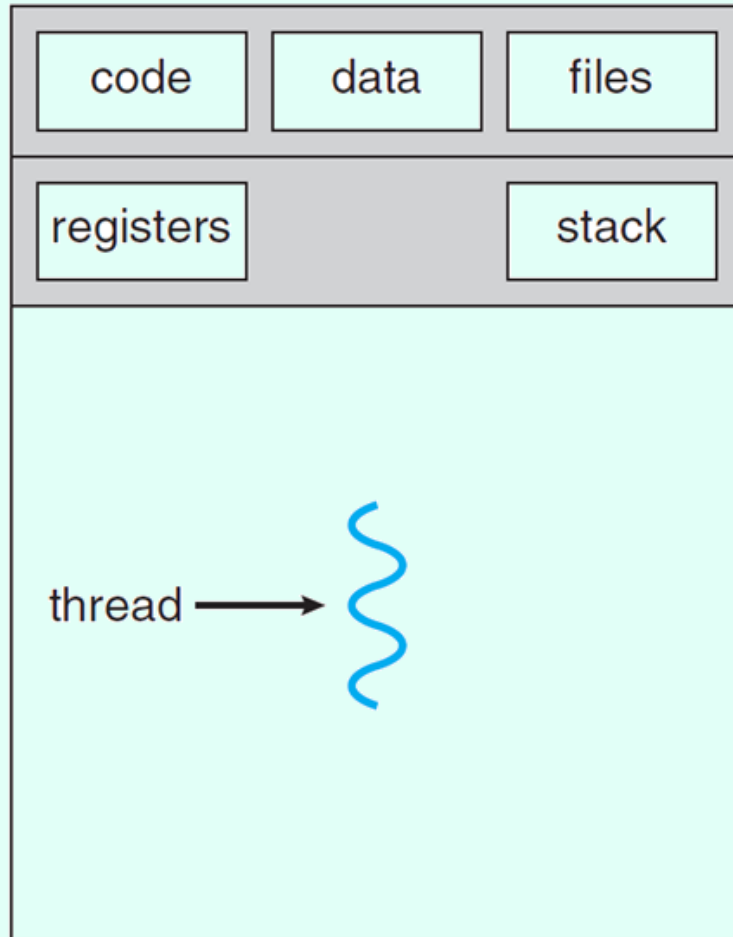
نخ (Thread)

در فصل قبل با فرآیندها آشنا شدیم و فرض کردیم فرآیندها، برنامه‌هایی هستند که فقط با یک نخ اجرا می‌شوند. درحالی‌که بسیاری از برنامه‌های دنیای امروز به صورت چندنخی (multithreaded) اجرا می‌شوند.

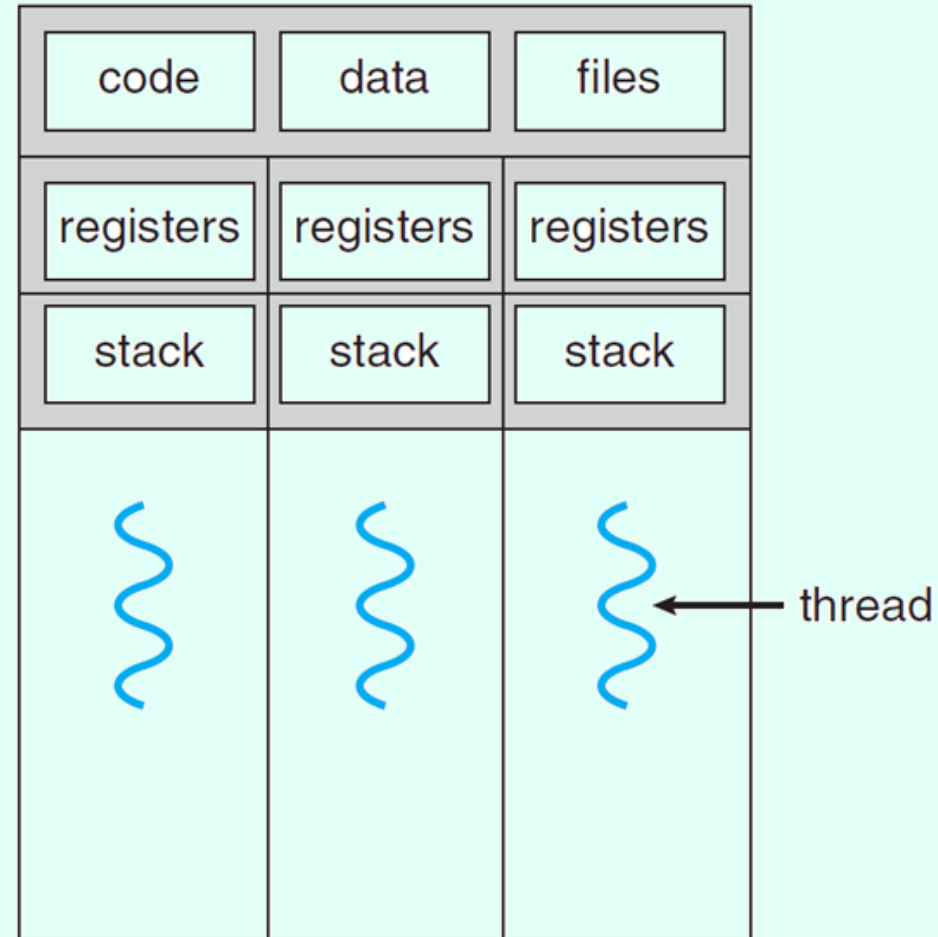
■ وظایف مختلف هر برنامه با یک نخ جدا قابل پیاده‌سازی است، برای مثال، در یک فرآیند می‌توان عملیات‌های روزرسانی صفحه، دریافت داده، چک کردن نوشتار، بررسی درخواست‌های شبکه و ... توسط نخ‌های مختلف اجرا شود.

■ مزیت استفاده از نخ به این دلیل است که ساخت فرآیند جدید پرهزینه است ولی ساخت نخ جدید، هزینه زیادی برای سیستم ندارد. استفاده از چندنخی باعث سادگی و افزایش بهینگی می‌شود.

نخ (Thread)

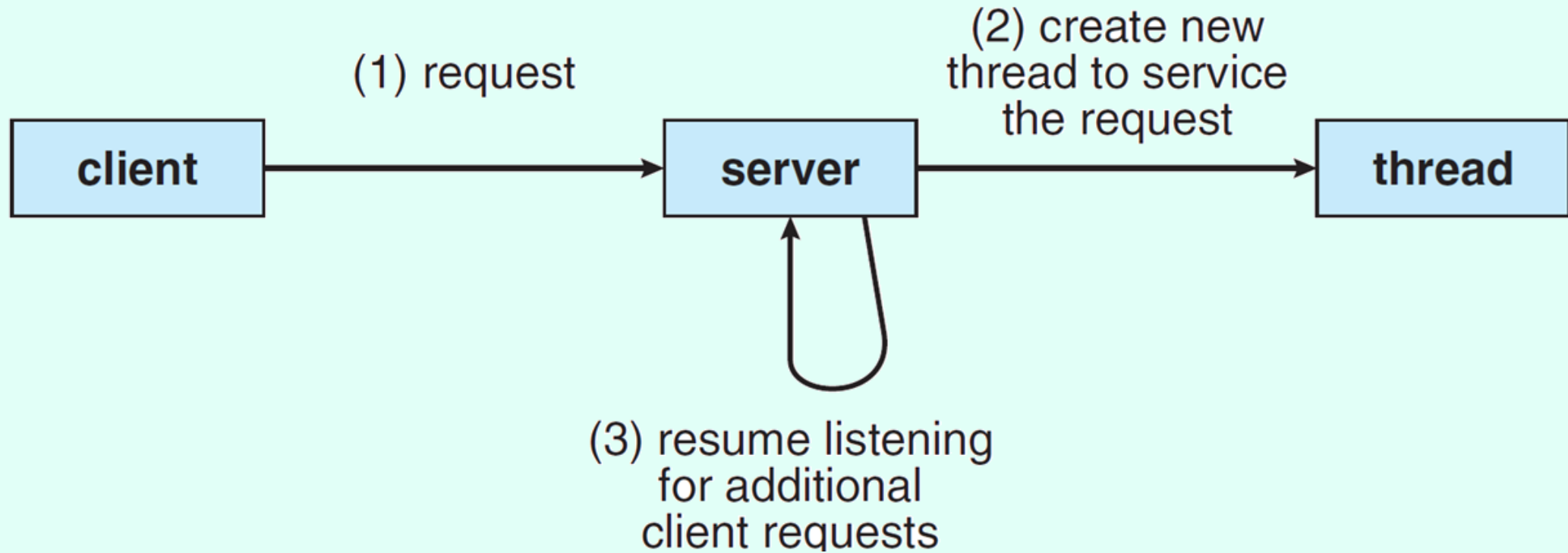


single-threaded process



multithreaded process

معماری سرور چندنخی



نخ (Thread)

■ مزایای استفاده از برنامه‌نویسی چندنخی عبارت‌اند از:

پاسخگویی: اگر قسمتی از فرآیند بلاک شود، اجرا قطع نمی‌شود (کاربردی برای رابط کاربری).

اشتراک منابع: نخ‌ها منابع درون یک فرآیند را به اشتراک می‌گذارند که ساده‌تر از روش‌های حافظه مشترک و ارسال پیام است.

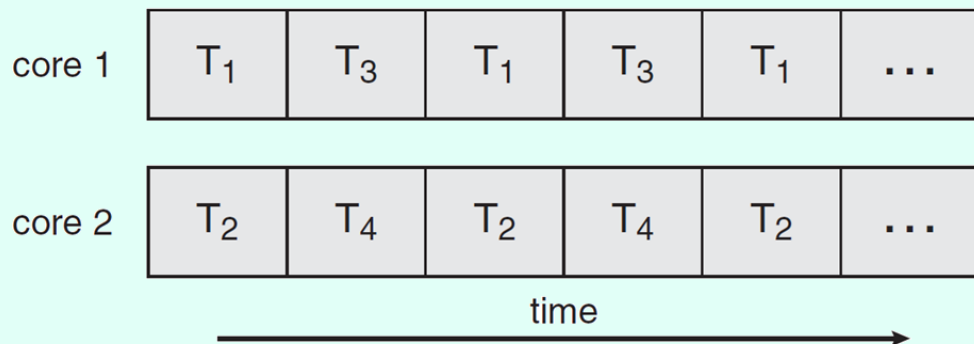
اقتصادی: ایجاد نخ و جابه‌جایی بسیار ارزان‌تر از ایجاد فرآیند و context switching است.

مقیاس‌پذیری: چندنخی در سیستم‌های با چندپردازنده یا چندهسته بسیار کارآمد خواهد بود.

چندهسته (Multicore)

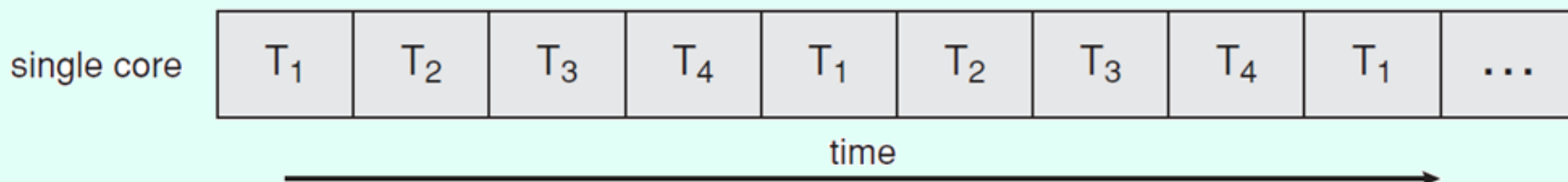
■ در سیستم‌های چندهسته (multicore) یا چندپردازنده (multiprocessor)، می‌توان از parallelism بهره برد، به این صورت که چند نخ به طور همزمان اجرا شوند.

■ به تفاوت موازی بودن (parallelism) و همروندی (concurrency) باید توجه کرد:



parallelism: سیستم چند کار را همزمان انجام دهد.

concurrency: سیستم چند کار را همزمان پیش ببرد.



قانون امدال (Amdahl's Law)

■ این قانون مقدار کارایی قابل دستیابی براساس میزان موازی بودن سیستم که با افزودن تعداد هسته‌ها می‌توان دست یافت را محاسبه می‌کند:

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

S : میزان سریال بودن سیستم

$1 - S$: میزان موازی بودن سیستم

N : تعداد هسته‌ها

قانون امدال (Amdahl's Law)

مثال) در یک سیستم که 60 درصد به صورت موازی اجرا می‌شود، تعداد هسته‌ها را از 2 به 4 تغییر می‌دهیم. سیستم چند درصد سریع‌تر خواهد شد؟

CompYaar

قانون امدال (Amdahl's Law)

مثال) در یک سیستم که 75 درصد به صورت موازی اجرا می‌شود، اگر تعداد هسته‌ها را به بی‌نهایت میل دهیم، میزان speedup چقدر خواهد بود؟

CompYaar

نخ (Thread)

■ چالش‌های برنامه‌نویسان در سیستم‌های چندنخی عبارت‌اند از:

شناسایی وظایف: شناسایی این که چه وظایفی (tasks) مناسب برای موازی‌سازی یا هم‌روندی هستند.

تعادل: وظایفی که به هر نخ در هر هسته انتساب می‌شود، ارزش کافی برای اختصاص دادن هسته یا پردازنده را داشته باشد.

تقسیم داده: همانطور که وظایف بین هسته‌ها/پردازنده‌ها تقسیم می‌شوند، داده نیز براساس هر وظیفه باید تقسیم شود.

وابستگی داده: اگر وظیفه‌ای نیاز به داده‌ای از وظیفه دیگر داشته باشد، باید دسترسی آن به داده همگام‌سازی شود.

تست و دیباگ: تست و دیباگ برنامه‌های هم‌روند دشوارتر از برنامه‌های تک‌نخی هستند.

موازی سازی (Parallelism)

■ دو نوع موازی سازی وجود دارد:

موازی سازی داده (Data Parallelism): تمرکز بر تقسیم مجموعه های مختلف یک داده بین چند هسته و انجام عملیات یکسان بر هر هسته است. برای مثال فرض کنید آرایه ای به طول N از اعداد صحیح داریم و می خواهیم تمام عناصر را با هم جمع کنیم، در یک سیستم تک هسته ای، جمع بین عناصر $[0]$ تا $[N - 1]$ انجام می شود، در سیستم دو هسته ای، یک هسته جمع بین عناصر $[0]$ تا $[N/2 - 1]$ را انجام می دهد و هسته دیگر عناصر $[N/2]$ تا $[N - 1]$ را جمع می کند.

موازی سازی وظیفه (Task Parallelism): تمرکز بر تقسیم وظایف بین هسته ها و اجرای عملیات متفاوت بر هر هسته است. برای مثال آرایه قبلی را در نظر بگیرید، این بار دو هسته داریم که در هر کدام یک عملیات آماری متفاوت را بر هر آرایه اجرا می کنیم.

مدل‌های چندنخی

■ نخ‌ها یا از نوع کاربر هستند (user thread) یا از نوع کرنل (kernel thread). نخ‌های کاربر بدون کرنل مدیریت می‌شوند. نخ‌های کرنل به طور مستقیم توسط سیستم عامل مدیریت می‌شوند.

■ رابطه بین نخ‌های کاربر و نخ‌های کرنل به یکی از سه حالت زیر است:

Many-to-One

One-to-One

Many-to-One

Many-to-One Model

■ چند نخ کاربر به یک نخ کرنل نگاشت می‌شود.

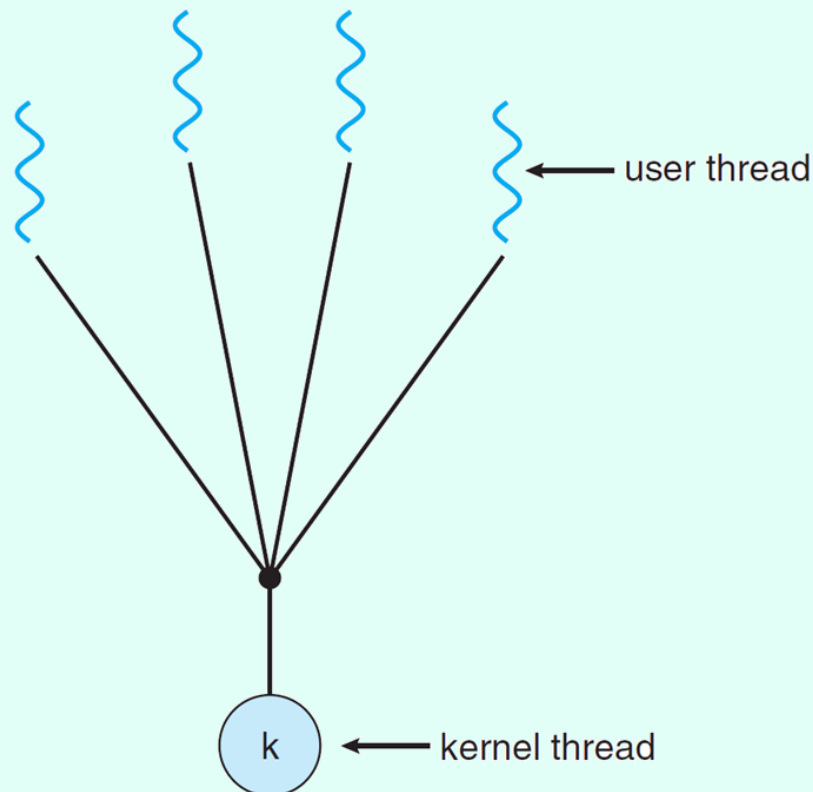
■ بلاک شدن یک نخ کاربر، باعث بلاک شدن تمامی نخ‌ها می‌شود

■ چند نخ به صورت همزمان قابلیت اجرا ندارند، زیرا فقط یک نخ کرنل داریم (همروندی ممکن است، ولی موازی‌سازی خیر)

■ امروزه سیستم‌های بسیار کمی از این مدل استفاده می‌کنند:

- Solaris Green Threads

- GNU Portable Threads



One-to-One Model

■ هر نخ کاربر به یک نخ کرنل نگاشت می‌شود.

■ با ساخت هر نخ کاربر، یک نخ کرنل ایجاد می‌شود

■ همروندی بیشتری به نسبت Many-to-One دارد

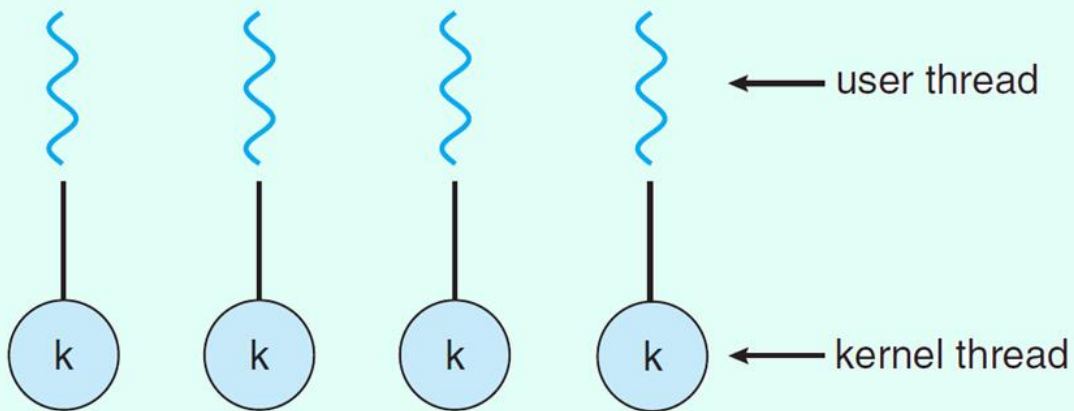
■ به سبب overhead، تعداد نخ‌های هر فرآیند ممکن است محدود شود.

■ سیستم‌هایی که از این مدل استفاده می‌کنند:

- Windows

- Linux

- Solaris 9 and later



Many-to-Many Model

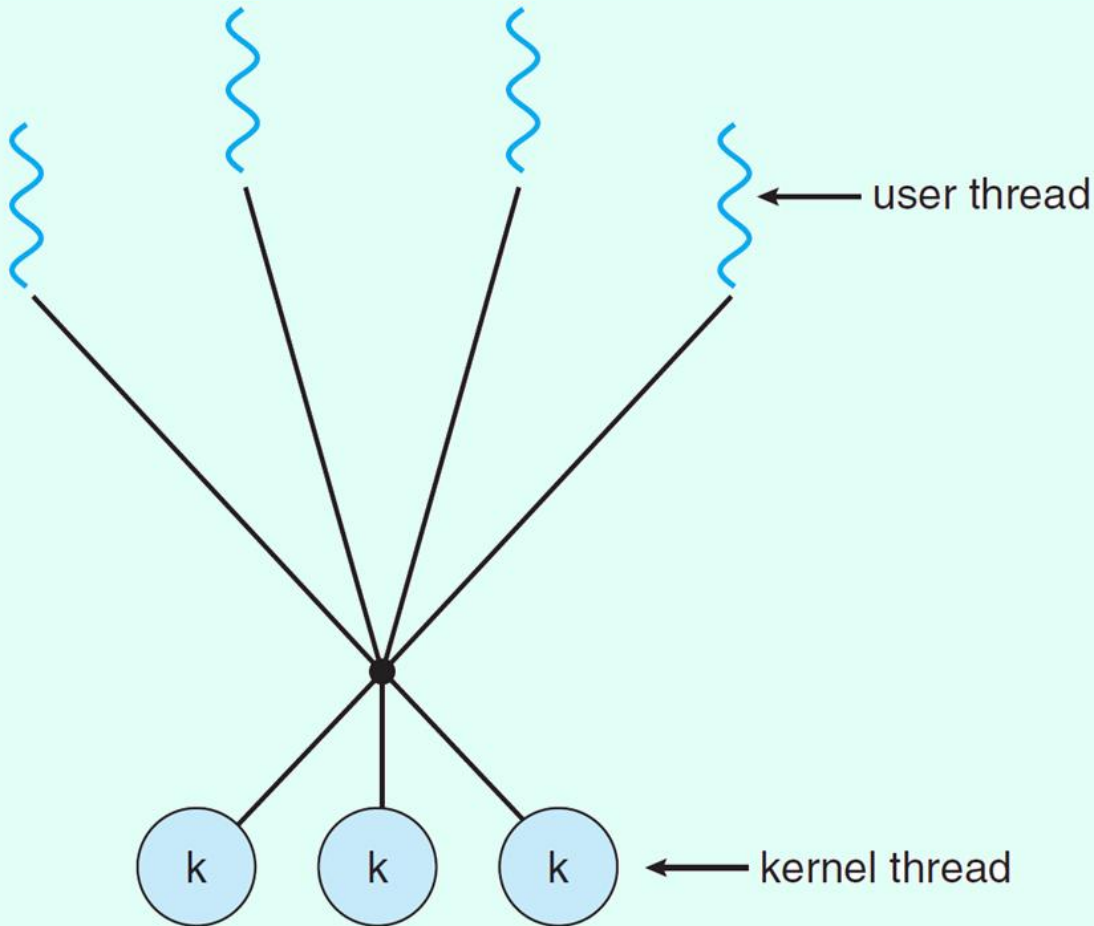
■ چند نخ کاربر به چند نخ کرنل نگاشت می‌شوند.

■ سیستم عامل به تعداد کافی نخ کرنل ایجاد می‌کند

■ سیستم‌هایی که از این مدل استفاده می‌کنند:

- Solaris prior to version 9

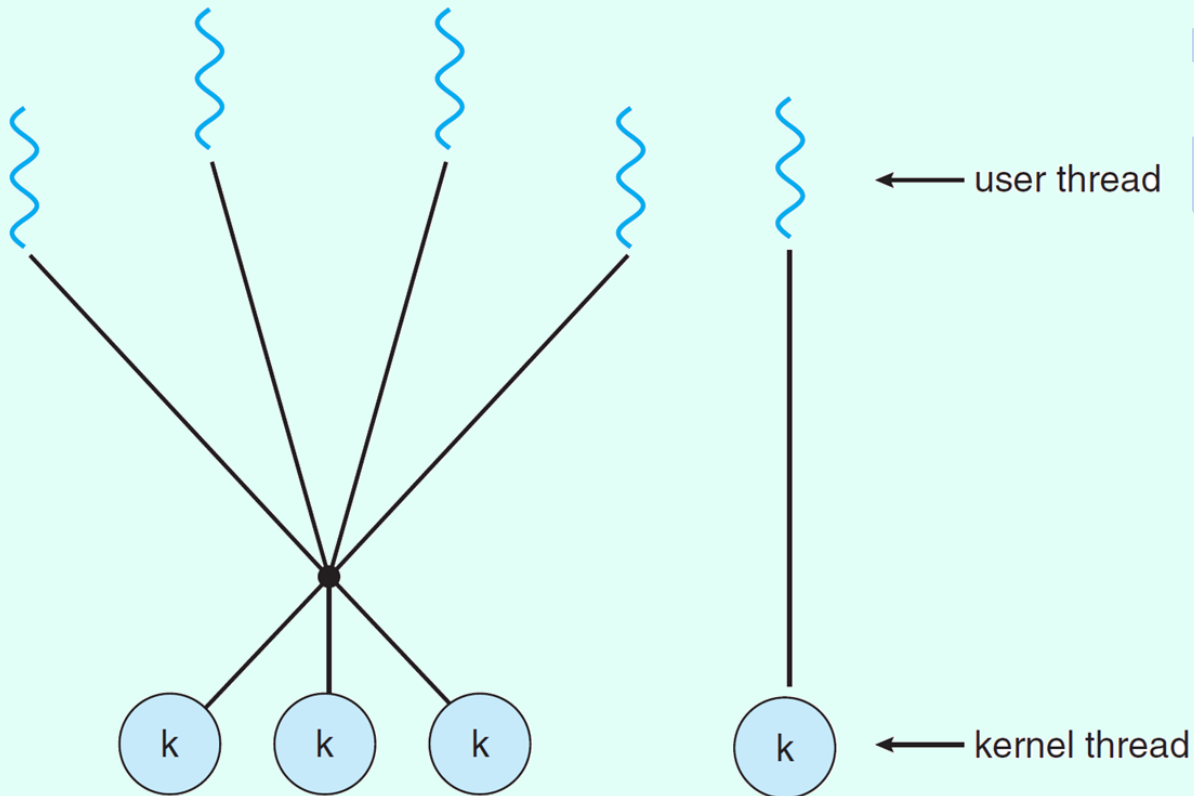
- Windows with the ThreadFiber package



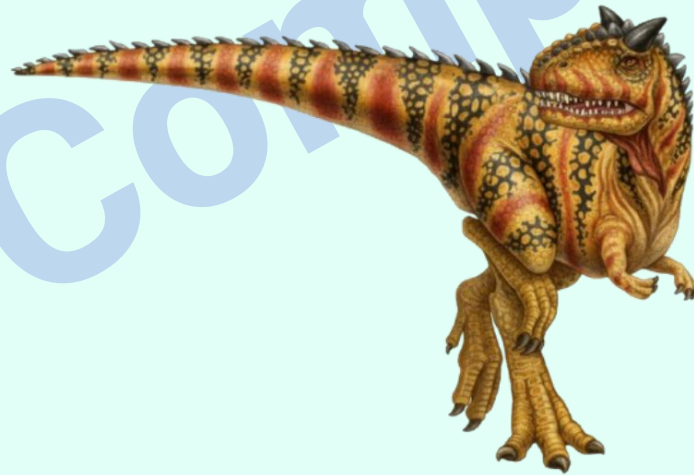
Two-Level Model

■ حالت دیگری از Many-to-Many است

■ اجازه می‌دهد یک نخ کاربر به یک نخ کرنل محدود شود



Chapter 5: Process Synchronization



Data Inconsistency

■ همانطور که در فصل 3 دیدیم، یکی از راه‌های ارتباطی فرآیندها، حافظه مشترک (Shared Memory) است.

■ در این روش سناریوهایی وجود دارد که ممکن است باعث ناسازگاری داده (Data Inconsistency) شود.

برای مثال مسئله تولیدکننده-مصرف‌کننده از فصل 3 را در نظر بگیرید. فرض کنید متغیر جدیدی به نام counter تعریف می‌کنیم که پر بودن یا خالی بودن بافر را کنترل کند.

Data Inconsistency

```
item next_produced;
while(true){
    while(counter == BUFFER_SIZE)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

```
item next_consumed;
while(true){
    while(counter == 0)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
}
```

Data Inconsistency

■ نکته مهمی که وجود دارد این است که دستور counter++ و counter-- اتمی (atomic) نیست، یعنی به ظاهر این خط کد یک عملیات انجام می‌دهد، counter را یک واحد زیاد یا کم می‌کند، ولی در عمل در سه عملیات انجام می‌شود:

counter++ :

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

counter-- :

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

Data Inconsistency

■ حال فرض کنید همزمان فرآیند تولیدکننده و مصرفکننده اجرا می‌شوند، اگر مقدار ابتدایی counter برابر 5 باشد، ممکن است همچین حالتی رخ دهد:

S0: producer execute	<code>register1 = counter</code>	{register1 = 5}
S1: producer execute	<code>register1 = register1 + 1</code>	{register1 = 6}
S2: consumer execute	<code>register2 = counter</code>	{register2 = 5}
S3: consumer execute	<code>register2 = register2 - 1</code>	{register2 = 4}
S4: producer execute	<code>counter = register1</code>	{counter = 6}
S5: consumer execute	<code>counter = register2</code>	{counter = 4}

Critical Section Problem

■ ناحیه بحرانی (critical section) قسمتی از کد است که فرآیندهای مشترک را تغییر دهد.

■ **مسئله ناحیه بحرانی** برای این مطرح شده است که ساز و کاری ارائه شود که فرآیندها بدون ایجاد ناسازگاری داده وارد ناحیه بحرانی شده، کار خود را انجام دهند و از آن خارج شوند.

برای حل مسئله ناحیه بحرانی، کد را به 4 قسمت تقسیم می‌کنیم:

entry section

critical section

exit section

remainder section

Critical Section Problem

do {

entry section

critical section

exit section

remainder section

} while(true);

Critical Section Problem

■ جواب مسئله ناحیه بحرانی باید هر سه شرط زیر را داشته باشد:

انحصار متقابل (Mutual Exclusion): اگر فرآیندی در ناحیه بحرانی در حال اجرا باشد، هیچ فرآیند دیگری نمی‌تواند در ناحیه بحرانی اجرا شود.

پیشرفت (Progress): اگر هیچ فرآیندی در ناحیه بحرانی در حال اجرا نباشد، و تعدادی فرآیند مایل به ورود به ناحیه بحرانی باشند، انتخاب فرآیندی جهت ورود به ناحیه بحرانی نباید برای مدت نامحدودی به تعویق بیفتد.

انتظار محدود (Bounded Waiting): اگر فرآیندی در ناحیه بحرانی باشد و فرآیند دیگری وجود داشته باشد که تمایل به ورود به ناحیه بحرانی را داشته باشد، باید محدودیت زمانی برای فرآیند درون ناحیه بحرانی وجود داشته باشد.

Critical Section Problem

■ به دو روش می‌توان مسئله ناحیه بحرانی را در کرنل مدیریت کرد:

غیر انحصاری (Preemptive): می‌توان اجرای فرآیند را قطع کرد و فرآیند دیگری را جایگزین کرد.

انحصاری (Non-Preemptive): فرآیند تا زمانی که اجرائیش تمام نشود، بلاک نشود یا داوطلبانه از کرنل خارج نشود، اجرائیش ادامه دارد.

Peterson's Solution

- راه حلی برای مسئله ناحیه بحرانی یا دو فرآیند P_i و P_j
- با فرض اینکه دستورات load و store در زبان ماشین اتمی هستند.
- دو فرآیند متغیرهای `int turn` و `Boolean flag[2]` را به اشتراک می‌گذارند
- turn مشخص می‌کند نوبت کدام فرآیند جهت ورود به ناحیه بحرانی است و آرایه `flag` مشخص می‌کند که فرآیند آماده ورود به ناحیه بحرانی است یا خیر.

Peterson's Solution

Pi :

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
        critical section  
    flag[i] = false;  
        remainder section  
} while(true);
```

Synchronization Hardware

- روش پترسون یک روش نرم افزاری (software-based) برای حل مسئله ناحیه بحرانی است.
- برای روش‌های نرم افزاری تضمینی وجود ندارد که در کامپیوترهای مدرن عملیاتی شوند.
- روش‌های دیگری که برای حل مسئله ناحیه بحرانی استفاده می‌شوند، بر اساس قفل کردن (locking) هستند.

Synchronization Hardware

■ مسئله ناحیه بحرانی در سیستم‌های تک پردازنده (single-processor) به سادگی قابل حل هستند. کافی است اطمینان حاصل کنیم که دستورات درون ناحیه بحرانی، به طور انحصاری اجرا شوند.

■ در سیستم‌های چندپردازنده (multi-processor) این روش بهینه نیست. به همین جهت سخت افزار دستورات اتمی را در اختیار ما قرار می‌دهد.

■ این دستورات اتمی را به صورت `test_and_set()` و `compare_and_swap()` نشان می‌دهیم.

Synchronization Hardware

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
  
    return rv;  
}
```

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
  
    return temp;  
}
```

Critical Section Problem

یادآوری: شروط 3گانه جواب مسئله ناحیه بحرانی:

انحصار متقابل (Mutual Exclusion): اگر فرآیندی در ناحیه بحرانی در حال اجرا باشد، هیچ فرآیند دیگری نمی‌تواند در ناحیه بحرانی اجرا شود.

پیشرفت (Progress): اگر هیچ فرآیندی در ناحیه بحرانی در حال اجرا نباشد، و تعدادی فرآیند مایل به ورود به ناحیه بحرانی باشند، انتخاب فرآیندی جهت ورود به ناحیه بحرانی نباید برای مدت نامحدودی به تعویق بیفتد.

انتظار محدود (Bounded Waiting): اگر فرآیندی در ناحیه بحرانی باشد و فرآیند دیگری وجود داشته باشد که تمایل به ورود به ناحیه بحرانی را داشته باشد، باید محدودیت زمانی برای فرآیند درون ناحیه بحرانی وجود داشته باشد.

Synchronization Hardware

انحصار متقابل و پیشرفت با استفاده از :test_and_set()

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
  
    /* remainder section */  
} while (true);
```

Synchronization Hardware

انحصار متقابل و پیشرفت با استفاده از `:compare_and_swap()`

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
} while (true);
```

Synchronization Hardware

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;

    /* critical section */

    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;

    if (j == i)
        lock = false;
    else
        waiting[j] = false;

    /* remainder section */
} while (true);
```

دو روش قبل خاصیت انتظار محدود (bounded-waiting) را برآورده نمی‌کردند. روش زیر این کار را انجام می‌دهد:

```
boolean waiting[n];
boolean lock;
```

Mutex Lock

■ در ادامه ارائه روش‌های حل مسئله ناحیه بحرانی، طراحان سیستم عامل روش دیگری بر مبنای نرم افزار ارائه کردند که به آن Mutex Lock (Mutual Exclusion Lock) گوئیم. در این روش از دو تابع acquire() و release() استفاده می‌کنیم:

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;;  
}
```

```
release() {  
    available = true;  
}
```

Mutex Lock

do {

acquire lock

critical section

release lock

remainder section

} while (true);

Semaphore

■ یکی دیگر از ابزارهای همگام‌ساز فرآیندها سمافور (Semaphore) است.

■ سمافور در واقع یک ساختمان داده است که مقدار یک عدد صحیح را در هنگام مقداردهی اولیه می‌توان به آن منتسب کرد و پس از آن، تغییر مقدار آن فقط از طریق دو تابع `wait()` و `signal()` انجام می‌شود:

```
Semaphore S = 4; // initialization
```

```
S=2; // not supported
```

```
S++; // not supported
```

```
wait(S); //  $S = S - 1 = 3$ 
```

```
signal(S); //  $S = S + 1 = 4$ 
```

Semaphore

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

Semaphore

■ دو نوع سمافور داریم:

Counting Semaphore: دامنه آن مقادیر نامحدودی از اعداد صحیح است.

Binary Semaphore: دامنه آن فقط صفر و یک است.

Semaphore

مثال) دو فرآیند همروند P1 و P2 وجود دارند که به ترتیب عملیات‌های S1 و S2 را انجام می‌دهند. می‌خواهیم مطمئن شویم که S1 به طور کامل قبل از S2 اجرا می‌شود. با استفاده از Semaphore این مسئله را حل کنید.

```
Semaphore synch = 0;
```

```
P1:
```

```
S1;
```

```
signal(synch);
```

```
P2:
```

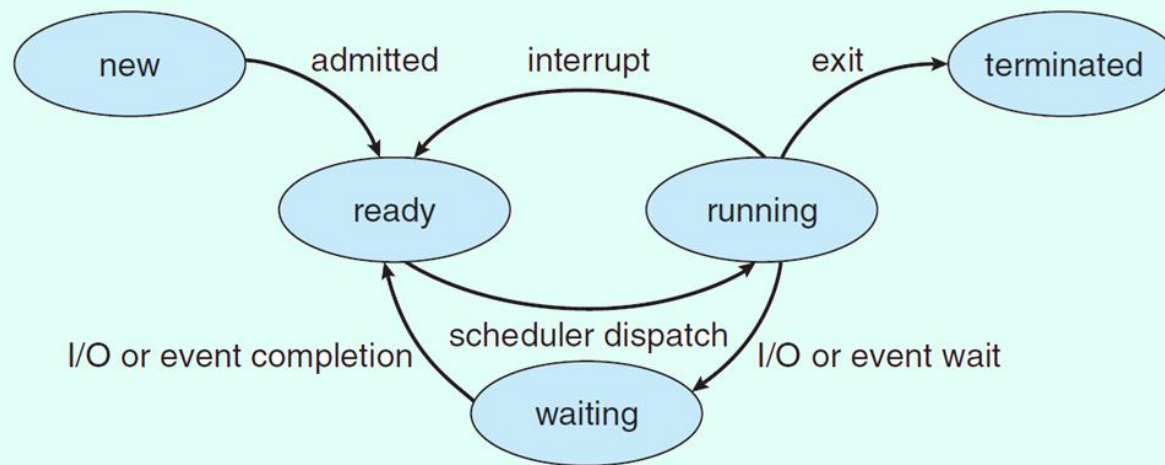
```
wait(synch);
```

```
S2;
```

Semaphore

■ تابع `wait()` سمافور هم مانند `mutex lock` مشکل `busy waiting` دارد.

■ برای حل این مشکل در ساختار سمافور علاوه بر مقدار عددی، یک صف به نام `waiting queue` نیز در نظر می‌گیریم که فرآیندهایی که هنگام اجرای `wait()` دچار `busy waiting` می‌شوند، وارد این صف می‌شوند و حالتشان از `running` به `waiting` تغییر می‌کند.



یادآوری:

Semaphore

■ با اجرای دستور `signal()` نیز یک فرآیند از صف بیرون می‌آید و به حالت `running` تغییر می‌کند.

■ انجام عملیات‌های رفتن به حالت `waiting` و `running` در سمافور با توابع `block()` و `wakeup()` انجام می‌شود.

`block()`: فرآیند را به `waiting queue` می‌فرستد و حالت فرآیند از `running` به `waiting` تغییر می‌کند.

`wakeup()`: یک فرآیند از `waiting queue` خارج می‌شود و حالت آن از `waiting` به `running` تغییر می‌کند.

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Semaphore

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

بن بست (Deadlock)

■ اگر دو یا چند فرآیند منتظر رویدادی باشند که توسط بقیه فرآیندهای منتظر رخ می‌دهد، به این حالت بن بست (deadlock) گوییم.

مثال) دو سمافور S و Q را با مقدار اولیه 1 در نظر بگیرید. فرآیندهای P1 و P2 با شرایط زیر دچار بن بست هستند:

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

گرسنگی (Starvation)

■ اگر فرآیندی برای مدت خیلی طولانی در صف سمافور بماند، گوییم دچار گرسنگی (starvation) شده است.

نکته: تفاوت busy waiting و starvation در وضعیت فرآیندها است، فرآیندی که دچار busy waiting می‌شود حالت Running دارد و فرآیندی که دچار starvation می‌شود حالت waiting دارد.

وارونگی اولویت (Priority Inversion)

■ به حالتی که یک فرآیند با اولویت بالا نیاز به قفلی داشته باشد که در اختیار فرآیند با اولویت پایین است، وارونگی اولویت (priority inversion) گوئیم.

(مثال) سه فرآیند L ، M و H را با اولویت $L < M < H$ در نظر بگیرید. منبع مورد نیاز فرآیند H در اختیار L است و فرآیند M در حال اجرا است، بنابراین فرآیند H که اولویت بالاتری از L دارد، باید در انتظار بماند.

نکته: وارونگی اولویت فقط در سیستم‌های با بیش از دو فرآیند رخ می‌دهد.

راه حل: استفاده از priority inheritance protocol. در مثال بالا اگر از این پروتکل استفاده کنیم، فرآیند L اولویت اولویت فرآیند H را به ارث می‌برد و اجرای M را متوقف می‌کند.

مسائل کلاسیک همگام سازی

■ با ابزارهایی که تا الان آموخته ایم، برخی مسائل کلاسیک و مهم همگام سازی را حل می کنیم. این مسائل عبارت هستند از:

1. بافر محدود (Bounded Buffer)
2. خواننده ها - نویسندگان (Readers - Writers)
3. شام خوردن فیلسوف ها (Dining Philosophers)

مسئله بافر محدود (Bounded-Buffer)

■ این مسئله، همان مسئله تولیدکننده-مصرفکننده (Producer-Consumer) است. یک بافر داریم که تعداد محدودی خانه دارد، مثلاً n خانه. یک فرآیند تولیدکننده داریم که آیتم تولید می‌کند و در بافر می‌گذارد و یک فرآیند مصرفکننده داریم که آیتم‌های درون بافر را مصرف می‌کند.

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0
```

مسئله بافر محدود (Bounded-Buffer)

```
do {  
    . . .  
    /* produce an item in next_produced */  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    /* add next_produced to the buffer */  
    . . .  
    signal(mutex);  
    signal(full);  
} while (true);
```

Figure 5.9 The structure of the producer process.

```
do {  
    wait(full);  
    wait(mutex);  
    . . .  
    /* remove an item from buffer to next_consumed */  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    /* consume the item in next_consumed */  
    . . .  
} while (true);
```

Figure 5.10 The structure of the consumer process.

مسئله خواننده‌ها-نویسنده‌ها (Readers-Writers)

■ یک پایگاه داده را در نظر بگیرید که چند فرآیند بخواهند به آن دسترسی داشته باشند. اگر فرآیندها فقط قصد خواندن داده‌های این پایگاه داده را داشته باشند، مشکلی پیش نمی‌آید، ولی اگر یک فرآیند قصد تغییر محتوای پایگاه داده را داشته باشد و هر فرآیند دیگری با هر هدفی به همان داده‌ها دسترسی داشته باشد، دچار مشکل خواهیم شد.

	Process1 Type	Process2 Type	Permission
Case1	writing	writing	Not Allowed
Case2	writing	reading	Not Allowed
Case3	reading	writing	Not Allowed
Case4	reading	reading	Allowed

مسئله خواننده‌ها-نویسنده‌ها (Readers-Writers)

■ این مسئله سه نوع راه حل دارد:

روش 1 (First): هیچ فرآیند reader ای صبر نمی‌کند مگر یک writer در ناحیه بحرانی باشد (writer ممکن است دچار starvation شود)

روش 2 (Second): هیچ فرآیند writer ای صبر نمی‌کند مگر یک هر فرآیند دیگری در ناحیه بحرانی باشد (reader ممکن است دچار starvation شود)

روش starve-free: هیچ نوع فرآیندی بر دیگری اولویت ندارد

روش 1

```
do {  
    wait(rw_mutex);  
    . . .  
    /* writing is performed */  
    . . .  
    signal(rw_mutex);  
} while(true);
```

Writer

```
do {  
    wait(r_mutex);  
    read_count++;  
    if(read_count == 1)  
        wait(rw_mutex);  
    signal(r_mutex);  
    . . .  
    /* reading is performed */  
    . . .  
    wait(r_mutex);  
    read_count--;  
    if(read_count == 0)  
        signal(rw_mutex);  
    signal(r_mutex);  
} while(true);
```

Reader

```
semaphore rw_mutex = 1;  
semaphore r_mutex = 1;  
int read_count = 0;
```

COMPYAAR YOUTUBE CHANNEL

روش 2

```
do {
    wait(w_mutex);
    write_count++;
    if(write_count == 1)
        wait(read_try);
    signal(w_mutex);
    wait(rw_mutex);
    . . .
    /* writing is performed */
    . . .
    signal(rw_mutex);
    wait(w_mutex);
    write_count--;
    if(write_count == 0)
        signal(read_try);
    signal(w_mutex);
} while(true);
```

Writer

```
do {
    wait(read_try);
    wait(r_mutex);
    read_count++;
    if(read_count == 1)
        wait(rw_mutex);
    signal(r_mutex);
    signal(read_try);
    . . .
    /* reading is performed */
    . . .
    wait(r_mutex);
    read_count--;
    if(read_count == 0)
        signal(rw_mutex);
    signal(r_mutex);
} while(true);
```

Reader

```
semaphore rw_mutex = 1;
semaphore r_mutex = 1;
semaphore w_mutex = 1;
semaphore read_try = 1;
int read_count = 0;
int write_count = 0;
```

COMPYAAR **YOUTUBE** CHANNEL

روش 3

```
do {
    wait(entry_mutex);
    wait(rw_mutex);
    signal(entry_mutex);
    . . .
    /* writing is performed */
    . . .
    signal(rw_mutex);
} while(true);
```

Writer

```
do {
    wait(entry_mutex);
    wait(r_mutex);
    read_count++;
    if(read_count == 1)
        wait(rw_mutex);
    signal(r_mutex);
    signal(entry_mutex);
    . . .
    /* reading is performed */
    . . .
    wait(r_mutex);
    read_count--;
    if(read_count == 0)
        signal(rw_mutex);
    signal(r_mutex);
} while(true);
```

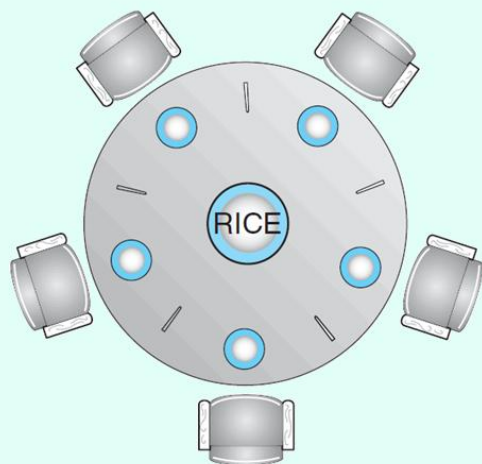
Reader

```
semaphore rw_mutex = 1;
semaphore r_mutex = 1;
semaphore entry_mutex = 1;
int read_count = 0;
```

COMPYAAR YOUTUBE CHANNEL

مسئله فیلسوف‌ها (Dining Philosophers)

■ پنج فیلسوف داریم که دور یک میز دایره‌ای نشسته‌اند. رو به روی هر فیلسوف یک بشقاب است و در مرکز میز یک ظرف برنج. همچنین پنج چاپستیک روی میز قرار داده شده است. کار فیلسوف‌ها فکر کردن و غذا خوردن است. اگر فیلسوفی فکر کند، گرسنه‌اش نیست، ولی هر لحظه ممکن است گرسنه شود و نیاز به خوردن برنج داشته باشد. به این منظور نیاز به دو چاپستیک نزدیک خود دارد. برداشتن هر چاپستیک فقط در یک مرحله انجام می‌شود. اگر یک فیلسوف چاپستیک داشته باشد، فیلسوف بقی نمی‌تواند آن را از دستش بگیرد.



مسئله فیلسوف‌ها (Dining Philosophers)

■ می‌توان برای حل این مسئله از کد زیر استفاده کرد:

```
semaphore chopstick[5];

do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    . . .
    /* eat for awhile */
    . . .
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    . . .
    /* think for awhile */
    . . .
} while (true);
```

برای هر چاپستیک یک سمافور تعریف کنیم و مقدار اولیه‌شان را 1 دهیم. هر بار که فیلسوفی چاپستیکی را برمیدارد wait می‌شود، و هر بار که چاپستیک را روی میز می‌گذارد، signal می‌شود.

مشکل: اگر هر فیلسوف دقیقاً یک چاپستیک بردارد، دچار بن‌بست (deadlock) می‌شویم.

مسئله فیلسوف‌ها (Dining Philosophers)

■ اگر برای برداشتن چابستیک‌ها یک سمافور تعریف کنیم، مشکل بن‌بست حل می‌شود:

```
do {  
    wait(mutex);  
    wait(chopstick[i]);  
    wait(chopstick[(i+1)%5];  
    . . .  
    /* eat for awhile */  
    . . .  
    signal(chopstick[i]);  
    signal(chopstick[(i+1)%5];  
    . . .  
    /* think for awhile */  
    . . .  
} while(true);
```

مانیتور (Monitor)

■ اگرچه سمافور ابزار بسیار موثری برای همگام‌سازی فرآیندها است، ولی اگر برنامه‌نویس به درستی از آن استفاده نکند، ممکن است دچار مشکلات timing شویم. برای مثال می‌دانیم که به سمافوری مانند mutex جهت کنترل ورود به ناحیه بحرانی و خروج از آن نیاز داریم که به صورت زیر استفاده می‌شود:

```
wait(mutex);  
...  
critical section  
...  
signal(mutex);
```

حال اگر برنامه‌نویس به درستی از سمافور استفاده نکند چه اتفاقی می‌افتد؟ برای مثال ابتدا signal سپس wait کند، یا دو بار wait کند و ...

مانیتور (Monitor)

■ برای مقابله با این مشکل، ساختار همگام‌سازی جدیدی به نام مانیتور (monitor) معرفی خواهد شد.

مانیتور کلاس یا ماژولی است که موارد زیر را کپسوله می‌کند:

1. متغیرهای مشترک
2. عملیات‌های متغیرهای مشترک
3. متغیرهای شرطی (condition variables) که برای wait و signal استفاده می‌شوند

وظیفه مانیتور اطمینان از این است که در هر لحظه فقط یک فرآیند در آن فعال باشد. برنامه‌نویس نیازی ندارد این مورد را به طور صریح کد بزند.

مانیتور (Monitor)

■ مانیتور مانند اتاقی است که یک کلید دارد و هر بار فقط یک فرآیند که کلید را دارد می‌تواند وارد آن شود. بقیه فرآیندها برای ورود به مانیتور باید wait کنند. مانیتور mutual exclusion را به طور خودکار بدون نیاز به `wait(mutex)` و `signal(mutex)` برقرار می‌کند.

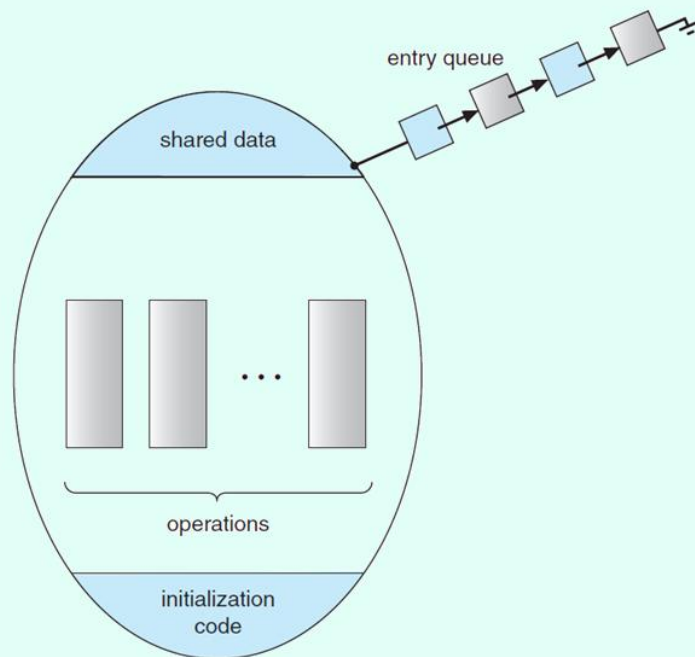


Figure 5.16 Schematic view of a monitor.

مانیتور (Monitor)

■ اگر فرآیندی نتواند وارد مانیتور شود، wait شدن آن و سپس wake up شدن آن با متغیرهای شرطی (condition variable) انجام می‌شود. این متغیرها به یک شرط در داده مشترک منتسب می‌شوند و فرآیندها می‌توانند روی این شرط wait کنند (متوقف شوند تا زمانی که رویدادی رخ دهد) یا signal کنند. (یک فرآیند wait شده wake up شود)

```
condition x;
```

```
x.wait() /* suspends the calling process on condition variable x. it will resume  
only when another process calls x.signal */
```

```
x.signal() /* wakes up one process waiting on x. if no process is waiting, the  
signal is ignored */
```

مانیٲور (Monitor)

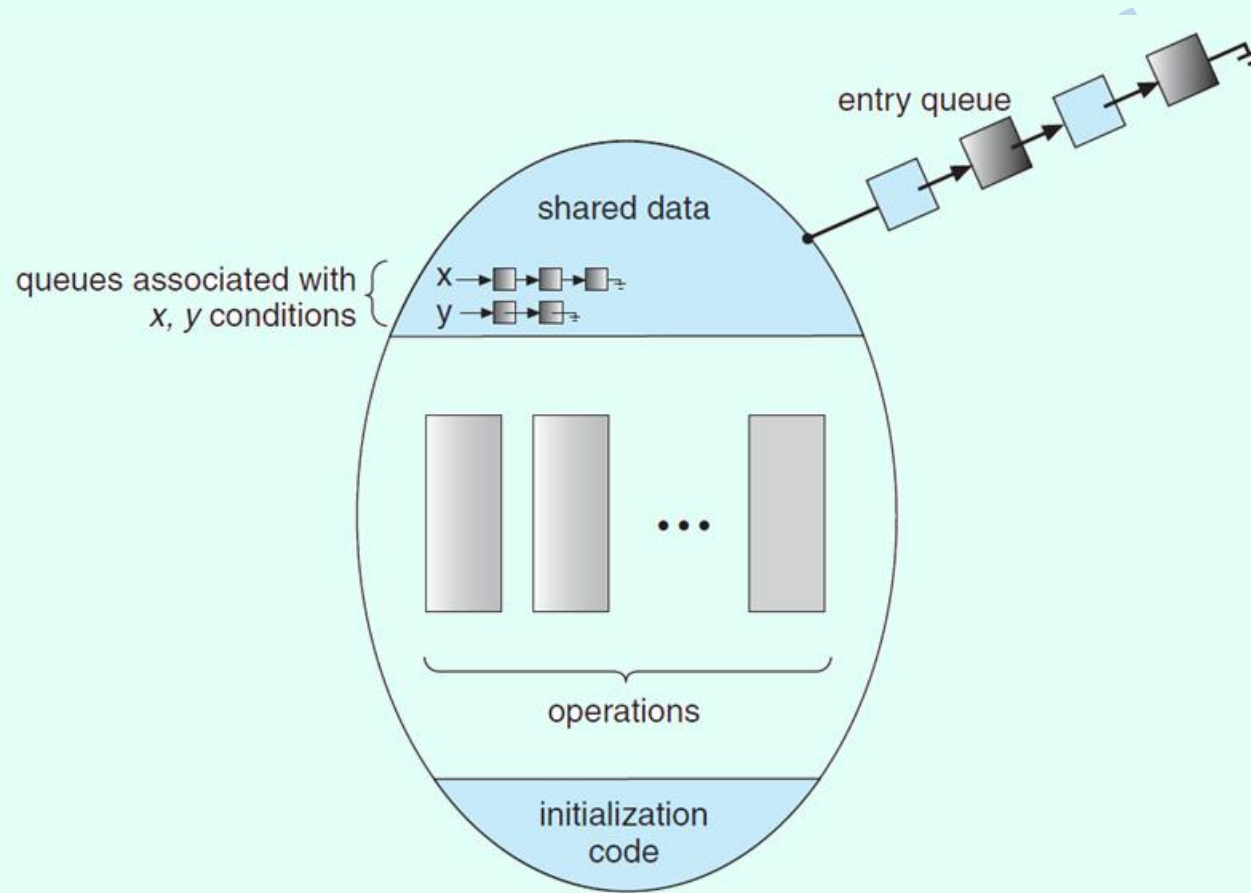


Figure 5.17 Monitor with condition variables.

مانیتور (Monitor)

■ اگر فرآیند Q به سبب `x.wait()` معلق شده باشد، و فرآیند P خط `x.signal()` را اجرا کند، کدام فرآیند باید فعال شود؟ P یا Q؟ دو سناریو وجود دارد:

Signal and Wait: P صبر می‌کند تا Q مانیتور را ترک کند یا به سبب شرطی `wait` کند

Signal and Continue: Q صبر می‌کند تا P مانیتور را ترک کند یا به سبب شرطی `wait` کند

جفت روش‌ها معایب و مزایایی دارند و کسی که زبان را پیاده‌سازی می‌کند باید انتخاب کند از کدام سناریو استفاده کند.

مسئله فیلسوفها با مانیتور

```
monitor DiningPhilosophers
{
    enum {THINKING, HUNGRY, EATING} state[5];
    condition self[5];

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }

    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}
```

```
void test(int i) {
    if ((state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
```

پیاده‌سازی مانیتور با سمافور

■ برای پیاده‌سازی مانیتور با سمافور، به 2 سمافور mutex و next به ترتیب به جهت ورود و خروج به مانیتور و wake up کردن فرآیندهای در حال wait نیاز داریم. next_count نیز تعداد فرآیندهای تعلیق شده را نشان می‌دهد:

```
semaphore mutex = 1;
semaphore next = 0;
int next_count = 0;

wait(mutex);          /* enter monitor */
    body of F          /* monitor function F */
if(next_count > 0)
    signal(next);      /* if a process is waiting to enter monitor */
else
    signal(mutex)      /* release the monitor lock */
```

پیاده‌سازی مانیتور با سمافور

■ برای پیاده‌سازی متغیر شرطی x و عملیات‌های wait و signal آن، به صورت زیر عمل می‌کنیم:

```
semaphore x_sem = 0; /* waiting queue for this condition */  
int x_count = 0;      /* number of processes waiting on it */
```

```
x_count++;  
if(next_count > 0)  
    signal(next);  
else  
    signal(mutex);  
wait(x_sem);  
x_count--;
```

wait

```
if(x_count > 0){  
    next_count++;  
    signal(x_sem);  
    wait(next);  
    next_count--;  
}
```

signal

ادامه اجرای فرآیندها در مانیتور

■ فرض کنید در مانیتوری تعدادی فرآیند روی شرط x معلق شده‌اند، و یک فرآیند $x.signal()$ را اجرا می‌کند. حال کدام فرآیند معلق شده باید اجرای خود را از سر گیرد؟ به چه ترتیبی فرآیندهای معلق باید اجرا شوند؟ یک راه حل ساده استفاده از FCFS (First-Come, First-Served) است. ولی این راه حل شاید به اندازه کافی خوب نباشد. برای مثال اگر فرآیندی کار مهمی داشته باشد و وقت نداشته باشد ولی فرآیندهای دیگری زودتر از آن $wait$ شده باشند، این فرآیند از دست می‌رود.

برای حل این مشکل، از ساختار conditional-wait استفاده می‌کنیم:

`wait(c);`

که c یک عدد صحیح است که اولویت فرآیند معلق شده است. هر چه c کوچک‌تر باشد، اولویت فرآیند بیشتر است.

ادامه اجرای فرآیندها در مانیتور

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;

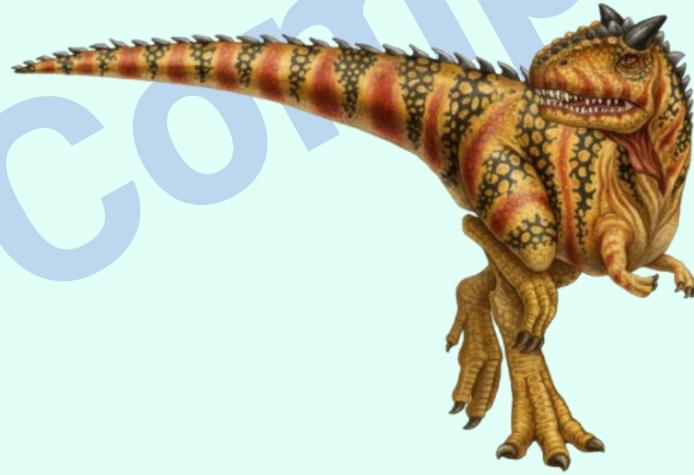
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = true;
    }

    void release() {
        busy = false;
        x.signal();
    }

    initialization_code() {
        busy = false;
    }
}
```

برای مثال مانیتور ResourceAllocator را در نظر بگیرید. وظیفه آن اختصاص یک منبع واحد بین چندین فرآیند است. هر فرآیند یک زمان اجرا دارد و مانیتور فرآیندی را جهت اجرا انتخاب می‌کند که کمترین زمان اجرا را دارد.

Chapter 6: CPU Scheduling



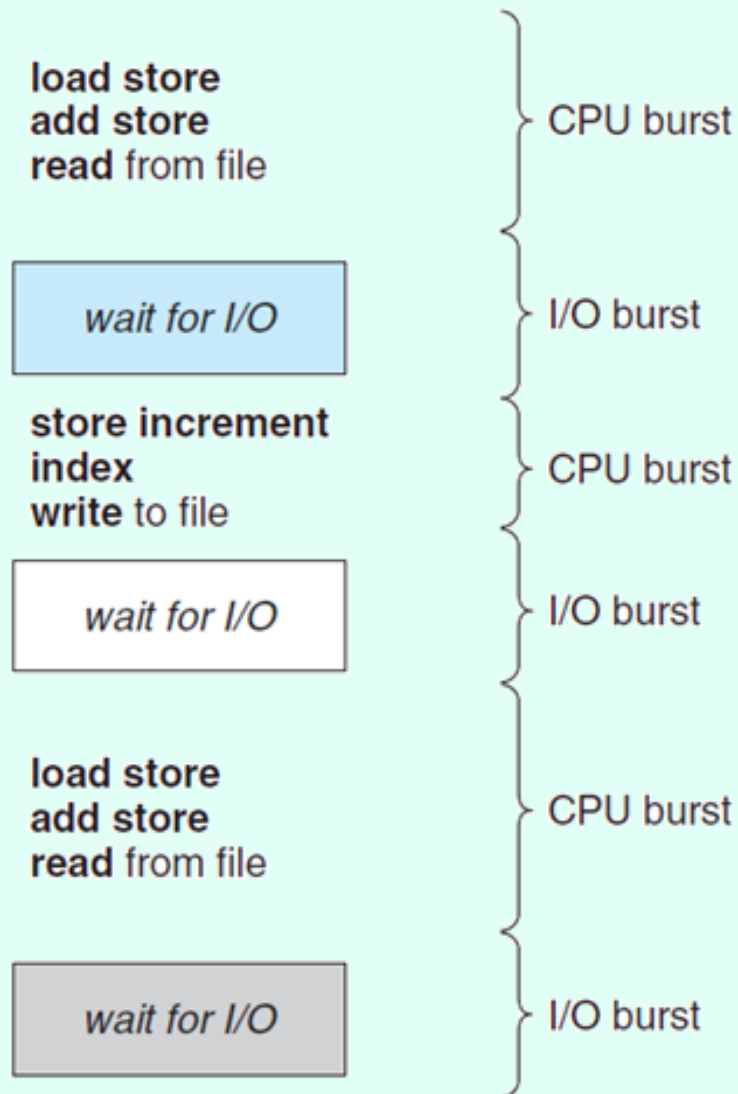
Multiprogramming

■ در یک سیستم تک پردازنده (Single Processor)، در CPU در یک زمان فقط یک فرآیند اجرا می‌شود. اجرای فرآیند تا زمانی طول می‌کشد که یا تمام شود یا به واسطه رویدادی مانند I/O، به حالت waiting برود. در این وقفه زمانی که فرآیند به waiting می‌رود تا بازگردد، عملاً CPU بیکار (idle) است و اتلاف زمانی داریم.

■ با تکنیک Multiprogramming، از این وقفه زمانی استفاده می‌کنیم و فرآیندی دیگری را به CPU می‌دهیم تا اجرا کند.

■ اینکه کدام فرآیندها را در چه زمانی به CPU دهیم و کی اجرای فرآیندها را متوقف کنیم را الگوریتم‌های زمان‌بندی (Scheduling) تعیین می‌کنند.

CPU-I/O Burst Cycle

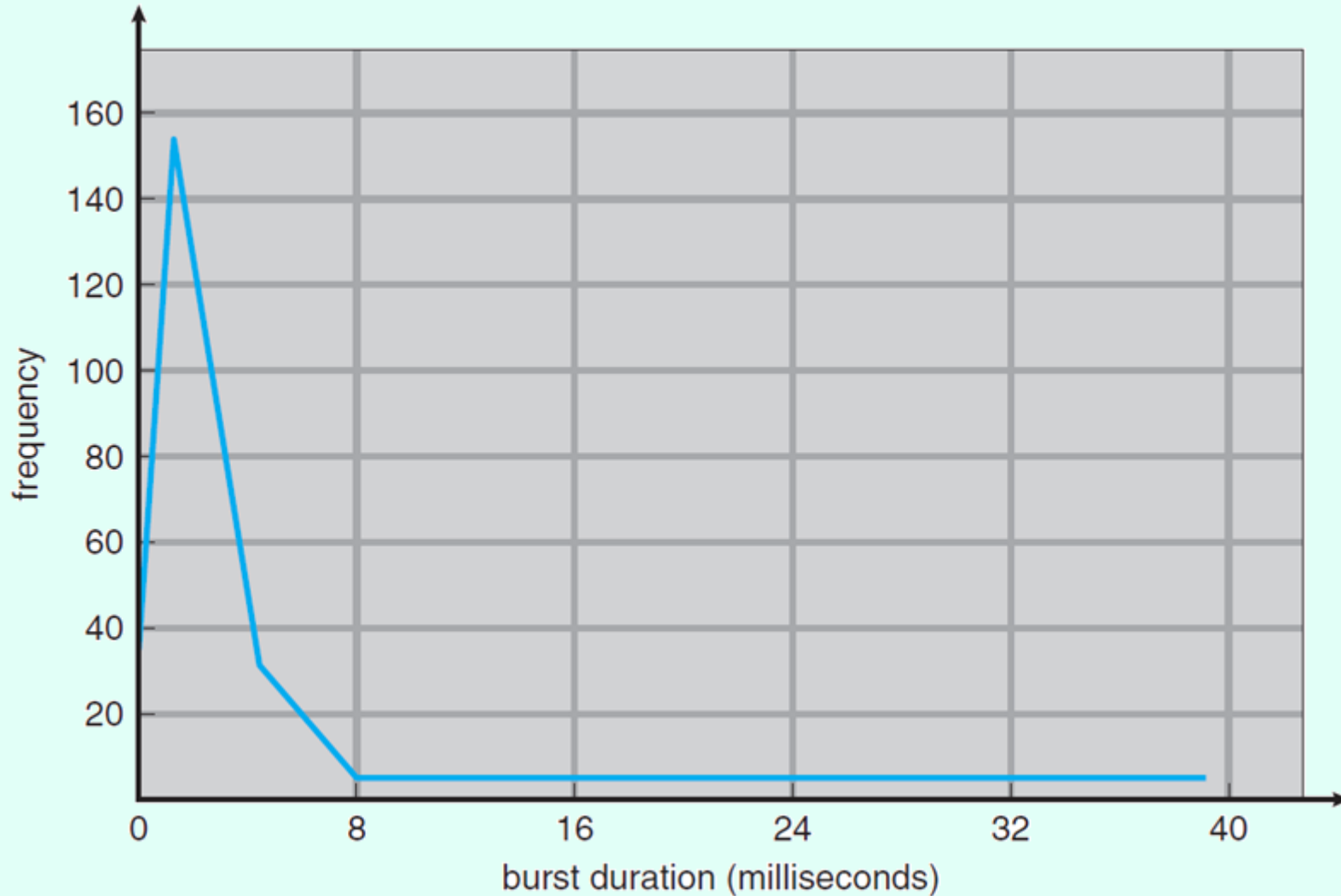


■ اجرای هر برنامه دو قسمت است: CPU Burst و I/O Burst

■ این چرخه نشان دهنده این است که یک برنامه در چه زمان هایی در پردازنده در حال اجرا است و در چه زمان هایی نیاز به I/O دارد.

■ برنامه های CPU Bound تعدادهای کم ولی طولانی CPU Burst دارند و برنامه های I/O Bound تعدادهای زیاد ولی کوتاه CPU Burst دارند.

Burst Duration - Frequency



CPU Scheduler

■ در فصل 3 (فرآیندها) با زمان‌بندهای مختلف سیستم آشنا شدیم. زمان‌بند کوتاه مدت (Short-Term Scheduler) موظف است از بین فرآیندهای ready queue ، یکی را انتخاب کند و CPU را به آن اختصاص دهد.

■ زمان‌بندی در یکی از 4 لحظه زیر انجام میشود:

حالت 1 و 4 را انحصاری (non-preemptive) گوئیم و
حالت 2 و 3 را انحصاری (preemptive) گوئیم.

زیرا در حالت 1 و 4 فرآیند خودش از CPU خارج
میشود، ولی در 2 و 3 سیستم عامل اجرا را کنترل
میکند.

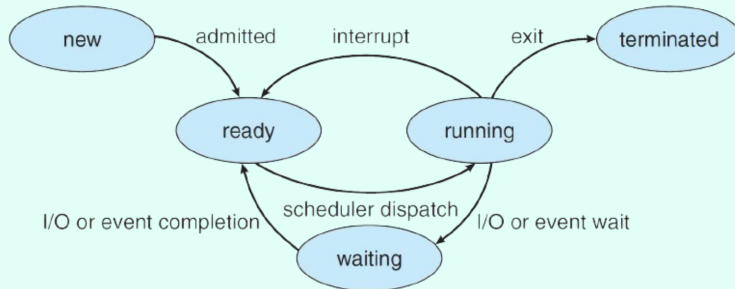
1. حالت فرآیندی از running به waiting تغییر کند.
2. حالت فرآیندی از running به ready تغییر کند.
3. حالت فرآیندی از waiting به ready تغییر کند.
4. فرآیندی خاتمه یابد.

Dispatcher

■ زمانی که زمان‌بند (Scheduler) تصمیم می‌گیرد که کدام فرآیند اجرا شود، ماثولی که کنترل CPU را به فرآیند جهت اجرا می‌دهد، dispatcher است.

■ وظایف اصلی dispatcher عبارت‌اند از:

1. switching context: ذخیره ویژگی‌های فرآیند قبلی و بارگذاری فرآیند جدید.
2. switching to user mode: انتقال به مود کاربر
3. jumping to the right instruction: از سرگیری اجرای فرآیند از نقطه قبلی.



Scheduling Criteria

■ الگوریتم‌های زمان‌بندی متعددی وجود دارند و اینکه کدام الگوریتم مناسب است بر اساس قیود زیر تصمیم‌گیری می‌شود:

- CPU Utilization: میزان فعالیت CPU. در طول زمان چند درصد در حال اجرای فرآیندها بوده است؟
- Throughput: تعداد فرآیندهایی که در طول زمان اجرایشان کامل شده است.
- Turnaround Time: زمان مورد نیاز برای اجرای یک فرآیند.
- Waiting Time: زمانی که فرآیند در ready queue منتظر اجرا شدن است.
- Response Time: مدت زمانی که طول می‌کشد تا سیستم اولین واکنش را به درخواست فرآیند بدهد.

Scheduling Criteria

مثال) برای طراحی یک الگوریتم زمان بندی بهینه، ترجیح می دهیم کدام قیود بیشینه شوند و کدام قیود بهینه؟

CompYaar

الگوریتم‌های زمان‌بندی CPU

■ کتاب الگوریتم‌های زمان‌بندی را به صورت زیر دسته‌بندی می‌کند:

1. الگوریتم‌های پایه: FCFS، SJF، SRTF، RR، PS

2. الگوریتم‌های چندلایه: MLQ، MLFQ

3. الگوریتم‌های ویژه: RM، EDF

4. الگوریتم‌های اضافی: HRRN، LS، LLF

* الگوریتم‌های دسته 4 در کتاب نیستند ولی در آزمون‌ها مطرح می‌شوند.

First-Come First-Served (FCFS)

■ ساده‌ترین الگوریتم زمان‌بندی همین الگوریتم است.

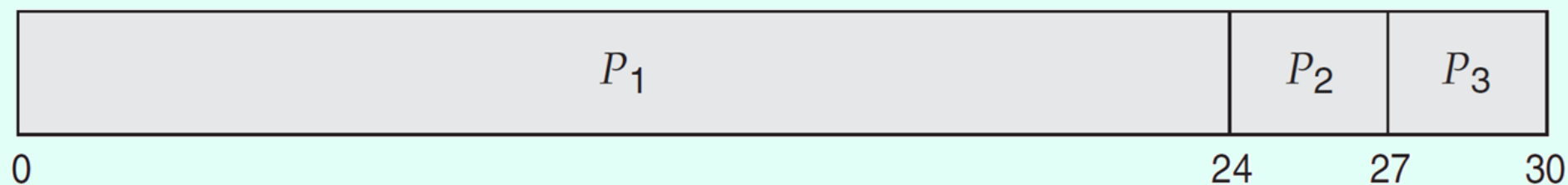
الگوریتم FCFS یک الگوریتم انحصاری (non-preemptive) است.

هر فرآیندی که زودتر وارد صف ready شود، زودتر به حالت running در می‌آید.

First-Come First-Served (FCFS)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. ترتیب ورود فرآیندها به حالت ready به صورت P_1 سپس P_2 سپس P_3 است. با استفاده از الگوریتم FCFS، این فرآیندها را زمان بندی کنید. زمان انتظار متوسط را محاسبه کنید.

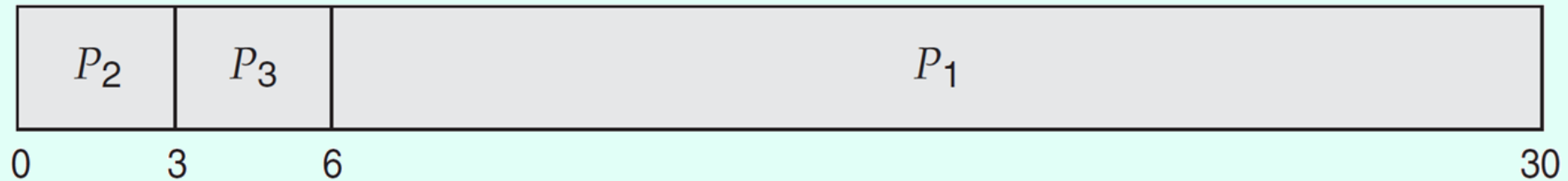
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3



First-Come First-Served (FCFS)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. ترتیب ورود فرآیندها به حالت ready به صورت P_3 سپس P_2 سپس P_1 است. با استفاده از الگوریتم FCFS، این فرآیندها را زمان بندی کنید. زمان انتظار متوسط را محاسبه کنید.

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3



First-Come First-Served (FCFS)

■ سیستمی را در نظر بگیرید که یک فرآیند CPU Bound و تعداد زیادی فرآیند I/O Bound دارد. فرض می‌کنیم فرآیند CPU Bound زودتر از باقی فرآیندها وارد CPU می‌شود. اگر با الگوریتم FCFS زمان‌بندی را انجام دهیم، فرآیند CPU Bound زمان طولانی اجرا خواهد شد و فرآیندهای I/O Bound برای اجرایی کوتاه باید منتظر پایان اجرای فرآیند CPU Bound باشند. در این حالت دستگاه‌های I/O بیکار خواهند ماند. بعد از پایان اجرای فرآیند طولانی، به I/O می‌رود و فرآیندهای I/O Bound به سرعت در CPU اجرا می‌شوند. در این حالت CPU بیکار خواهد ماند. این چرخه بارها و بارها تکرار خواهد شد و به آن **convoy effect** گوییم، یعنی فرآیندی کوتاه پشت سر فرآیندی طولانی گیر کند.

Shortest Job First (SJF)

■ این الگوریتم بر اساس زمان مورد نیاز فرآیندها جهت اجرا عمل می‌کند.

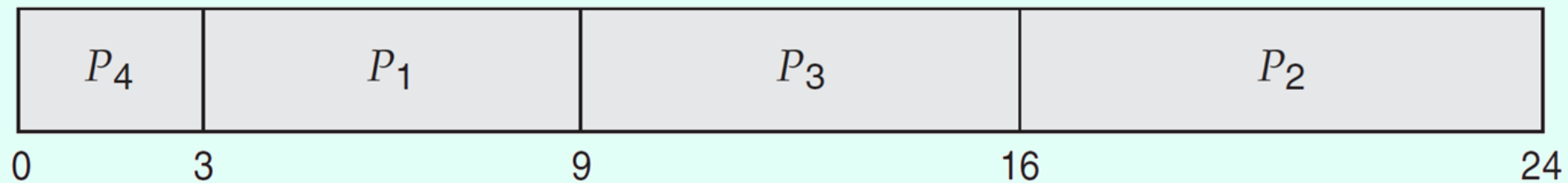
الگوریتم SJF یک الگوریتم انحصاری (non-preemptive) است.

هر فرآیندی که زمان اجرای کمتری داشته باشد، زودتر اجرا می‌شود.

Shortest Job First (SJF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم SJF، این فرآیندها را زمان‌بندی کنید. زمان انتظار متوسط این الگوریتم را با FCFS مقایسه کنید.

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3



پیشبینی طول فرآیند بعدی

■ الگوریتم SJF بهینه است، ولی یک مشکل دارد، آن هم این است که طول اجرای تمام فرآیندها را در ابتدا نیاز دارد و می‌دانیم در دنیای واقعی همچنین چیزی ممکن نیست.

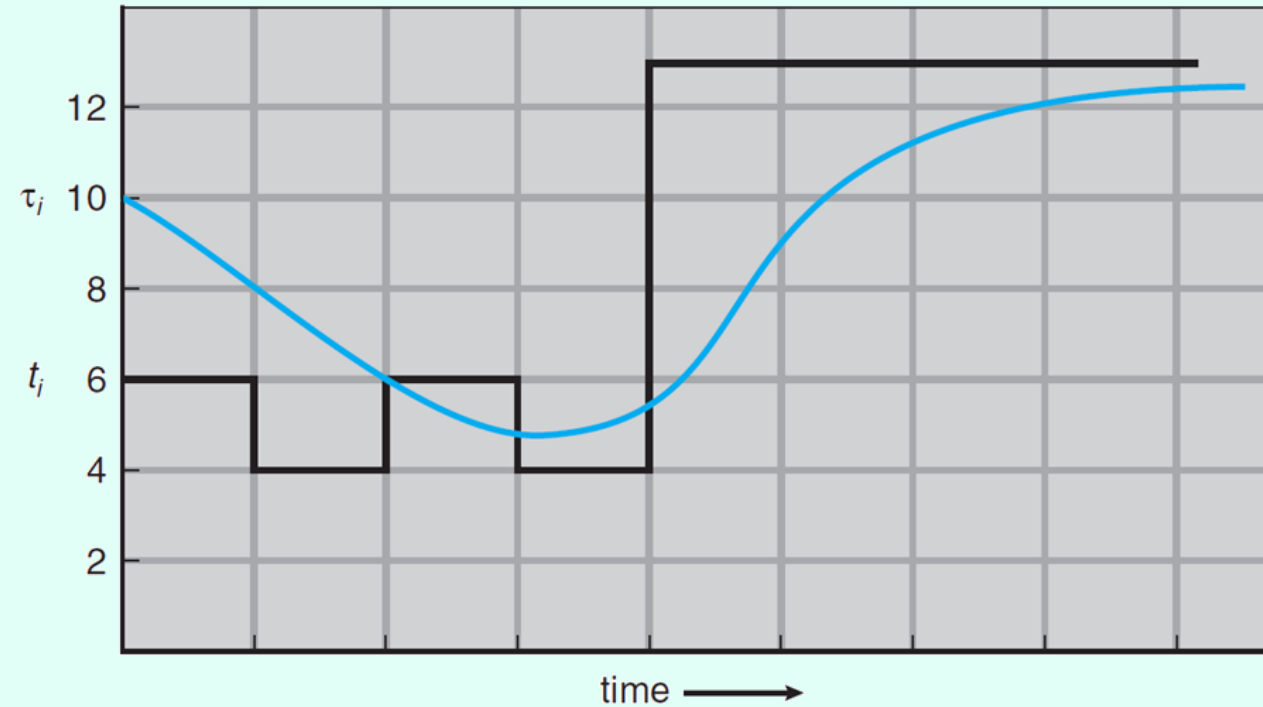
■ به همین منظور، روش‌هایی ارائه می‌شوند تا با در اختیار داشتن طول فرآیندهای گذشته، طول فرآیندهای آینده را پیشبینی کنیم.

از روش میانگین‌گیری نمایی (exponential averaging) برای این کار استفاده می‌کنیم:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \cdots + (1 - \alpha)^j \alpha t_{n-j} + \cdots + (1 - \alpha)^{n+1} \tau_0$$

پیشبینی طول فرآیند بعدی



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Shortest Remaining Time First (SRTF)

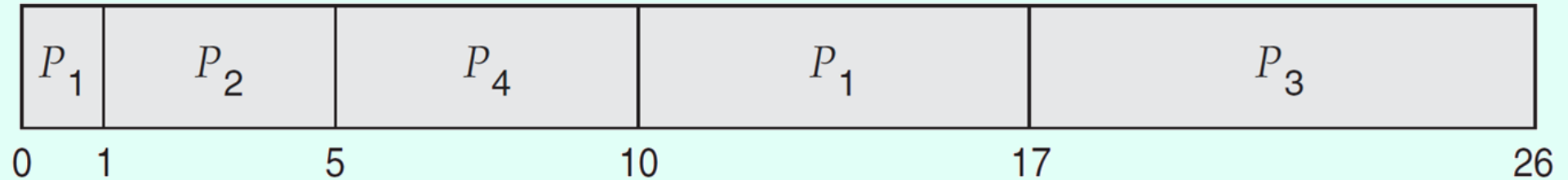
■ این الگوریتم حالت غیرانحصاری (preemptive) الگوریتم SJF است.

دیگر مانند SJF در لحظه صفر طول اجرای تمام فرآیندها معلوم نیست و الگوریتم به طور داینامیک باید زمان‌بندی را انجام دهد.

Shortest Remaining Time First (SRTF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم SRTF، این فرآیندها را زمان‌بندی کنید. زمان انتظار متوسط این الگوریتم را محاسبه کنید.

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5



Priority Scheduling (PS)

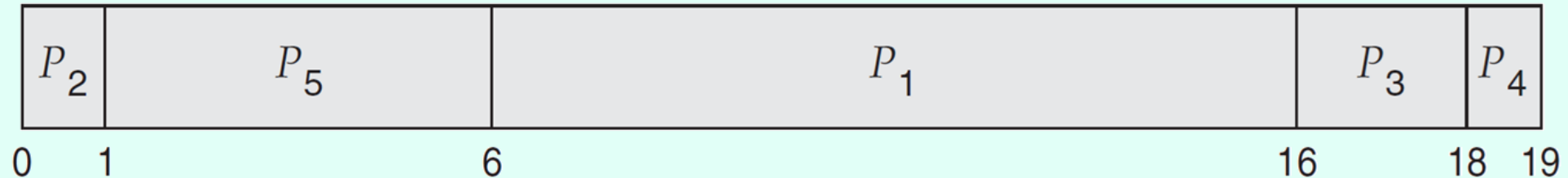
■ هر فرآیند یک اولویت (priority) دارد و بر اساس آن زمان بندی انجام می شود.

اولویت یک عدد است و هرچه کمتر باشد، یعنی اولیت اجرای فرآیند بیشتر است.

Priority Scheduling (PS)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم PS، این فرآیندها را زمان‌بندی کنید. زمان انتظار متوسط این الگوریتم را محاسبه کنید.

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



Round Robin (RR)

■ این الگوریتم مشابه FCFS است با این تفاوت که غیرانحصاری (preemptive) است

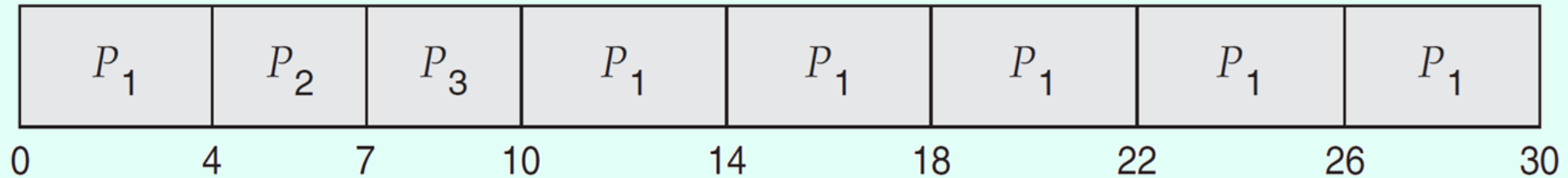
■ واحد کوچکی از زمان به نام time quantum یا time slice تعریف می‌شود و فرآیندها به اندازه این کوانتوم زمانی می‌توانند در CPU اجرا شوند، پس از آن فرآیند دیگری باید اجرا شود.

■ فرآیندها پس از اجرای کوانتوم زمانی، وارد ready queue می‌شوند و صبر می‌کنند تا مجدد نوبت اجرایشان برسد.

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 4 در نظر بگیرید.

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3



Round Robin (RR)

■ در مقایسه با SJF، turnaround time افزایش پیدا می‌کند، ولی response time کمتر می‌شود.

■ کوانتوم زمانی اگر خیلی بزرگ باشد، الگوریتم به FCFS میل می‌کند، و اگر خیلی کوچک باشد بسیار غیربهبود می‌شود.

■ کوانتوم زمانی باید مقدار بیشتری از زمان context switch داشته باشد.

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1					
----	--	--	--	--	--

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1	P1				
---------------	----	--	--	--	--

P1					
----	--	--	--	--	--

0 2

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1	P1	P2	P1		
---------------	---------------	----	----	--	--

P1	P1				
----	----	--	--	--	--

0 2 4

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1	P1	P2	P1	P3	
---------------	---------------	---------------	----	----	--

P1	P1	P2			
----	----	----	--	--	--

0 2 4 6

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1	P1	P2	P1	P3	
---------------	---------------	---------------	---------------	----	--

P1	P1	P2	P1		
----	----	----	----	--	--

0 2 4 6 7

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:

P1	P1	P2	P1	P3	P3
---------------	---------------	---------------	---------------	---------------	----

P1	P1	P2	P1	P3	
----	----	----	----	----	--

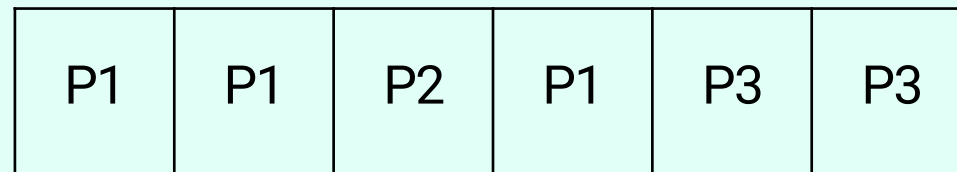
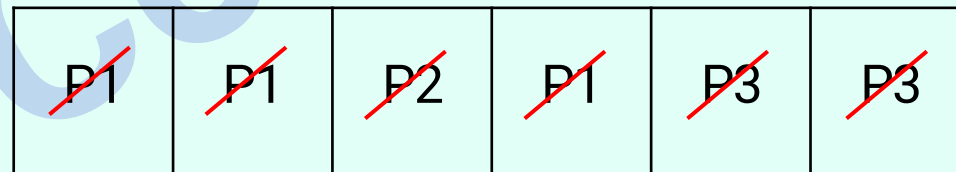
0 2 4 6 7 9

Round Robin (RR)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم RR، این فرآیندها را زمان‌بندی کنید. کوانتوم زمانی را 2 در نظر بگیرید.

Process	Arrival Time	Burst Time
P1	0	5
P2	4	2
P3	5	4

Ready Queue:



0 2 4 6 7 9 11

Multilevel Queue (MLQ)

■ این نوع زمان‌بندی مناسب زمانی است که بتوان فرآیندها را به راحتی به گروه‌های مختلف تقسیم کرد.

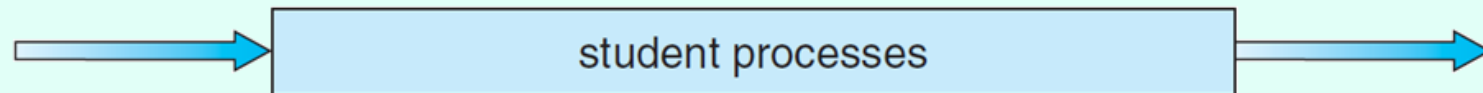
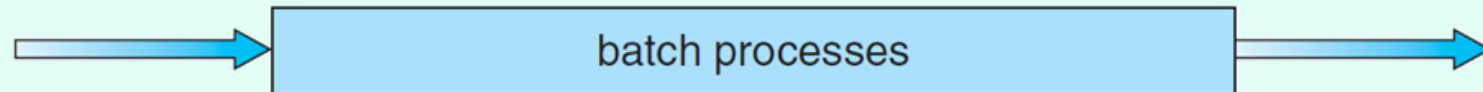
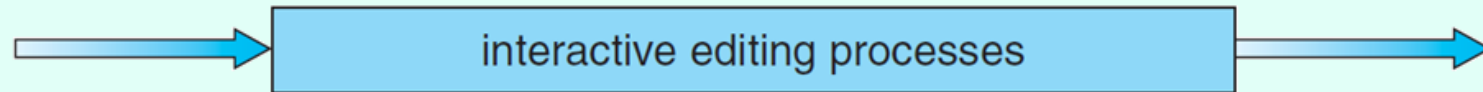
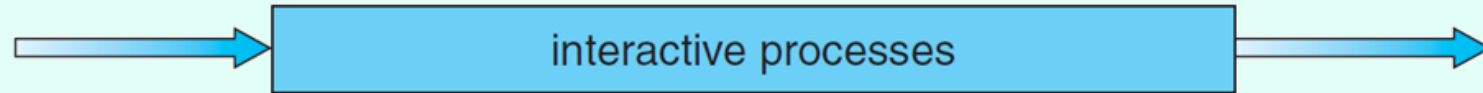
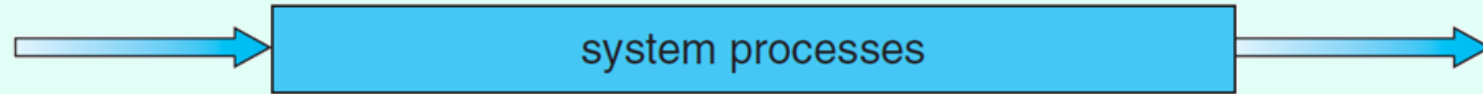
■ برای مثال یک گروه‌بندی رایج، تقسیم فرآیندها به دو گروه foreground (interactive) و background (batch) است. این دو نوع فرآیند نیاز به response time متفاوت و نیازهای زمان‌بندی متفاوتی هستند. به طور کلی اولویت فرآیندهای foreground بیشتر است.

■ در این روش، ready queue به چند صف مجزا تقسیم می‌شود و هر فرآیند با توجه به ویژگی‌هایش در یکی از این صف‌ها قرار می‌گیرد. ویژگی‌هایی مانند حجم حافظه، اولویت فرآیند، نوع فرآیند و ...

■ الگوریتم زمان‌بندی هر صف مخصوص فرآیندهای آن صف است و با بقیه متفاوت است. برای مثال فرآیندهای foreground را با RR و فرآیندهای background را با FCFS زمان‌بندی می‌کنیم.

Multilevel Queue Scheduling (MLQ)

highest priority



lowest priority

Multilevel Feedback Queue (MLFQ)

■ در روش MLQ، هر فرآیند که وارد یک صف می‌شود، تا ابد در همان صف خواهد ماند. ولی اگر ساز و کاری را پیاده کنیم که فرآیندهای بتوانند بین صف‌های مختلف جابجا شوند، می‌توان **aging** را پیاده‌سازی کرد.

aging: مکانیزمی است جهت جلوگیری از starvation. در Priority Scheduling، اگر فرآیندی اولویت کمی داشته باشد و دائماً فرآیندهای جدیدی با اولویت بالا وارد سیستم شوند، فرآیند ما دچار گرسنگی (starvation) می‌شود. برای جلوگیری از این اتفاق، اولویت فرآیند با گذشت زمان بالا می‌رود تا در نهایت اجرا شود.

■ روش MLFQ با در نظر گرفتن جابجایی فرآیندها بین صف‌ها، امکان aging را فراهم می‌کند.

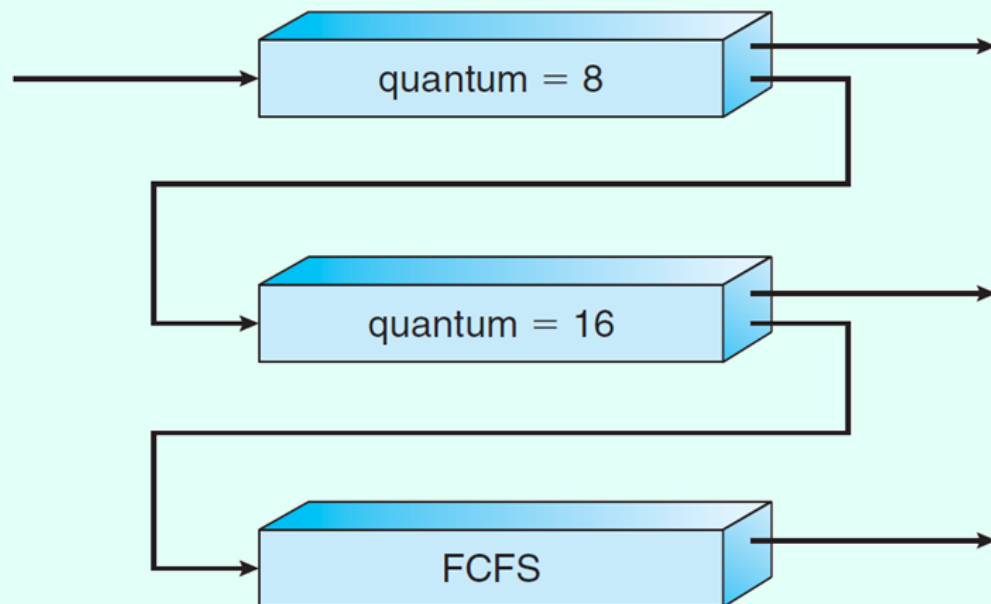
Multilevel Feedback Queue (MLFQ)

■ برای تعریف یک زمان‌بند MLFQ به پارامترهای زیر نیاز داریم:

1. تعداد صف‌ها
2. الگوریتم زمان‌بندی برای هر صف
3. روشی برای تغییر یک فرآیند به صفی با اولویت بالاتر/پایین‌تر
4. روشی برای تشخیص اینکه یک فرآیند هنگام ورود به سیستم، وارد کدام صف شود

Multilevel Feedback Queue (MLFQ)

مثال) 3 صف در اختیار داریم که به ترتیب با $RR(q=8)$ ، $RR(q=16)$ و FCFS فرآیندها را زمان‌بندی می‌کنند. فرآیند جدید ابتدا وارد صف بالا با الگوریتم $RR(q=8)$ می‌شود. اگر این کوانتوم برایش کافی نبود به صف دوم با الگوریتم $RR(q=16)$ می‌شود و در نهایت اگر این کوانتوم هم برایش کافی نبود، وارد صف آخر با الگوریتم FCFS می‌شود.



Real-Time CPU Scheduling

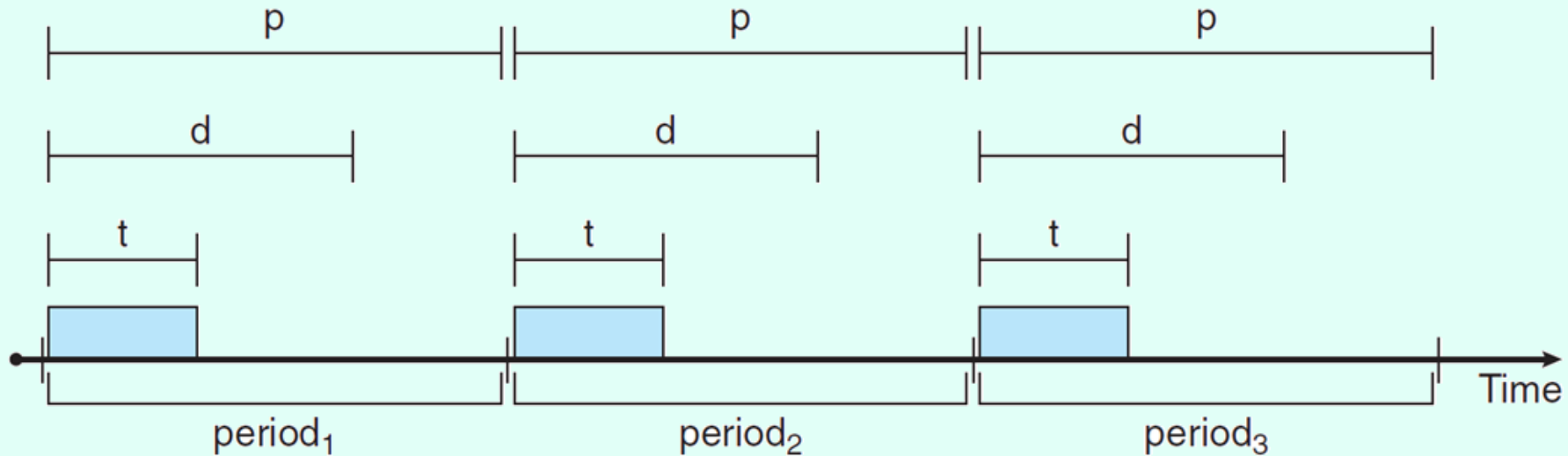
■ تا الان فرآیندهایی را که زمان‌بندی می‌کردیم، ضرب الاجل (deadline) نداشتند. ددلاین یعنی فرآیند تا یک زمانی مهلت اجرا شدن دارد و اگر این زمان بگذرد، فرآیند از دست می‌رود (miss می‌شود).

Soft Real-Time System: سیستمی که در آن از دست رفتن ددلاین فرآیند زیاد مهم نیست و خسارت زیادی ندارد.

Hard Real-Time System: سیستمی که در آن هر فرآیند حتماً باید قبل از ددلاین به طور کامل اجرا شود در غیر این صورت خسارت زیادی وارد خواهد شد.

Periodic Task

■ فرآیندها همچنین می‌توانند متناوب (Periodic) باشند. یعنی به طور مداوم هر چند وقت یکبار وارد سیستم شده و نیاز به اجرا شدن داشته باشند.



$$0 \leq t \leq d \leq p$$

Rate Monotonic (RM)

■ این الگوریتم مخصوص زمان‌بندی فرآیندهای متناوب است.

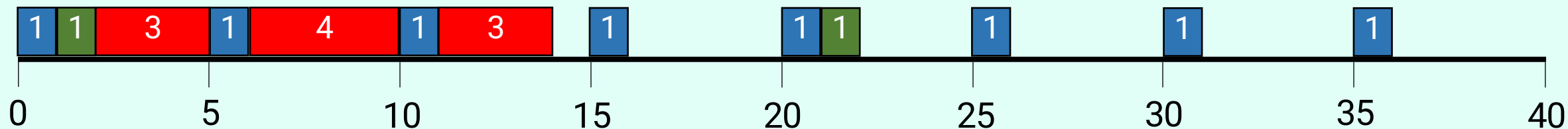
■ این الگوریتم بر اساس اولویت است، به این صورت که فرآیندها هرچه تناوبشان بیشتر باشد، اولویتشان کمتر است و برعکس.

مثال) تناوب فرآیند P1 برابر 35 میلی ثانیه و تناوب فرآیند P2 برابر 20 میلی ثانیه است. اولویت کدام فرآیند بیشتر است؟

Rate Monotonic (RM)

مثال) مجموعه فرآیندهای زیر در جدول مشخص شده‌اند. با الگوریتم RM آن‌ها را زمان‌بندی کنید.

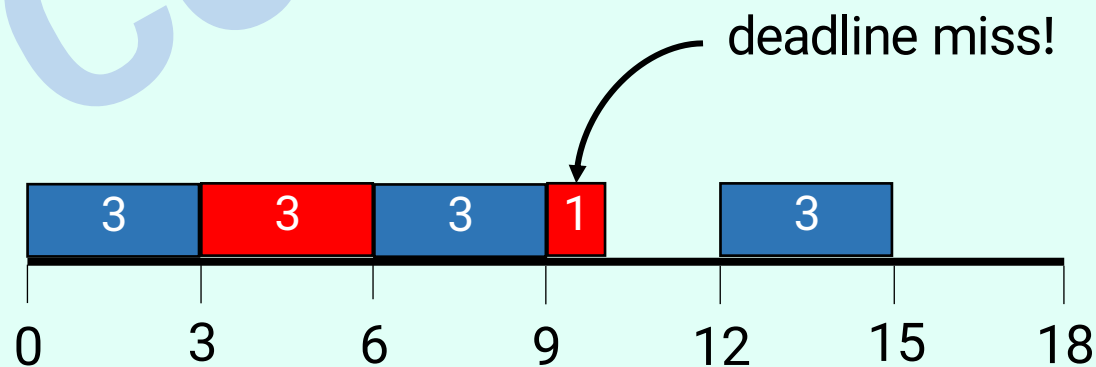
Process	Period	Burst Time	Deadline
P1	5	1	5
P2	40	10	15
P3	20	1	20



Rate Monotonic (RM)

مثال) مجموعه فرآیندهای زیر در جدول مشخص شده‌اند. با الگوریتم RM آن‌ها را زمان‌بندی کنید.

Process	Period	Burst Time	Deadline
P1	6	3	6
P2	9	4	9



Rate Monotonic (RM)

■ الگوریتم RM زمانی می‌تواند زمان‌بندی N فرآیند را بدون از دست رفتن ددلاینی تضمین کند، که میزان استفاده از CPU (CPU Utilization) از مقدار زیر کمتر باشد یا با آن برابر باشد:

$$N(2^{1/N} - 1)$$

مثال) میزان استفاده از CPU باید چقدر باشد تا مطمئن شویم الگوریتم RM می‌تواند 4 فرآیند را زمان‌بندی موفق کند؟

Earliest Deadline First (EDF)

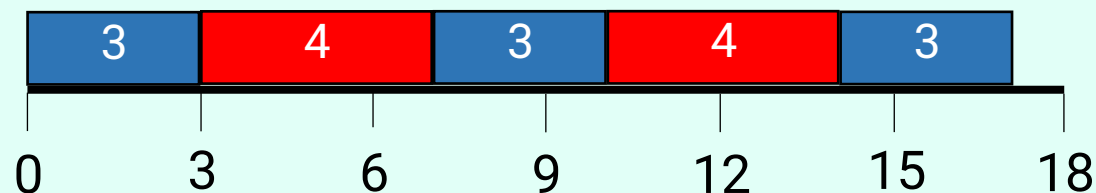
■ این الگوریتم هم مناسب زمان بندی فرآیندهای متناوب است.

■ این الگوریتم بر اساس ددلاین است، به این صورت که فرآیندها هرچه ددلاینشان نزدیکتر باشد، اولویتشان بیشتر است.

Earliest Deadline First (EDF)

مثال) مجموعه فرآیندهای زیر در جدول مشخص شده‌اند. با الگوریتم EDF آن‌ها را زمان‌بندی کنید.

Process	Period	Burst Time	Deadline
P1	6	3	6
P2	9	4	9



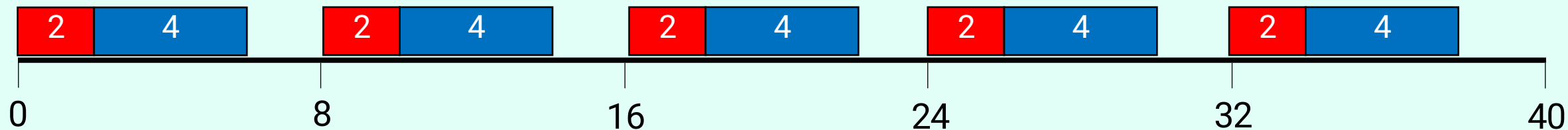
Proportional Share Scheduling

- این الگوریتم هم مناسب زمان بندی فرآیندهای متناوب است.
- این الگوریتم بر اساس میزان استفاده هر فرآیند از CPU زمان بندی را انجام می دهد.
- الگوریتم واحدهای زمانی به اندازه $s = \gcd(\forall_i, P_i)$ ایجاد می کند، و در هر واحد زمانی، هر فرآیند به اندازه $utilization * s$ اجرا می شود.

Proportional Share Scheduling

مثال) مجموعه فرآیندهای زیر در جدول مشخص شده‌اند. با الگوریتم Proportional Share آن‌ها را زمان‌بندی کنید.

Process	Period	Burst Time	Deadline
P1	40	20	40
P2	16	4	16



Lottery Scheduling

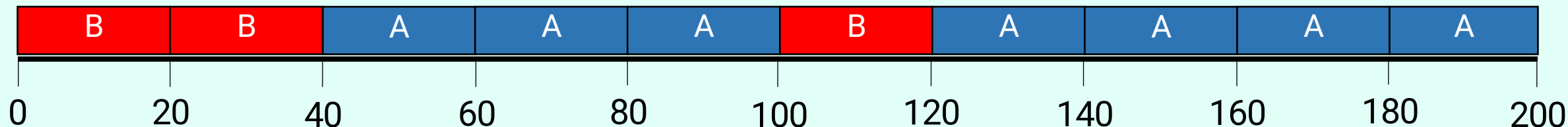
■ در این الگوریتم مانند الگوریتم RR یک واحد زمانی داریم که فرآیندها در این واحد زمانی اجرا می‌شوند. به هر فرآیند تعدادی Lottery Ticket می‌دهیم و هر بار یک Ticket را انتخاب می‌کنیم و فرآیندی که آن Ticket را در اختیار دارد، در واحد زمانی اجرا می‌شود.

■ اگر تعداد اجراها را بالا ببریم، هر فرآیند با توجه به تعداد ticketهایی که در اختیار دارد، شانس اجرا در CPU را خواهد داشت.

Lottery Scheduling

مثال) دو فرآیند A و B را در نظر بگیرید. می‌خواهیم با الگوریتم Lottery این فرآیندها را زمان‌بندی کنیم. فرآیند A از 1 تا 60 بلیط دارد و فرآیند B از 61 تا 100 بلیط دارد. اگر زمان اجرای فرآیندها هر بار 20 میلی ثانیه باشد، با توجه به اعداد تصادفی تولید شده در شکل زیر، این فرآیندها را زمان‌بندی کنید.

Ticket Number: 73, 82, 23, 45, 32, 87, 49, 39, 12, 9



Highest Response Ratio Next (HRRN)

■ این الگوریتم بر اساس میزان انتظار فرآیندها، اولویت هر فرآیند را تعیین می‌کند:

$$P_i = \frac{w_i + c_i}{c_i}$$

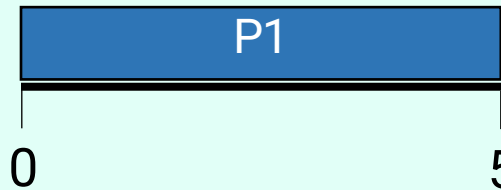
$w_i = \text{wait time}$, $c_i = \text{burst time}$

■ این الگوریتم به خوبی مکانیزم aging را پیاده‌سازی می‌کند و از starvation فرآیندها جلوگیری می‌کند.

Highest Response Ratio Next (HRRN)

(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6



Highest Response Ratio Next (HRRN)

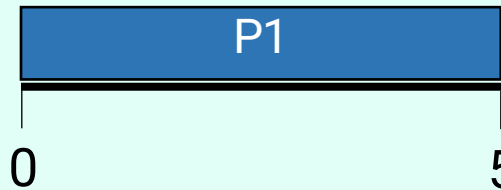
(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

$$P_2 = \frac{4 + 3}{3} = 2.33$$

$$P_3 = \frac{3 + 8}{8} = 1.375$$

$$P_4 = \frac{2 + 6}{6} = 1.33$$



Highest Response Ratio Next (HRRN)

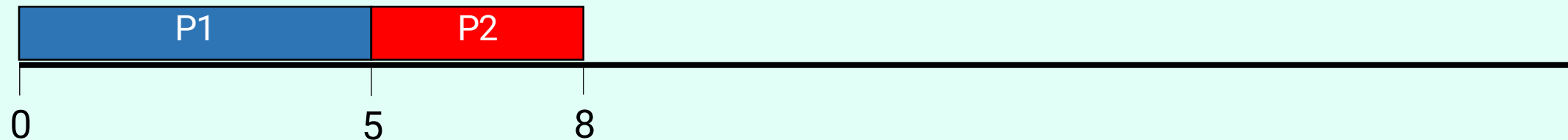
(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

$$P_2 = \frac{4 + 3}{3} = 2.33$$

$$P_3 = \frac{3 + 8}{8} = 1.375$$

$$P_4 = \frac{2 + 6}{6} = 1.33$$



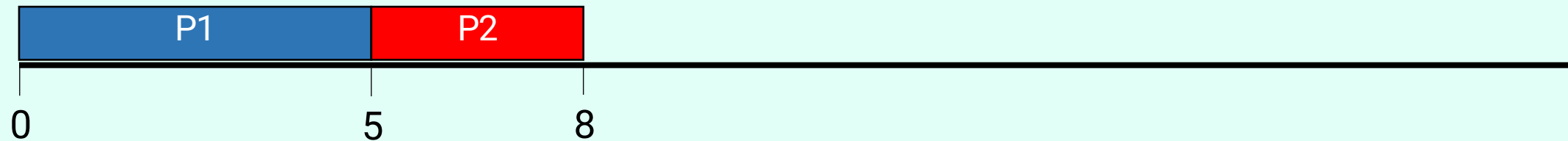
Highest Response Ratio Next (HRRN)

(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

$$P_3 = \frac{6 + 8}{8} = 1.75$$

$$P_4 = \frac{5 + 6}{6} = 1.83$$



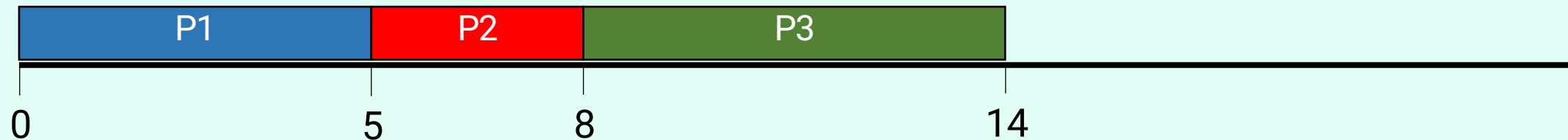
Highest Response Ratio Next (HRRN)

(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

$$P_3 = \frac{6 + 8}{8} = 1.75$$

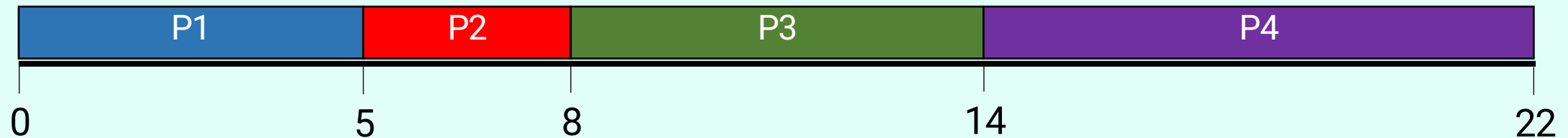
$$P_4 = \frac{5 + 6}{6} = 1.83$$



Highest Response Ratio Next (HRRN)

(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم HRRN، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6



Least Laxity First (LLF)

■ این الگوریتم مخصوص زمان‌بندی سیستم‌های real-time است (دارای ددلاین برای فرآیندها)

■ در هر زمان محاسبه می‌کنیم که هر فرآیند تا رسیدن زمان ددلاین خود، چه مقدار می‌تواند اجرایش را به تاخیر بیندازد، و این مقدار را ℓ می‌نامیم. فرآیند با کمترین ℓ بیشترین اولویت را دارد.

$$\ell_i = d_i - t - r_i$$

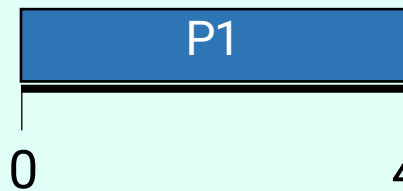
$$d_i = \text{deadline}, t = \text{current time}, r_i = \text{remaining time}$$

■ نام دیگر این الگوریتم (LST) Least Slack Time است.

Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29



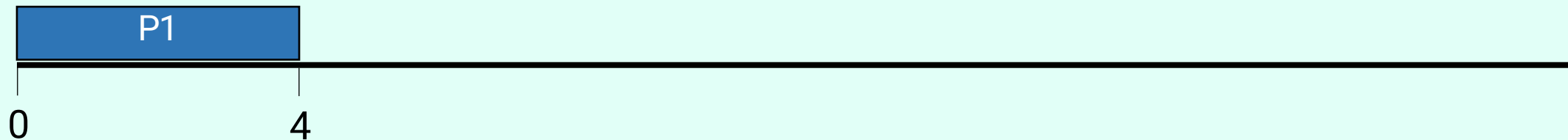
Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 4 - 6 = 23$$

$$\ell_2 = 28 - 4 - 3 = 21$$



Least Laxity First (LLF)

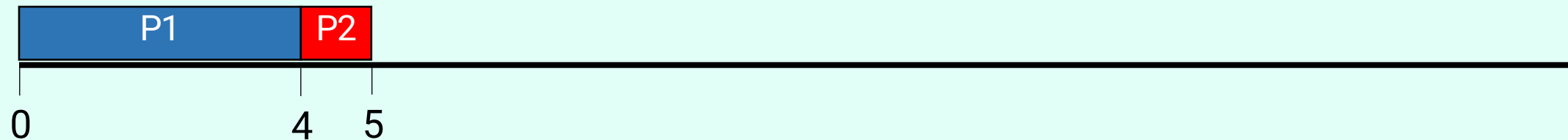
مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 5 - 6 = 22$$

$$\ell_2 = 28 - 5 - 2 = 21$$

$$\ell_3 = 29 - 5 - 10 = 14$$



Highest Response Ratio Next (HRRN)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 5 - 6 = 22$$

$$\ell_2 = 28 - 5 - 2 = 21$$

$$\ell_3 = 29 - 5 - 10 = 14$$



Least Laxity First (LLF)

(مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 13 - 6 = 14$$

$$\ell_2 = 28 - 13 - 2 = 13$$

$$\ell_3 = 29 - 13 - 2 = 14$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 13 - 6 = 14$$

$$\ell_2 = 28 - 13 - 2 = 13$$

$$\ell_3 = 29 - 13 - 2 = 14$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 15 - 6 = 12$$

$$\ell_3 = 29 - 15 - 2 = 12$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 15 - 6 = 12$$

$$\ell_3 = 29 - 15 - 2 = 12$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 16 - 6 = 11$$

$$\ell_3 = 29 - 16 - 1 = 12$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 16 - 6 = 11$$

$$\ell_3 = 29 - 16 - 1 = 12$$



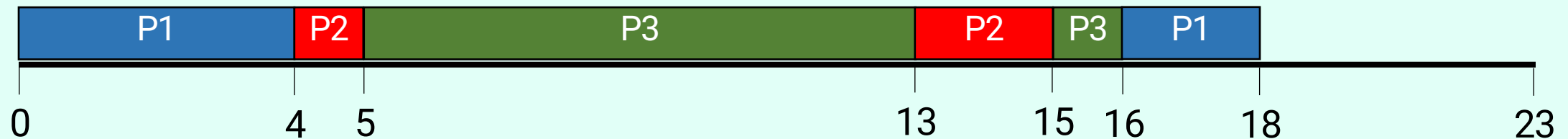
Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 18 - 4 = 11$$

$$\ell_3 = 29 - 18 - 1 = 10$$



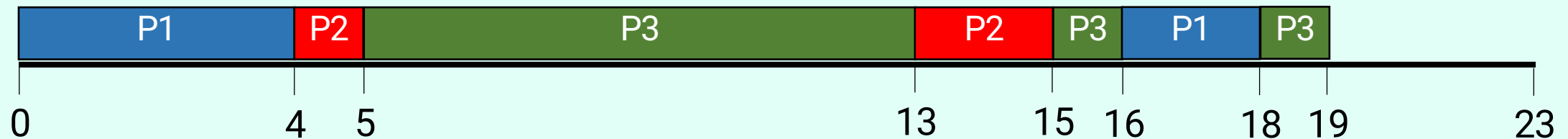
Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29

$$\ell_1 = 33 - 18 - 4 = 11$$

$$\ell_3 = 29 - 18 - 1 = 10$$



Least Laxity First (LLF)

مثال) مجموعه فرآیندهای زیر را در نظر بگیرید. با استفاده از الگوریتم LLF، این فرآیندها را زمان‌بندی کنید.

Process	Arrival Time	Burst Time	Deadline
P1	0	10	33
P2	4	3	28
P3	5	10	29



ارزیابی الگوریتم (Algorithm Evaluation)

■ در این فصل با الگوریتم‌های زمان‌بندی متعددی آشنا شدیم. سوالی که مطرح می‌شود این است که چه الگوریتم مناسب است؟

■ قاعده‌تاً علاقه‌مندیم الگوریتمی را انتخاب کنیم که بهترین عملکرد را میان قیود زمان‌بندی داشته باشد (به اسلاید مراجعه کنید 160)

■ برای اینکه چگونه با توجه به نیازهای سیستم و قیود زمان‌بندی، الگوریتم مناسب را انتخاب کنیم، روش‌های مختلفی وجود دارد که با آن‌ها آشنا می‌شویم:

1. مدل‌سازی قطعی (Deterministic Modeling)

2. مدل‌های صف‌بندی (Queueing Models)

3. شبیه‌سازی (Simulation)

4. پیاده‌سازی (Implementation)

مدل سازی قطعی (Deterministic Modeling)

■ یک کلاس مهم ارزیابی الگوریتم، کلاس ارزیابی تحلیلی (Analytic Evaluation) است. در این روش، هر الگوریتم بر روی مجموعه فرآیندها اجرا می شود، و بهینگی آن با عددی توصیف می شود. با عدد به دست آمده، می توان بهینه ترین الگوریتم را پیدا کرد.

■ در روش مدل سازی قطعی که نوعی ارزیابی تحلیلی است، تمام الگوریتم های نامزد بر روی فرآیندها اجرا شده و با یکی از قیود زمان بندی مثل waiting time، بهینگی الگوریتم ها تعیین می شود.

مدل سازی قطعی (Deterministic Modeling)

مثال) می‌خواهیم فرآیندهای زیر را با یکی از الگوریتم‌های FCFS، SJF و RR ($q=10$) زمان‌بندی کنیم. با روش مدل‌سازی قطعی و با استفاده از معیار waiting time، بهترین الگوریتم را پیدا کنید.

<u>Process</u>	<u>Burst Time</u>
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

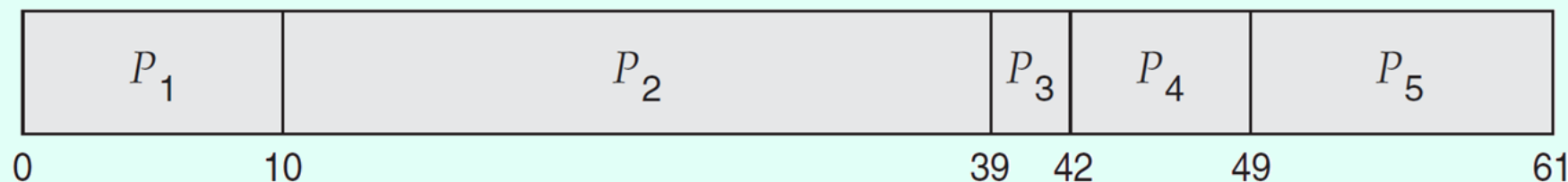
مدل سازی قطعی (Deterministic Modeling)

مثال) می‌خواهیم فرآیندهای زیر را با یکی از الگوریتم‌های FCFS، SJF و RR ($q=10$) زمان‌بندی کنیم. با روش مدل‌سازی قطعی و با استفاده از معیار waiting time، بهترین الگوریتم را پیدا کنید.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

$$\frac{0 + 10 + 39 + 42 + 49}{5} = 28$$

FCFS



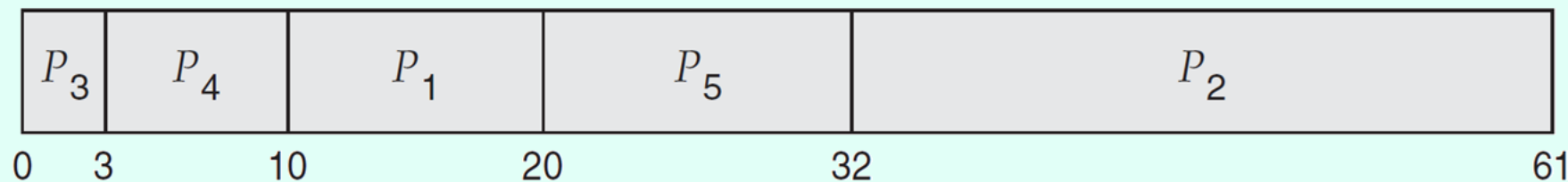
مدل سازی قطعی (Deterministic Modeling)

مثال) می‌خواهیم فرآیندهای زیر را با یکی از الگوریتم‌های FCFS، SJF و RR ($q=10$) زمان‌بندی کنیم. با روش مدل‌سازی قطعی و با استفاده از معیار waiting time، بهترین الگوریتم را پیدا کنید.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

$$\frac{10 + 32 + 0 + 3 + 20}{5} = 13$$

SJF



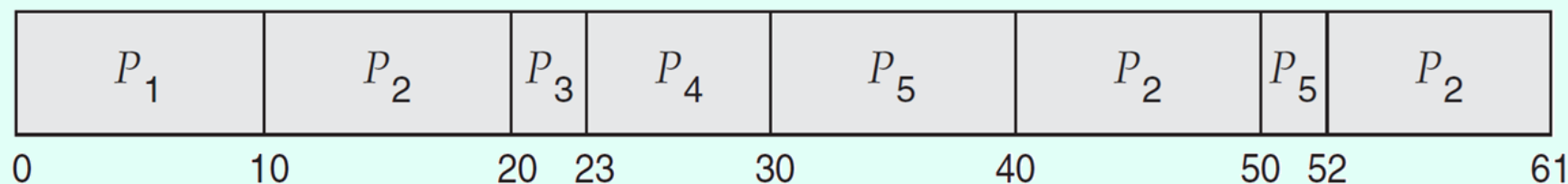
مدل سازی قطعی (Deterministic Modeling)

مثال) می‌خواهیم فرآیندهای زیر را با یکی از الگوریتم‌های FCFS، SJF و RR ($q=10$) زمان‌بندی کنیم. با روش مدل‌سازی قطعی و با استفاده از معیار waiting time، بهترین الگوریتم را پیدا کنید.

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

$$\frac{0 + 32 + 20 + 23 + 40}{5} = 23$$

RR



مدل سازی قطعی (Deterministic Modeling)

مثال) می‌خواهیم فرآیندهای زیر را با یکی از الگوریتم‌های FCFS، SJF و RR ($q=10$) زمان‌بندی کنیم. با روش مدل‌سازی قطعی و با استفاده از معیار waiting time، بهترین الگوریتم را پیدا کنید.

<u>Process</u>	<u>Burst Time</u>
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

بهترین عملکرد برای SJF است با $\text{waiting time} = 13$

مدل‌های صف‌بندی (Queueing Models)

- در بسیاری از سیستم‌ها، فرآیندها دائماً در حال تغییر و عوض شدن هستند و ثابت نیستند که بتوان با روش مدل‌سازی قطعی بهترین الگوریتم را پیدا کرد.
- در این سیستم‌ها، با توجه به داده‌های گذشته، توزیع CPU Burst و I/O Burst محاسبه می‌شود و CPU Burst هر فرآیند با توجه به این توزیع، پیشبینی می‌شود.
- معمولاً توزیع CPU Burst از نوع توزیع نمایی (Exponential Distribution) است و میانه به عنوان CPU Burst بعدی پیشبینی می‌شود.
- با استفاده از این توزیع‌ها، می‌توان قیود زمان‌بندی را به طور متوسط محاسبه کرد.

مدل‌های صف‌بندی (Queueing Models)

■ سیستم کامپیوتری به عنوان شبکه‌ای از سرورها توصیف می‌شود. هر سرور یک صف برای فرآیندهای waiting دارد. CPU نیز یک سرور با صف ready است و سیستم I/O نیز سروری با صف‌های device است.

■ با دانستن نرخ arrival و نرخ service، می‌توان utilization، طول متوسط صف، waiting time متوسط و ... را محاسبه کرد. به این کار تحلیل شبکه صف‌بندی (queueing-network analysis) گوییم.

■ برای مثال little's formula یکی از معادله‌هایی است که کمک می‌کند waiting time را محاسبه کنیم:

$$n = \lambda \times W$$

n : average queue length

λ : average arrival rate

W : average waiting time

مدل‌های صف‌بندی (Queueing Models)

مثال) می‌دانیم در سیستمی به طور متوسط هر ثانیه 7 فرآیند از راه می‌رسند و به طور متوسط 14 فرآیند در صف حضور دارند. Waiting time متوسط را بیابید.

$$n = \lambda \times W$$

$$n = 14$$

$$\lambda = 7$$

$$\rightarrow W = \frac{14}{7} = 2$$

مدل‌های صف‌بندی (Queueing Models)

- مدل‌های صف‌بندی مناسب مقایسه الگوریتم‌های مختلف هستند، ولی محدودیت‌های ریاضیاتی دارند. تعداد الگوریتم‌هایی که با این روش قابل تحلیل هستند بسیار محدود است.
- دقت تحلیل این الگوریتم‌ها به دلیل فرض‌های زیاد، سوال برانگیز است.

شبیه‌سازی (Simulation)

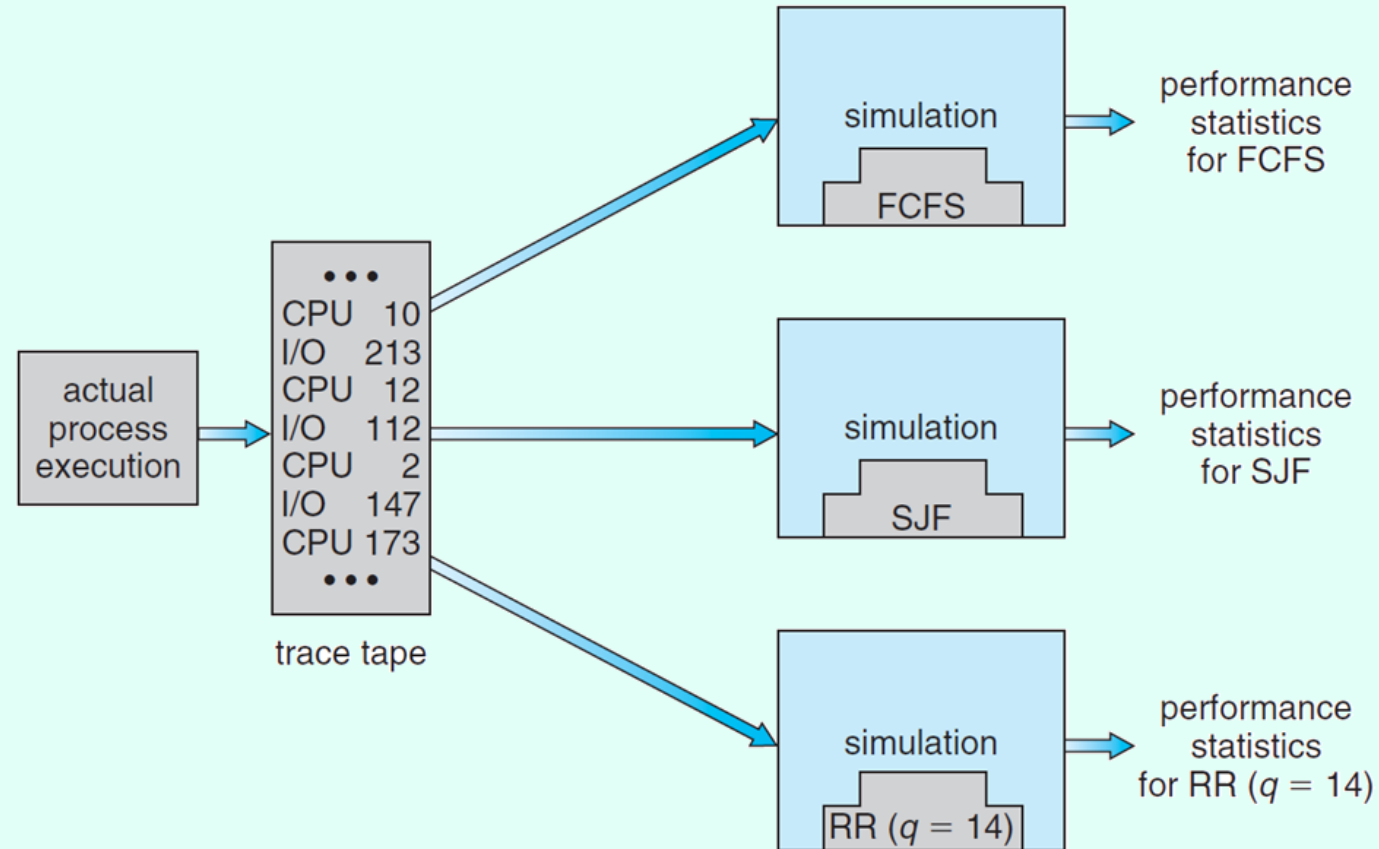
■ در این روش مدلی از سیستم کامپیوتری ساخته می‌شود و با متغیری به نام clock، الگوریتم‌های مختلف روی این مدل شبیه‌سازی می‌شوند.

■ نتیجه حاصل شبیه‌سازی آمار و اطلاعات مختلف حاصل از هر الگوریتم است.

■ روش‌های مختلفی برای رساندن اطلاعات به شبیه‌ساز وجود دارد. یک روش تولید داده تصادفی است. یک روش ایجاد داده جدید بر اساس داده قدیمی است، و یک روش monitor کردن سیستم و استفاده از همان داده است که به آن trace tapes گوییم.

■ شبیه‌سازی نتایج دقیق و قابل اعتمادی تولید می‌کند، ولی زمان اجرای بالایی دارد و بسیار کند است.

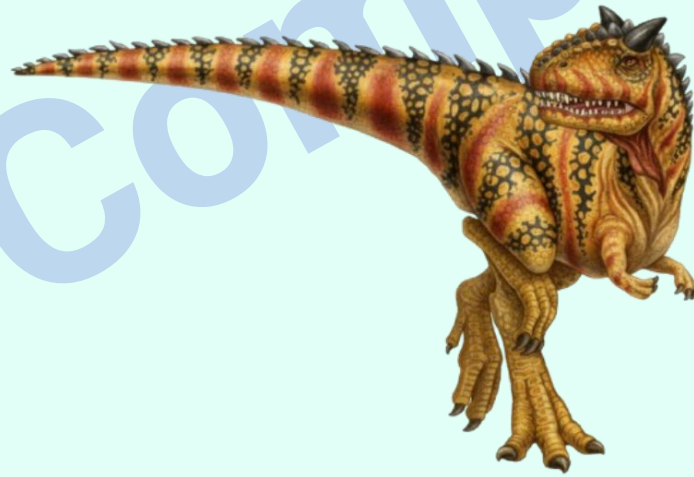
شبیه‌سازی (Simulation)



پیاده‌سازی (Implementation)

- دقیق‌ترین روش برای مقایسه الگوریتم‌ها، اجرای آن‌ها روی سیستم عامل به معنای واقعی کلمه است.
- در این روش به دقیق‌ترین شکل ممکن می‌توان عملکرد الگوریتم‌ها را مقایسه کرد، ولی این روش بسیار گران است.
- این روش نیاز به تغییرات زیاد سیستم عامل به طور مداوم دارد که هم ریسک بالایی دارد و هم کاربران این موضوع را نمی‌پسندند.

Chapter 7: Deadlocks



بن بست (Deadlock)

■ در یک سیستم multiprogramming، چندین فرآیند برای تعداد محدودی منابع با هم رقابت می کنند. هر فرآیند برای منبع مورد نظر خود درخواست می کند، اگر منبع مورد نظر در دسترس نباشد، فرآیند وارد حالت waiting می شود. اگر این فرآیند دیگر نتواند از حالت waiting خارج شود، گوییم بن بست (Deadlock) پیش آمده.

■ در واقع منبع مورد نیاز فرآیند ما، خود در اختیار فرآیند دیگری است، که آن فرآیند دوم در انتظار منبع دیگری است که آن را هیچ وقت به دست نمی آورد، بنابراین منبع خود را هیچ وقت آزاد نمی کند.

مثال) در فصل 5 (همگام سازی)، یک نمونه بن بست در مسئله فیلسوف ها رخ میداد. زمانی که هر فیلسوف دقیقاً چاپستیک برمیداشت. در این حالت هر فیلسوف منتظر چاپستیک دیگری است و چاپستیک خود را زمین نمی گذارد.

مدل سیستم (System Model)

■ منظور از منبع، سایکل پردازنده، فایل، دستگاه I/O و ... است.

■ یک فرآیند قبل از استفاده از منبع، باید برای آن درخواست (Request) دهد، و پس از پایان کارش، آن را آزاد کند (Release). بنابراین می‌توان تعامل فرآیند با منبع را به 3 حالت زیر تقسیم‌بندی کرد:

درخواست (Request)

استفاده (Use)

آزادسازی (Release)

■ درخواست و آزادسازی می‌توانند از طریق system call ها انجام شوند، مانند request() و release()، یا allocate() و free(). یا از طریق عملیات‌های همگام‌سازی مانند wait() و signal() انجام شوند.

مشخصات بن بست

■ بن بست زمانی رخ می دهد که 4 شرط زیر همزمان برقرار باشند:

1. **انحصار متقابل (Mutual Exclusion):** فقط یک فرآیند در هر زمان می تواند از هر منبع استفاده کند.
2. **برداشتن و صبر کردن (Hold and Wait):** یک فرآیند حداقل یک منبع را در اختیار دارد و منتظر منبع دیگری است که در اختیار فرآیند دیگری است.
3. **انحصار (No Preemption):** یک منبع تنها زمانی آزاد می شود که یا خود فرآیند داوطلبانه آن را آزاد کند یا اجرائیش تمام شود و آن را آزاد کند.
4. **صبر دایره ای (Circular Wait):** مجموعه ای از فرآیندها مانند $\{P_0, P_1, \dots, P_n\}$ وجود دارد که P_0 منتظر منبعی است که در اختیار P_1 است، P_1 منتظر منبعی است که در اختیار P_2 است و ... در نهایت P_n منتظر منبعی است که در اختیار P_0 است.

گراف منبع-تخصیص (Resource-Allocation)

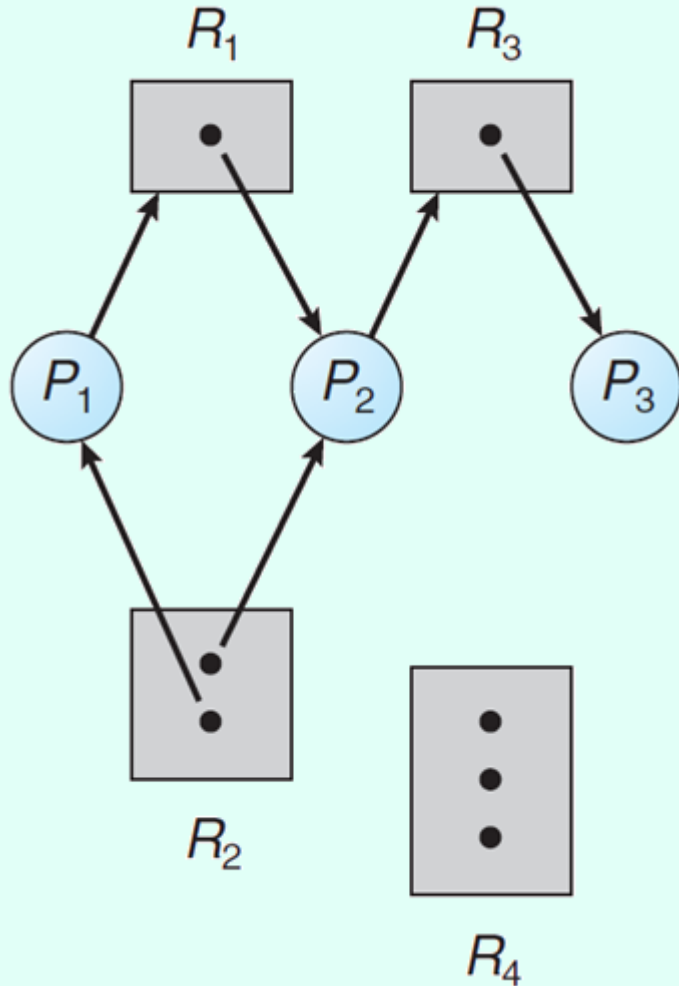
■ از این گراف جهت دار برای نشان دادن وضعیت فرآیندها، منابع، درخواستها و تخصیصها استفاده می شود

رئوس گراف: در این گراف رأس یا فرآیند است، یا منبع. همچنین در رأسهای منبع، تعداد نمونه های آن منبع مشخص می شود.

یالهای گراف: در این گراف یال یا از فرآیند به منبع است، که نشان دهنده درخواست است، یا از منبع به فرآیند است، که نشان دهنده تخصیص است.

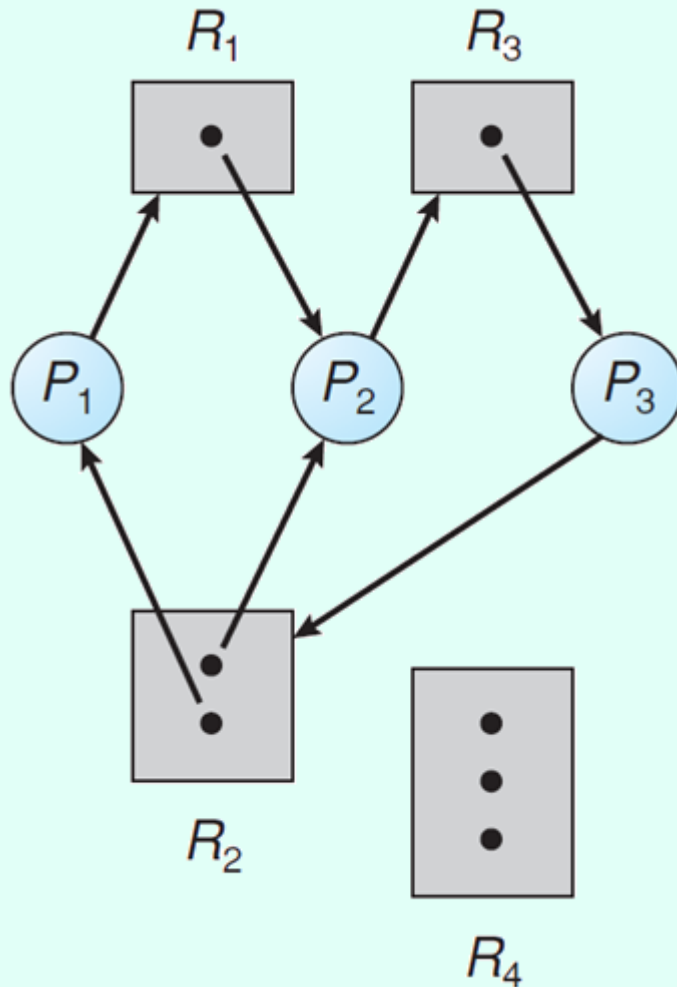
گراف منبع-تخصیص (Resource-Allocation)

آیا در سیستم زیر، ددلاک رخ می‌دهد؟

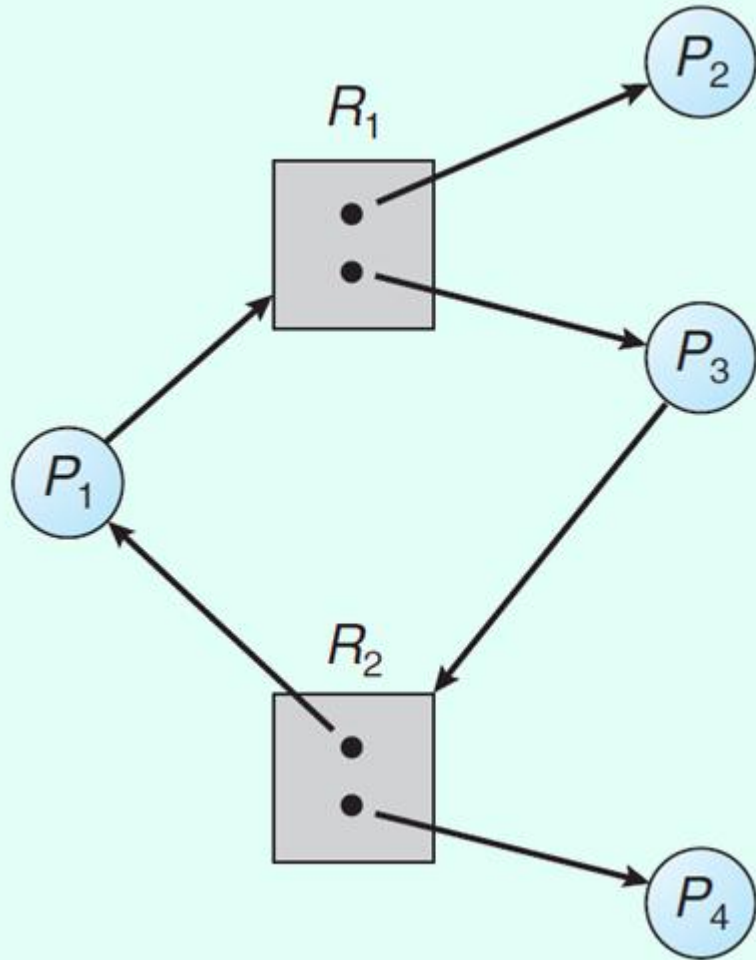


گراف منبع-تخصیص (Resource-Allocation)

در سیستم زیر چطور؟



گراف منبع-تخصیص (Resource-Allocation)



در سیستم زیر حلقه داریم.
آیا ددلاک رخ می‌دهد؟

هندل کردن بن بست (Handling Deadlock)

■ برای مدیریت بن بست، سه روش وجود دارد:

1. از آنجایی که احتمال وقوع ددلاک کم است، آن را نادیده گرفته و فرض کنیم سیستم هیچگاه دچار ددلاک نمی شود.

2. از پروتکلی استفاده کنیم که مانع ورود سیستم به ددلاک شود.

3. اجازه دهیم سیستم وارد ددلاک شود، آن را شناسایی و رفع کنیم.

نکته جالب این است که بسیاری از سیستم عامل ها مانند ویندوز و لینوکس از روش 1 استفاده می کنند! در این سیستم عامل ها رسیدگی به ددلاک وظیفه توسعه دهنده اپلیکیشن است.

هندل کردن بن بست (Handling Deadlock)

در صورت استفاده از روش 2، باید یکی از دو راه حل جلوگیری از بن بست (deadlock prevention) یا اجتناب از بن بست (deadlock avoidance) را استفاده کنیم.

Deadlock Prevention: مطمئن شویم حداقل یکی از 4 شرط ایجاد ددلاک، برقرار نباشد. در این روش بررسی می‌کنیم که هر درخواست برای منابع به چه صورتی ایجاد می‌شوند.

Deadlock Avoidance: در این روش سیستم عامل به اطلاعات اضافه نیاز دارد. اطلاعاتی مانند اینکه هر فرآیند در طول عمرش برای چه منابعی درخواست می‌دهد. سپس برای هر درخواست هر فرآیند، بررسی می‌کند که آیا این تخصیص باعث ایجاد ددلاک خواهد شد یا خیر.

Deadlock Prevention

- **Mutual Exclusion:** برای برقرار نبودن این شرط، هیچ منبعی نباید غیرقابل اشتراک باشد. و این موضوع ممکن نیست. برای مثال mutex lock را نمی‌توان به صورت shareable استفاده کرد.
- **Hold and Wait:** برای برقرار نبودن این شرط، یا هر فرآیند باید قبل از شروع اجرا تمام منابع خود را دریافت کند، یا فقط زمانی منبع جدیدی بگیرد که منبع قبلی خود را آزاد کند. جفت روش‌ها دو ایراد بزرگ دارند: cpu utilization بسیار کم می‌شود و احتمال starvation بالا می‌رود.
- **No Preemption:** اگر فرآیند برای منبعی درخواست داد و آن منبع در دسترس نبود، تمام منابع در اختیار فرآیند آزاد می‌شوند. یا فرآیندی که آن منبع را در اختیار دارد چک می‌کنیم، اگر خود آن فرآیند منتظر منبع دیگری بود، منابعش را آزاد می‌کنیم و به فرآیند خود می‌دهیم، در غیر این صورت فرآیند باید صبر کند.
- **Circular Wait:** به هر منبع یک عدد اختصاص می‌دهیم، و فرآیندها منابع را فقط به صورت صعودی (بر اساس عدد منابع) می‌توانند انتخاب کنند. به این صورت هیچگاه حلقه‌ای ایجاد نمی‌شود.

Deadlock Avoidance

در این روش سیستم عامل اطلاعات اضافه‌ای را قبل از اجرای فرآیندها در اختیار دارد. اطلاعاتی مانند منابع مورد نیاز هر فرآیند و تعداد هر کدام از منابع. با این اطلاعات، می‌توان الگوریتم‌هایی پیاده‌سازی کرد قبل از اجرا می‌توان فهمید آیا سیستم با این مشخصات، دچار بن‌بست می‌شود یا خیر.

در ادامه با الگوریتم بانک‌دار (Banker's Algorithm) آشنا می‌شویم که دقیقاً نمونه‌ای از همین الگوریتم‌ها است.

حالت ایمن (Safe State)

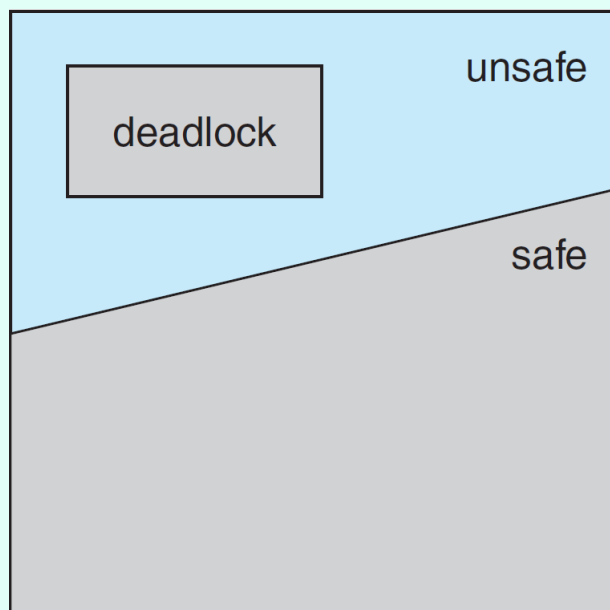
منظور از حالت ایمن، وضعیتی است که در آن سیستم دچار ددلاک نمی‌شود. یک سیستم در حالت ایمن است، اگر و تنها اگر برای آن دنباله ایمنی (Safe Sequence) وجود داشته باشد.

دنباله $\langle P_1, P_2, \dots, P_n \rangle$ را در نظر بگیرید. این دنباله ایمن است اگر فرآیندی مانند P_i درخواست منابعی بکند که برخی از آن‌ها در اختیار P_j باشد ($j > i$)، و P_i صبر کند تا اجرای P_j تمام شود و سپس منابع را در اختیار بگیرد و اجرای خود را آغاز کند.

اگر سیستمی در حالت ایمن باشد، دچار ددلاک نمی‌شود.

اگر سیستمی دچار ددلاک باشد، در حالت ناایمن است.

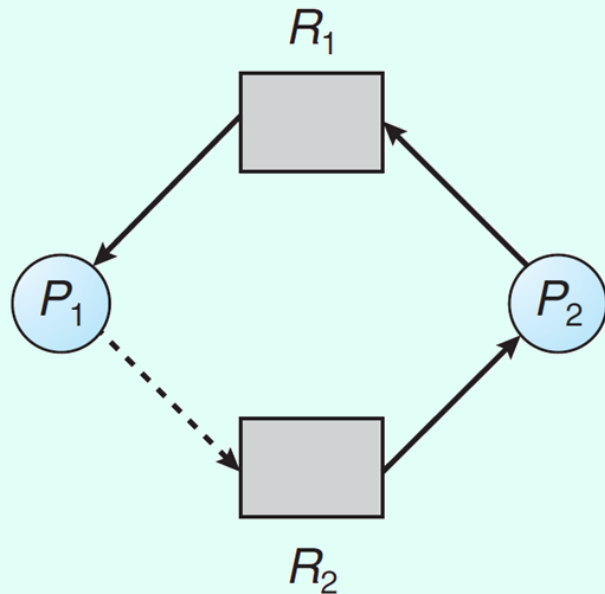
اگر سیستمی در حالت ناایمن باشد، ممکن است دچار ددلاک شود.



Deadlock Avoidance

در سیستم‌هایی که منابع فقط یک نمونه دارند، می‌توانیم با استفاده از گراف منبع-تخصیص، احتمال وقوع ددلاک را بررسی کنیم.

یک نوع یال جدید از فرآیند به منبع تعریف می‌کنیم به نام claim edge و آن را با نقطه‌چین نمایش می‌دهیم. این یال یعنی فرآیند احتمالاً در آینده برای منبع مورد نظر درخواست خواهد داد.



اگر در این گراف حلقه ایجاد شود، سیستم وارد حالت ناایمن خواهد شد و احتمال وقوع ددلاک وجود دارد.

الگوریتم بانکدار (Banker's Algorithm)

- این الگوریتم یکی از روش‌های اجتناب از بن‌بست (Deadlock Avoidance) است.
- اگر منابع بیش از یک instance داشته باشند، دیگر بررسی حلقه داشتن گراف RAG کافی نیست، و باید از الگوریتم بانکدار استفاده کرد.
- هر فرآیند تعداد بیشینه منابعی که استفاده خواهد کرد را اعلام می‌کند.
- وقتی فرآیندی منابع خود را دریافت کند، باید در زمان محدودی آن‌ها را آزاد کند.

الگوریتم بانکدار (Banker's Algorithm)

■ اگر n تعداد فرآیندها، و m تعداد انواع منابع باشد، ساختمان‌های زیر را برای الگوریتم تعریف می‌کنیم:

Available: آرایه‌ای یک بعدی به طول m ، که تعداد instance‌های موجود برای هر منبع را نشان می‌دهد. اگر $Available[j]=k$ باشد یعنی تعداد instance‌های منبع R_j برابر k است.

Max: آرایه‌ای دو بعدی به طول $n*m$ که تعداد بیشینه درخواست‌های هر فرآیند برای هر منبع را نشان می‌دهد. $Max[i][j]=k$ یعنی تعداد بیشینه instance‌های درخواستی فرآیند P_i برای منبع R_j برابر k است.

Allocation: آرایه‌ای دو بعدی به طول $n*m$ که تعداد instance‌های تخصیص یافته هر منبع به هر فرآیند را نشان می‌دهد. $Allocation[i][j]=k$ یعنی تعداد instance‌های تخصیص داده شده منبع R_j به فرآیند P_i برابر k است.

Need: آرایه‌ای دو بعدی به طول $n*m$ که تعداد instance‌های مورد نیاز هر فرآیند از هر منبع را نشان می‌دهد. $Need[i][j]=k$ یعنی تعداد instance‌های مورد نیاز فرآیند P_i از منبع R_j برابر k است:

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

الگوریتم ایمنی (Safety Algorithm)

■ این الگوریتم تشخیص می‌دهد که آیا سیستم در وضعیت ایمن قرار دارد یا خیر

1. آرایه‌های Work و Finish را در نظر بگیرید به طرزى که $Finish[i]=false$ for $i=0,1,...,n-1$ و $Work = Available$

2. اندیس ای پیدا کنید به نحوی که :

a. $Finish[i]=false$

b. $Need_i \leq Work$

اگر هیچ ای دو شرط را نداشت، به گام 4 بروید.

3. $Work=Work+Allocation$

$Finish[i]=true$

به گام 2 بروید.

4. اگر به ازای تمام آنها، $Finish[i]=true$ باشد، سیستم در حالت ایمن است.

پیچیدگی زمانی این الگوریتم
از مرتبه $O(m \times n^2)$ است

الگوریتم ایمنی (Safety Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم در وضعیت safe قرار دارد؟

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

الگوریتم ایمنی (Safety Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم در وضعیت safe قرار دارد؟

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

ابتدا Need را حساب می‌کنیم:

	<u>Need</u>
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

الگوریتم ایمنی (Safety Algorithm)

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	<u>Need</u>
	A B C
P_0	7 4 3
✓ P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

Work=(3,3,2) → Work=(5,3,2)

الگوریتم ایمنی (Safety Algorithm)

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	<u>Need</u>
	A B C
P_0	7 4 3
✓ P_1	1 2 2
P_2	6 0 0
✓ P_3	0 1 1
P_4	4 3 1

Work=(5,3,2) → Work=(7,4,3)

الگوریتم ایمنی (Safety Algorithm)

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	<u>Need</u>
	A B C
P_0	7 4 3
✓ P_1	1 2 2
P_2	6 0 0
✓ P_3	0 1 1
✓ P_4	4 3 1

Work=(7,4,3) → Work=(7,4,5)

الگوریتم ایمنی (Safety Algorithm)

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

	<u>Need</u>
	A B C
✓ P_0	7 4 3
✓ P_1	1 2 2
P_2	6 0 0
✓ P_3	0 1 1
✓ P_4	4 3 1

Work=(7,4,5) → Work=(7,5,5)

الگوریتم ایمنی (Safety Algorithm)

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

سیستم در وضعیت ایمن است

Safe Sequence = <P1,P3,P4,P0,P2>

	<u>Need</u>
	A B C
✓ P_0	7 4 3
✓ P_1	1 2 2
✓ P_2	6 0 0
✓ P_3	0 1 1
✓ P_4	4 3 1

Work=(7,5,5) → Work=(10,5,7)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

■ این الگوریتم تشخیص می‌دهد که آیا یک درخواست به طور ایمن می‌تواند پاسخ داده شود. آرایه $Request_i$ برای فرآیند P_i مشخص می‌کند که از هر منبع چند instance می‌خواهد. $Request[j]=k$ یعنی از منبع j به تعداد k نمونه می‌خواهد.

1. اگر $Request_i \leq Need_i$ ، به گام 2 بروید. در غیر این صورت درخواست نامعتبر است.
2. اگر $Request_i \leq Available$ ، به گام 3 بروید. در غیر این صورت فرآیند باید صبر کند زیرا منبع موجود نیست.
3. محاسبات زیر را انجام می‌دهیم:

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

حال الگوریتم ایمنی را اجرا می‌کنیم. اگر حالت جدید ایمن بود، درخواست را انجام می‌دهیم.

الگوریتم منبع-درخواست (Resource-Request Algorithm)

(مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

ابتدا Need را حساب می‌کنیم:

Process	Need		
	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	2				1	2	2
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

$$\text{Request}_i \leq \text{Need}_i$$

$$(1,0,2) \leq (1,2,2)$$

درخواست از نیاز فرآیند بیشتر نیست
به گام بعدی می‌رویم.

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	2				1	2	2
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

$$\text{Request}_i \leq \text{Available}$$

$$(1,0,2) \leq (3,3,2)$$

منبع در دسترس است
به گام بعدی می‌رویم.

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	3	3	2	7	4	3
P1	2	0	0	3	2	2				1	2	2
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

$$\text{Available} = \text{Available} - \text{Request}_i$$

$$\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$$

$$\text{Need}_i = \text{Need}_i - \text{Request}_i$$

$$\text{Available} = (3, 3, 2) - (1, 0, 2) = (2, 3, 0)$$

$$\text{Allocation}_i = (2, 0, 0) + (1, 0, 2) = (3, 0, 2)$$

$$\text{Need}_i = (1, 2, 2) - (1, 0, 2) = (0, 2, 0)$$

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Max			Available			Need		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	2	3	0	7	4	3
P1	3	0	2	3	2	2				0	2	0
P2	3	0	2	9	0	2				6	0	0
P3	2	1	1	2	2	2				0	1	1
P4	0	0	2	4	3	3				4	3	1

Work=(2,3,0)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
✓ P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Work=(2,3,0) → Work=(5,3,2)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
✓ P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
✓ P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Work=(5,3,2) → Work=(7,4,3)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Need		
	A	B	C	A	B	C
✓ P0	0	1	0	7	4	3
✓ P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
✓ P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Work=(7,4,3) → Work=(7,5,3)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Need		
	A	B	C	A	B	C
✓ P0	0	1	0	7	4	3
✓ P1	3	0	2	0	2	0
✓ P2	3	0	2	6	0	0
✓ P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Work=(7,5,3) → Work=(10,5,5)

الگوریتم منبع-درخواست (Resource-Request Algorithm)

مثال) سیستم زیر را در نظر بگیرید. اگر فرآیند P1 درخواست (1,0,2) را انجام دهد، می‌توانیم این درخواست را طوری پاسخ دهیم که سیستم در حالت ایمن باشد؟

Process	Allocation			Need		
	A	B	C	A	B	C
✓ P0	0	1	0	7	4	3
✓ P1	3	0	2	0	2	0
✓ P2	3	0	2	6	0	0
✓ P3	2	1	1	0	1	1
✓ P4	0	0	2	4	3	1

Work=(10,5,5) → Work=(10,5,7)

Safe Sequence = <P1,P3,P0,P2,P4>

تشخیص بن‌بست (Deadlock Detection)

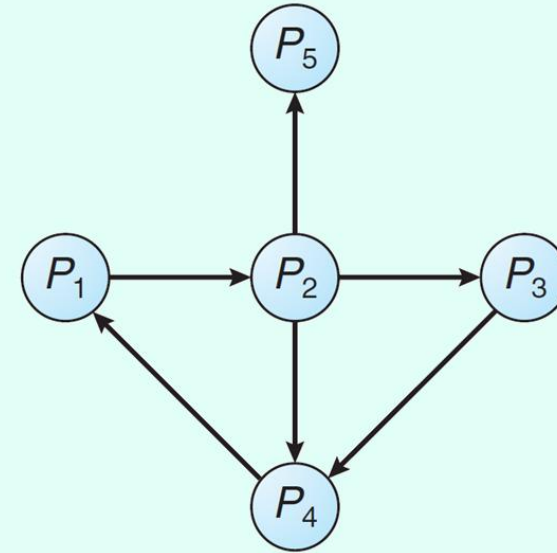
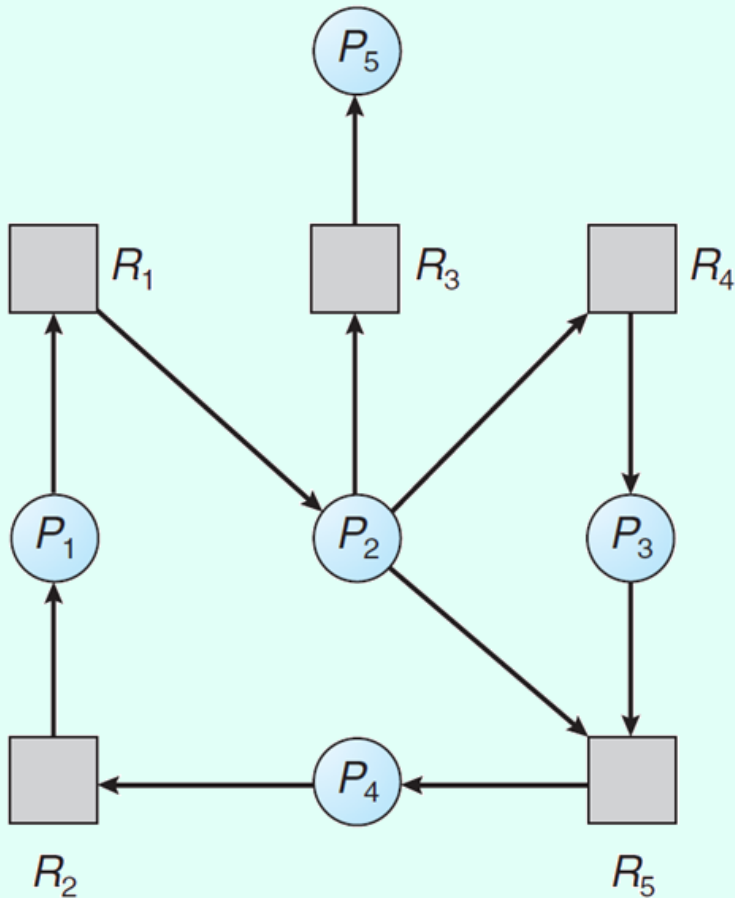
■ اگر سیستم از deadlock prevention یا deadlock avoidance استفاده نکند، در این حالت ممکن است بن‌بست رخ دهد.

■ در این حالت باید از الگوریتمی استفاده کنیم که مشخص کند آیا بن‌بست رخ داده است یا خیر.
(Deadlock Detection)

■ اگر بن‌بست رخ داده بود، از الگوریتمی استفاده می‌کنیم که سیستم را از ددلاک خارج کند.
(Recovery from Deadlock)

تشخیص بن‌بست (Deadlock Detection)

گراف wait-for: یال از رأس P_i به P_j یعنی فرآیند P_i منتظر فرآیند P_j است.

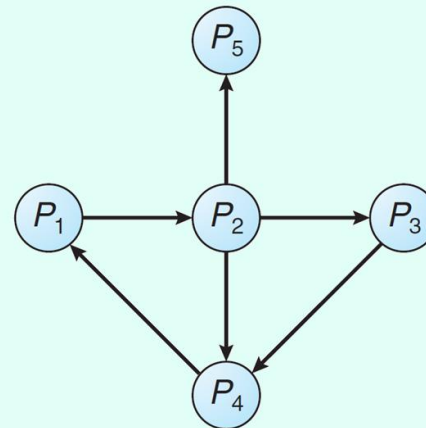


تشخیص بن‌بست (Deadlock Detection)

در سیستمی که منابع فقط یک *instance* دارند، اگر در گراف *wait-for* حلقه وجود داشته باشد، سیستم دچار بن‌بست شده است.

پیچیدگی زمانی تشخیص حلقه در گراف جهت‌دار با n رأس، برابر $O(n^2)$ است.

مثال) آیا سیستم زیر دچار بن‌بست است؟



تشخیص بن بست (Deadlock Detection)

اگر در سیستم منابع بیش از یک instance داشته باشند، از الگوریتم تشخیص (Detection Algorithm) استفاده می‌کنیم.

در این الگوریتم از ساختمان داده‌هایی که در الگوریتم بانک‌دار استفاده کردیم، بهره می‌بریم:

Available: آرایه‌ای یک بعدی به طول m ، که تعداد instance‌های موجود برای هر منبع را نشان می‌دهد. اگر $Available[j] = k$ باشد یعنی تعداد instance‌های منبع R_j برابر k است.

Allocation: آرایه‌ای دو بعدی به طول $n \times m$ که تعداد instance‌های تخصیص یافته هر منبع به هر فرآیند را نشان می‌دهد. $Allocation[i][j] = k$ یعنی تعداد instance‌های تخصیص داده شده منبع R_j به فرآیند P_i برای برابر k است.

Request: آرایه‌ای دو بعدی به طول $n \times m$ که تعداد instance‌های درخواستی فرآیند را برای منبع مشخص می‌کند. $Request[i][j] = k$ یعنی تعداد instance‌های درخواستی فرآیند P_i از منبع R_j برابر k است.

الگوریتم تشخیص (Detection Algorithm)

1. آرایه‌های Work و Finish را در نظر بگیرید به طرزى که $Work = Available$ و اگر $Allocation_i \neq 0$ ، Finish برابر می‌شود با: $Finish[i] = false$ for $i=0,1,...,n-1$

2. اندیس ای پیدا کنید به نحوی که :

a. $Finish[i] = false$

b. $Request_i \leq Work$

اگر هیچ ای دو شرط را نداشت، به گام 4 بروید.

3. $Work = Work + Allocation$

$Finish[i] = true$

به گام 2 بروید.

4. اگر ای پیدا شود که $Finish[i] == false$ ، انگاه سیستم دچار ددلاک است و فرآیند P_i دچار ددلاک است.

پیچیدگی زمانی این الگوریتم
از مرتبه $O(m \times n^2)$ است

الگوریتم تشخیص (Detection Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم زیر در بن بست قرار دارد؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Work=(0,0,0) → Work=(0,1,0)

الگوریتم تشخیص (Detection Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم زیر در بن بست قرار دارد؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
✓ P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Work=(0,1,0) → Work=(3,1,3)

الگوریتم تشخیص (Detection Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم زیر در بن بست قرار دارد؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P_0	0 1 0	0 0 0	0 0 0
✓ P_1	2 0 0	2 0 2	
✓ P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Work=(3,1,3) → Work=(5,1,3)

الگوریتم تشخیص (Detection Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم زیر در بن بست قرار دارد؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P_0	0 1 0	0 0 0	0 0 0
✓ P_1	2 0 0	2 0 2	
✓ P_2	3 0 3	0 0 0	
✓ P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Work=(5,1,3) → Work=(7,2,4)

الگوریتم تشخیص (Detection Algorithm)

مثال) سیستم زیر را در نظر بگیرید. آیا این سیستم زیر در بن بست قرار دارد؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P_0	0 1 0	0 0 0	0 0 0
✓ P_1	2 0 0	2 0 2	
✓ P_2	3 0 3	0 0 0	
✓ P_3	2 1 1	1 0 0	
✓ P_4	0 0 2	0 0 2	

Work=(7,2,4) → Work=(7,2,6)

Safe Sequence = <P0,P2,P1,P3,P4>

سیستم در بن بست نیست

الگوریتم تشخیص (Detection Algorithm)

مثال) در مثال قبل، اگر P2 برای منبع C یک instance درخواست کند چطور؟

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
✓ P ₀	0 1 0	0 0 0	0 0 0
P ₁	2 0 0	2 0 2	
P ₂	3 0 3	0 0 1	
P ₃	2 1 1	1 0 0	
P ₄	0 0 2	0 0 2	

از این مرحله جلوتر نمی‌توان رفت. $Work=(0,0,0) \rightarrow Work=(0,1,0)$

سیستم در بن‌بست قرار دارد.
 فرآیندهای P₁، P₂، P₃ و P₄
 دچار بن‌بست هستند.

رفع بن بست (Recovery from Deadlock)

- وقتی الگوریتم تشخیص، وجود ددلاک در سیستم را مشخص کند، اقدام‌های مختلفی می‌توان انجام داد:
 1. یک راه این است که بن بست را به اپراتور اطلاع دهد و خود اپراتور به طور دستی بن بست را رفع کند
 2. روش دیگر این است که سیستم به طور خودکار بن بست را رفع کند.

- برای شکستن بن بست، دو اقدام می‌توان انجام داد:
 1. یک یا چند فرآیند را جهت شکستن circular wait، abort کنیم. (Process Termination)
 2. منابع یک یا چند فرآیند را که دچار بن بست شده‌اند به طور موقت بگیریم. (Resource Preemption)

Process Termination

■ برای شکستن بن بست با abort کردن فرآیند، یکی از دو روش زیر استفاده می شود:

1. Abort کردن تمام فرآیندهای دچار بن بست: این روش تمامی فرآیندهای دچار ددلاک را abort می کند. این روش به طور قطعی ددلاک را از بین می برد، ولی هزینه بسیار بالایی دارد. این فرآیندها باید از اوّل مجدداً اجرا شوند و تمامی محاسبات خود را دوباره انجام دهند.

2. Abort کردن یک فرآیند در هر لحظه تا زمانی که بن بست شکسته شود: در این روش تعداد فرآیندهای کمتری Abort می شوند که نسبت به روش قبلی مزیت است، ولی سربار این روش بسیار بالا است زیرا پس از هر abort، باید الگوریتم شناسایی اجرا شود.

Process Termination

مثال) سیستمی با 300 فرآیند و 10000 منبع دچار بن‌بست شده است. شروع به abort کردن فرآیندها می‌کنیم تا زمانی که بن‌بست شکسته شود. اگر پس از abort کردن 20 فرآیند بن‌بست شکسته شود، تعداد عملیات‌های انجام شده چقدر است؟

Process Termination

■ در روش دوم، برای اینکه هر بار کدام فرآیند را جهت abort انتخاب کنیم، باید به موارد زیر دقت کنیم:

1. اولویت فرآیند
2. فرآیند چه مدت اجرا شده است و چه مدت از اجرایش باقی مانده است؟
3. منابعی که فرآیند استفاده می‌کند
4. منابعی که فرآیند جهت تکمیل کارش نیاز دارد
5. چه تعداد فرآیند باید abort شوند؟
6. فرآیند interactive است یا batch؟

Resource Preemption

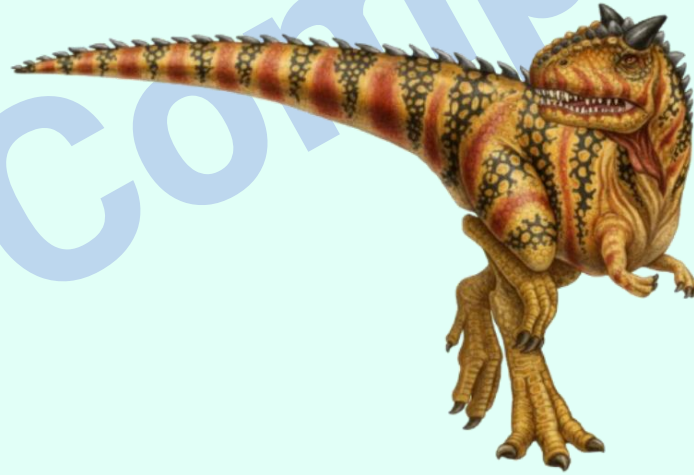
■ در این روش به طور پی در پی منابع را از فرآیندها می‌گیریم و به فرآیندهای دیگر می‌دهیم تا زمانی که حلقه ددلاک شکسته شود. در این روش باید به 3 نکته زیر توجه شود:

1. **انتخاب قربانی:** کدام فرآیند انتخاب و کدام منبع آن باید گرفته شود؟ باید ترتیب preemption به نحوی باشد که هزینه کمینه شود.

2. **Rollback:** وقتی منابع فرآیندی را می‌گیریم، باید فرآیند را به حالتی ایمن برگردانیم تا restart شود. تشخیص اینکه چه حالتی ایمن است خود چالش برانگیز است، به همین سبب گاهی از total rollback استفاده می‌شود که همان abort کردن فرآیند است.

3. **Starvation:** ممکن است همواره فقط برخی فرآیندها جهت preemption انتخاب شوند و دچار starvation شوند. برای همین باید مطمئن شویم که هر فرآیند فقط به تعداد محدودی منابعش سلب شود.

Chapter 8: Main Memory



Basic Hardware

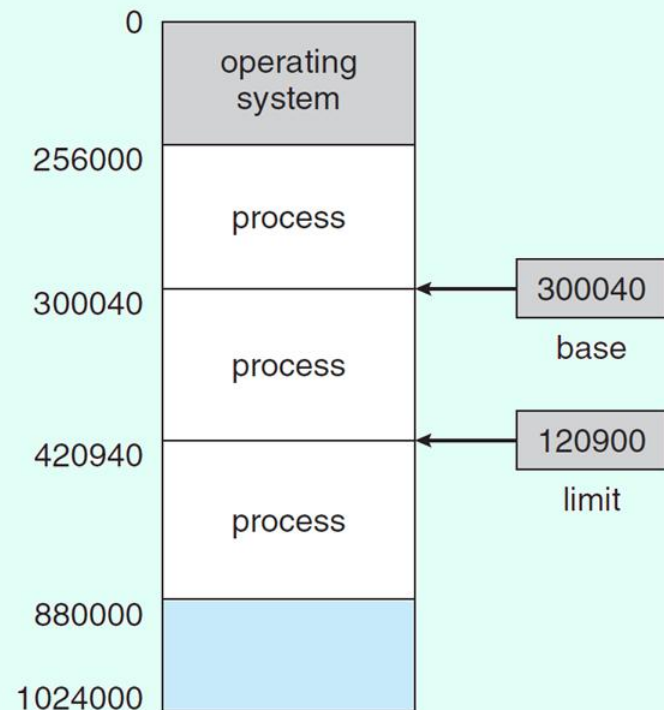
■ حافظه اصلی (Main Memory) و رجیسترهای درون پردازنده، تنها حافظه عمومی‌ای هستند که پردازنده به طور مستقیم به آن دسترسی دارد. دستورات ماشینی که از آدرس حافظه به عنوان آرگومان استفاده می‌کنند، و داده‌ای که استفاده می‌کنند، همگی در این حافظه دسترسی مستقیم (direct-access) است.

■ رجیسترها در یک سیکل کلاک قابل دسترسی هستند. بیشتر پردازنده‌ها می‌توانند دستورات را decode و بر روی رجیستر عملیات‌هایی را در یک کلاک انجام دهند. ولی این کار را با حافظه اصلی نمی‌توان انجام داد. دسترسی به حافظه اصلی ممکن است چند کلاک طول بکشد، به همین دلیل پردازنده نیاز دارد stall کند.

■ از آنجایی که stall تعداد سیکل‌های اجرا را زیاد می‌کند، افزودن حافظه‌ای سریع بین CPU و حافظه اصلی، مانند cache می‌تواند اجرا را سریع‌تر کند.

Basic Hardware

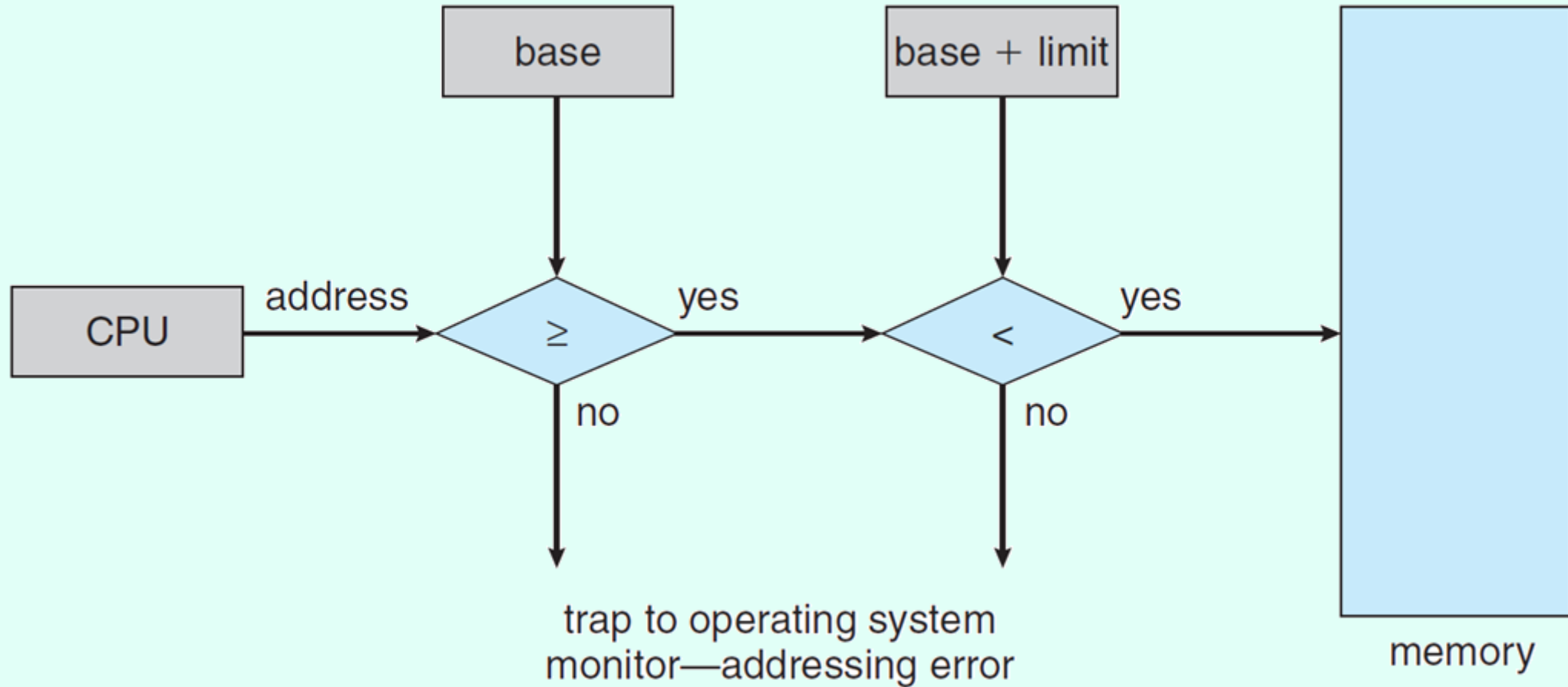
■ باید مطمئن شویم هر فرآیند فضای حافظه جدای خود را داشته باشد و فرآیندهای دیگر به این فضا دسترسی نداشته باشند. برای این موضوع باید توانایی مشخص کردن بازه آدرس قانونی (legal address) را داشته باشیم. این حفاظت توسط سخت افزار انجام می شود نه سیستم عامل.



■ برای این امر از دو رجیستر base و limit استفاده می کنیم:

$$\text{logical address space} = [\text{base}, \text{base} + \text{limit}]$$

Basic Hardware



Address Binding

■ معمولا یک برنامه (program) به صورت یک فایل باینری قابل اجرا بر روی دیسک قرار دارد. برای اجرا شدن، باید وارد حافظه شود و به عنوان یک فرآیند قرار گیرد. فرآیندهایی که بر روی دیسک منتظر ورود به حافظه برای اجرا شدن هستند، **صف ورودی (input queue)** را تشکیل می‌دهند.

■ در بیشتر مواقع، یک برنامه قبل از اجرا مراحل را طی می‌کند (که برخی از آن‌ها اختیاری است). آدرس‌ها به شکل‌های متفاوتی در این مراحل بیان می‌شوند. در سورس برنامه آدرس نمادین است (مانند متغیر count).

■ کامپایلر این آدرس‌های نمادین را آدرس‌های جابجائی‌پذیر (relocatable) تبدیل می‌کند. در نهایت loader این آدرس جابجائی‌پذیر را به آدرس مطلق (absolute) تبدیل می‌کند. به نداشت یک فضای آدرس به دیگری، **binding** گوئیم.

Address Binding

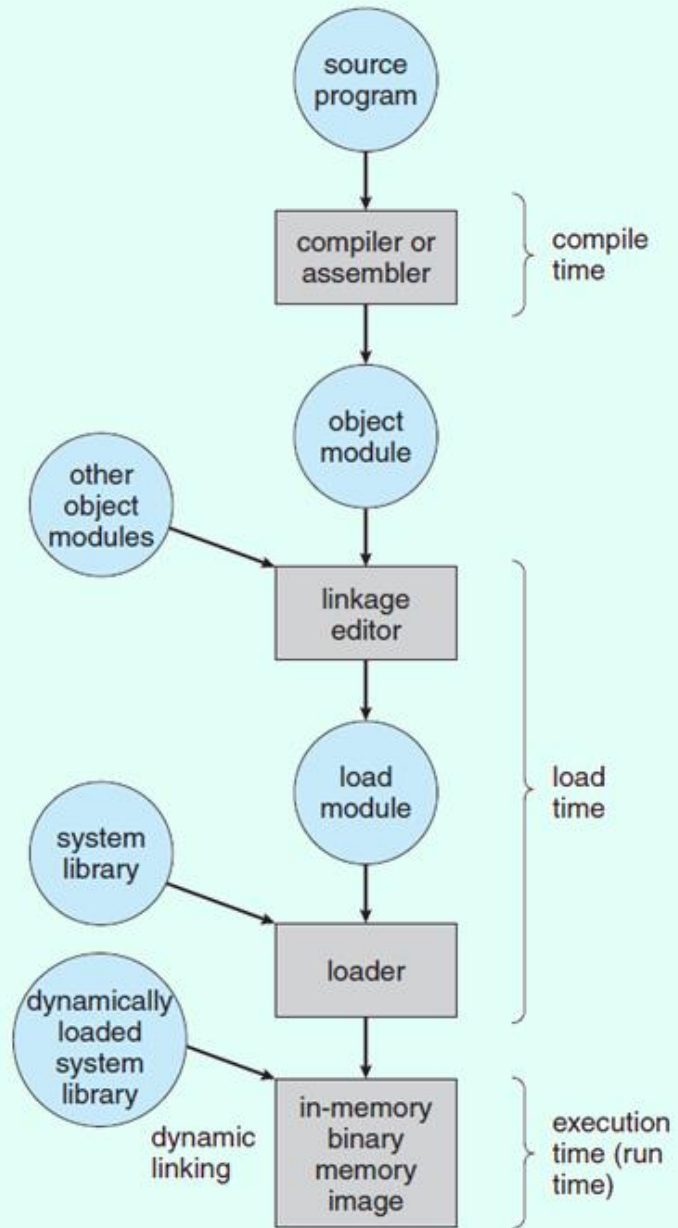
■ این فرایند در مراحل زیر انجام می‌شود:

زمان کامپایل (Compile Time): اگر در زمان کامپایل بدانیم فرآیند در کجای حافظه قرار می‌گیرد، کد مطلق (absolute code) ایجاد می‌شود. برای مثال اگر بدانیم یک فرآیند از موقعیت R در حافظه شروع می‌شود، کد تولید شده از آن موقعیت شروع به اجرا می‌شود. اگر موقعیت شروع حافظه تغییر کند، ضروری است که کد دوباره کامپایل شود.

زمان بارگذاری (Load Time): اگر در زمان کامپایل موقعیت فرآیند در حافظه مشخص نباشد، کامپایلر باید کد جابجائی‌پذیر (relocatable) را تولید کند. در این حالت binding تا زمان بارگذاری تأخیر می‌خورد.

زمان اجرا (Execution Time): اگر فرآیند در حین اجرا بتواند از یک قسمت حافظه به قسمت دیگری برود، binding باید تا زمان اجرا تأخیر بخورد. بیشتر سیستم‌های عامل‌های عمومی از این روش استفاده می‌کنند.

Address Binding



symbolic address → relocatable address → absolute address

COMPYAAR YOUTUBE CHANNEL

Logical vs Physical Address Space

آدرس منطقی (Logical Address): آدرسی که توسط CPU تولید می‌شود. به آن آدرس مجازی (Virtual Address) نیز گویند.

آدرس فیزیکی (Physical Address): آدرسی که در رجیستر حافظه-آدرس (memory-address) باگذاری می‌شود و توسط واحد حافظه دیده می‌شود.

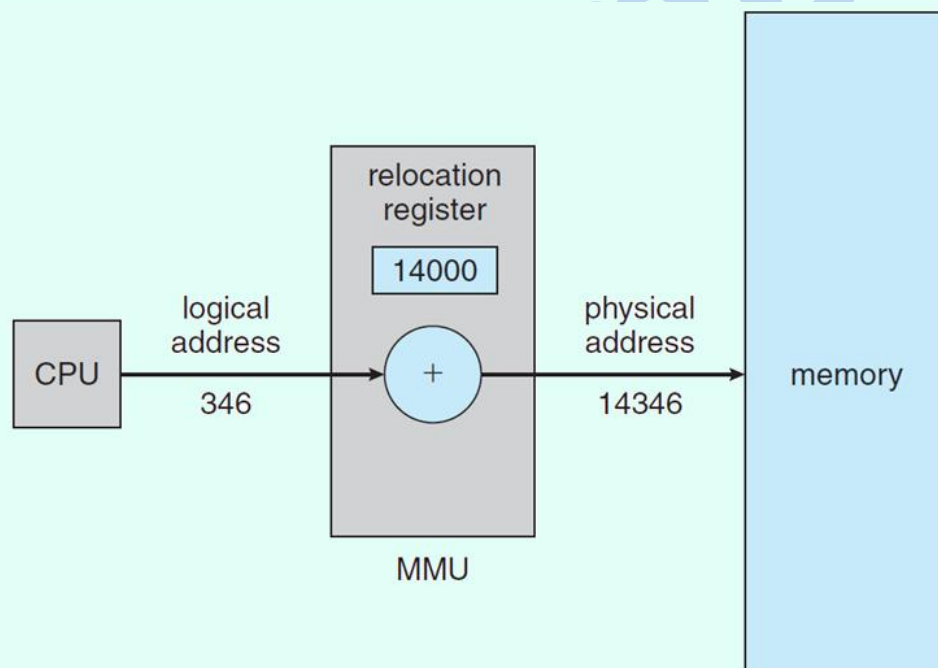
■ در زمان کامپایل و بارگذاری، این دو آدرس یکی هستند، اما در زمان اجرا متفاوت می‌شوند.

■ به مجموعه تمام آدرس‌های مجازی تولید شده توسط یک برنامه فضای آدرس مجازی (virtual address space) و به مجموعه تمام آدرس‌های فیزیکی متناظر نیز فضای آدرس فیزیکی (physical address space) گوییم.

Memory Management Unit

■ نگاشت آدرس مجازی به فیزیکی در زمان اجرا توسط دستگاه سخت‌افزاری واحد مدیریت حافظه (MMU) انجام می‌شود. به روش‌های مختلفی این نگاشت را می‌توان انجام داد که در ادامه فصل خواهیم دید.

■ یک روش ساده تولید آدرس فیزیکی، افزودن آدرس مجازی به مقدار رجیستر relocation است (همان رجیستر base که عقب‌تر دیدیم):



Dynamic Loading

■ تا اینجا دیدیم که برای اجرای فرآیند ضروری است که تمام برنامه و داده آن در حافظه فیزیکی قرار گیرند تا بتواند اجرا شود. اندازه فرآیند محدود به اندازه حافظه است. برای دست یافتن به memory-space utilization بهتر، می‌توان از **بارگذاری پویا (dynamic loading)** استفاده کرد. در این روش routine تا زمانی که فراخوانی نشود، بارگذاری نمی‌شود.

■ مزیت این روش در این است که یک routine تنها زمانی بارگذاری می‌شود که به آن نیاز است. برای سناریوهایی که حجم بالایی کد به ندرت نیاز به هندل شدن دارند، مناسب است، مثل error routines. در این مورد، اندازه برنامه بزرگ است، ولی میزان مورد استفاده از حافظه بسیار کمتر است.

■ بارگذاری پویا نیاز به پشتیبانی سیستم عامل ندارد، مسئولیت کاربران است که برنامه خود را جوری طراحی کنند که از مزیت این روش بهره ببرند.

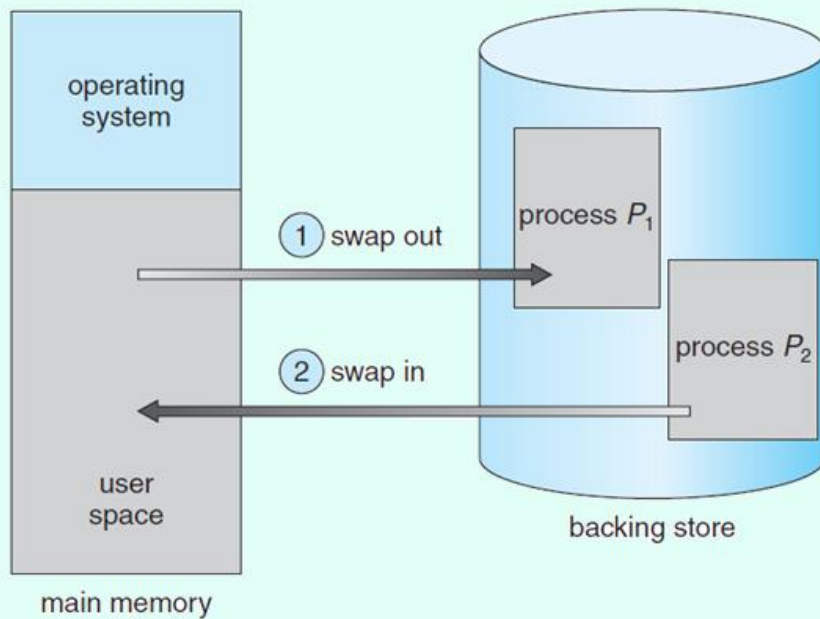
Dynamic Linking & Shared Libraries

- کتابخانه‌های dynamically linked زمانی به برنامه کاربر متصل می‌شوند که برنامه اجرا می‌شود. به این صورت در فضای دیسک، حافظه و زمان صرفه جویی می‌شود.
- با dynamic linking، یک stub همراه است. stub بررسی می‌کند که آیا کتابخانه در حافظه است یا خیر. اگر نبود، سیستم عامل کتابخانه را در حافظه بارگذاری می‌کند. در ادامه stub خود را با آدرس routine جایگزین می‌کند و آن را اجرا می‌کند.
- می‌توان کپی یک کتابخانه را به عنوان shared library استفاده کرد که در میزان حافظه مورد استفاده صرفه جویی بزرگی صورت می‌گیرد.

Swapping

■ یک فرآیند جهت اجرا شدن باید در حافظه باشد، اگرچه یک فرآیند می‌تواند به طور موقت از حافظه به یک انبار پشتیبان (backing store) منتقل می‌شود (swapping).

■ این عمل، باعث می‌شود فضای آدرس فیزیکی تمام فرآیندها از حافظه فیزیکی واقعی سیستم فراتر رود، بنابراین درجه multiprogramming در سیستم افزایش می‌یابد.



Standard Swapping

- مبادله استاندارد (standard swapping) شامل انتقال فرآیند بین حافظه اصلی و انبار پشتیبان است. انبار پشتیبان معمولاً یک دیسک سریع است.
- حافظه پشتیبان باید به قدری بزرگ باشد که بتواند تمام محتویات حافظه را برای تمامی کاربران در خود جا دهد. فرآیندهایی که در حافظه اصلی یا انبار پشتیبان هستند، در ready queue منتظر اجرا شدن هستند.
- زمانی که زمان‌بند CPU تصمیم به اجرای فرآیندی گیرد، dispatcher را صدا می‌زند. dispatcher چک می‌کند که آیا فرآیند مورد نظر در حافظه است یا خیر، اگر نبود، باید یک فرآیند که در حافظه است را swap out کند و فرآیند مد نظر را swap in کند.

Standard Swapping

■ زمان context-switch در فرآیند swapping بالا است.

مثال) انبار پشتیبان سیستم یک هارد دیسک استاندارد با نرخ انتقال 50 مگابایت بر ثانیه است. اگر فرآیند مد نظر کاربر جهت اجرا 100 مگابایت باشد و فرآیند در حافظه اصلی نباشد، زمان context-switch چقدر خواهد بود؟

Contiguous Memory Allocation

■ حافظه اصلی باید هم سیستم عامل و هم فرآیندهای کاربران را در خود جای دهد، به همین جهت نیاز است که تخصیص حافظه به شکل بهینه رخ دهد.

■ معمولاً حافظه به دو بخش تقسیم می‌شود: حافظه پایین (آدرس‌های شروع حافظه مثل 0x00000000) و حافظه بالا (آدرس‌های انتهای حافظه مثل 0xFFFF0000 و به بعد). یک بخش باید برای تخصیص سیستم عامل استفاده شود و دیگری برای تخصیص فرآیندهای کاربران.

■ از آنجایی که interrupt vector معمولاً در حافظه پایین است، سیستم عامل نیز در حافظه پایین قرار می‌گیرد.

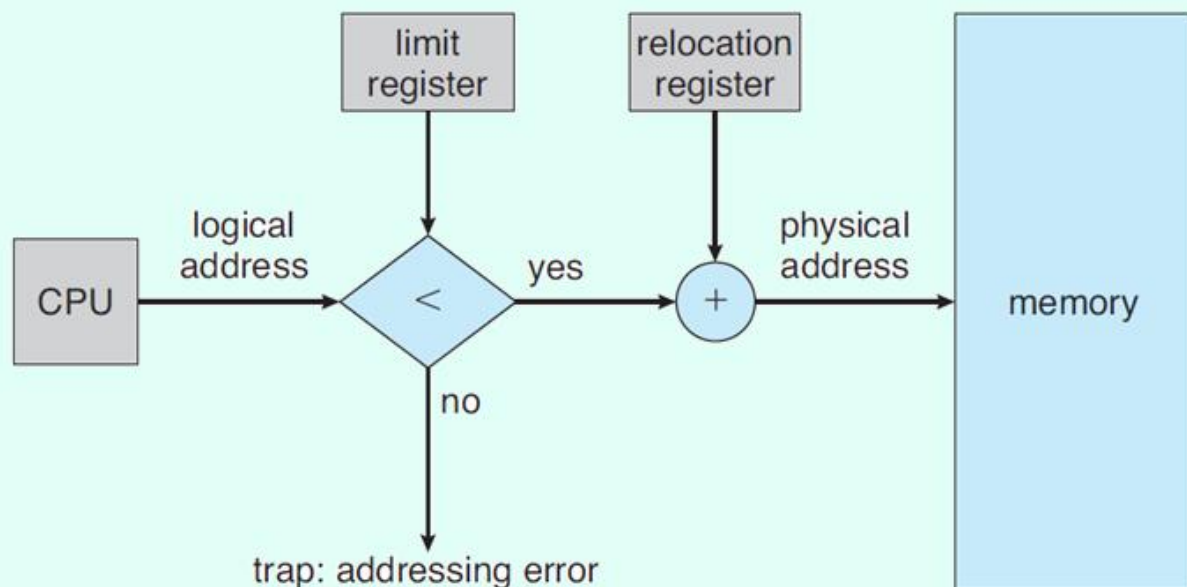
Contiguous Memory Allocation

- معمولاً نیاز داریم که چندین فرآیند را به طور همزمان در حافظه داشته باشیم، بنابراین نیاز داریم حافظه را به نوعی تخصیص دهیم که فرآیندهای waiting queue وارد حافظه شوند و در مکان مناسب خود قرار گیرند.
- در تخصیص حافظه پیوسته (contiguous memory allocation)، هر فرآیند قسمتی از حافظه را در اختیار دارد که پیوسته با قسمت دیگری است که فرآیند دیگری در آن قرار دارد.

Memory Protection

■ فرآیندها نباید بتوانند به قسمتی از حافظه که برای آنها نیست دسترسی داشته باشند. استفاده از relocation register و limit register ما را به هدفمان می‌رساند.

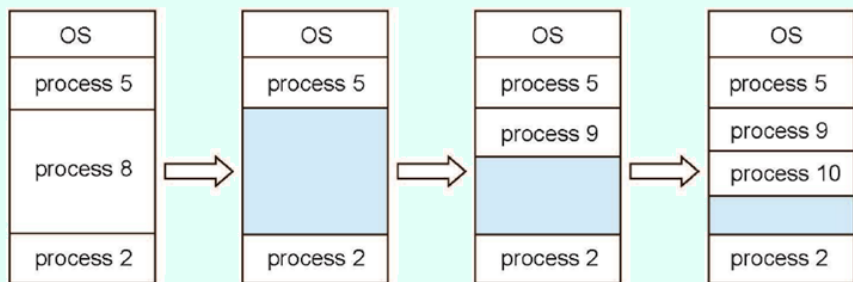
■ برای نگاشت آدرس منطقی به آدرس فیزیکی، MMU آدرس منطقی را با مقدار relocation register جمع می‌کند.



Memory Allocation

Multiple-Partition Method: حافظه را به چندین قسمت (partition) با حافظه ثابت تقسیم می‌کنیم. هر قسمت می‌تواند دقیقاً شامل یک فرآیند باشد. بنابراین درجه چندبرنامگی محدود به تعداد قسمت‌ها می‌شود. وقتی یک فرآیند اجرایش تمام می‌شود، قسمت مربوط به آن در دسترس می‌شود و فرآیند جدیدی از صف انتخاب می‌شود و در آن قسمت قرار می‌گیرد.

Variable-Partition Method: سیستم عامل در جدولی مشخص می‌کند چه بخش‌هایی از حافظه آزاد است و چه بخش‌هایی اشغال شده است. در ابتدا تمام حافظه برای فرآیندها آزاد است، و به عنوان یک بلوک حافظه آزاد شناخته می‌شود که به آن hole گوییم. وقتی فرآیندی قصد اجرا داشته باشد، سیستم عامل قسمتی به اندازه آن فرآیند می‌سازد و به آن تخصیص می‌دهد. وقتی اجرا فرآیند تمام می‌شود، آن قسمت هم از بین می‌رود.



Dynamic Storage Allocation Problem

در روش variable-partition، مجموعه‌ای از holeها در اختیار داریم که در سرتاسر حافظه پخش هستند. وقتی فرآیندی می‌خواهد وارد حافظه شود، باید یک hole به اندازه کافی بزرگ برای آن فرآیند پیدا کنیم. سه استراتژی برای انتخاب hole در این مرحله وجود دارد:

First fit: اختصاص اولین hole که به اندازه کافی بزرگ باشد.

Best fit: اختصاص کوچک‌ترین hole که به اندازه کافی بزرگ باشد.

Worst fit: اختصاص بزرگترین hole.

Dynamic Storage Allocation Problem

مثال) فرآیندهای زیر با اندازه‌هایشان و بلوک‌های حافظه را در نظر بگیرید. به سه روش first fit، best fit و worst fit تخصیص پویا انجام دهید.

90	50	30	40
P1	P2	P3	P4

20	100	40	200	10
----	-----	----	-----	----

Fragmentation

■ از لحاظ سرعت و storage utilization جفت روش‌های first fit و best fit بهتر از worst fit هستند. اما بین این دو نمی‌توان با قطعیت گفت کدام بهتر است.

■ جفت این روش‌ها مشکل **تکه تکه شدن خارجی (external fragmentation)** را دارند. این مشکل زمانی پیش می‌آید که مجموع حافظه‌های آزاد به قدر کافی بزرگ است که فرآیند را در خود جای دهد، ولی این حافظه‌ها پیوسته نیستند، بلکه به صورت چندین تکه در حافظه پخش است.

■ فرقی ندارد از کدام الگوریتم استفاده کنیم، Fragmentation اجتناب ناپذیر است و رخ می‌دهد. تحلیل‌های آماری الگوریتم first fit نشان می‌دهد که یک سوم حافظه به سبب fragmentation غیر قابل استفاده است! که به آن **قانون 50 درصد (50-percent rule)** گویند.

Fragmentation

■ نوع دیگری از fragmentation در حالتی رخ می‌دهد که سایز partitionها ثابت است و درون هر partition قسمتی از حافظه از دست می‌رود. برای مثال فرض کنید سایز یک partition برابر 18464 بایت است و فرآیندی که درون این قسمت قرار می‌گیرد، 18000 بایت است، به این صورت 464 بایت بلا استفاده می‌ماند که به آن **تکه تکه شدن داخلی (internal fragmentation)** گوئیم.

■ یک روش برای حل external fragmentation، استفاده از فشردن (compaction) است. در این روش محتویات حافظه بُر می‌زنیم تا فضاهای آزاد در کنار هم قرار گیرند و یک بلوک بزرگ را تشکیل دهند. فشردن زمانی ممکن است که **جابجائی (relocation)** پویا باشد. جابجائی یعنی بتوانیم برنامه را در حافظه جابجا کنیم و برای آن آدرس جدید به دست آوریم. اغلب فشردن سازی الگوریتم پرهزینه‌ای است.

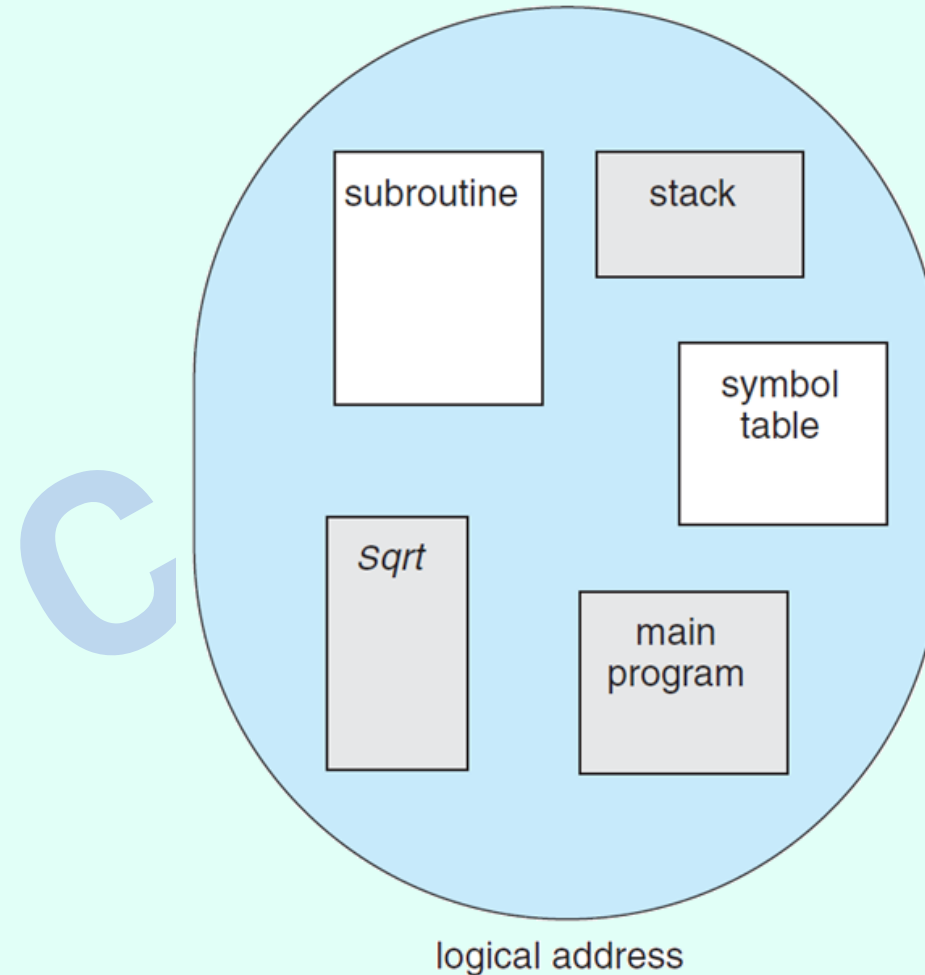
Segmentation

■ دید کاربر از حافظه، با حافظه فیزیکی واقعی متفاوت است. این موضوع برای دید برنامه‌نویس از حافظه نیز صادق است. سر و کله زدن با حافظه با توجه به ویژگی‌های فیزیکی‌اش دشوار است. به همین جهت علاقه‌مندیم به نحوی با مکانیزمی دید برنامه‌نویس را به حافظه فیزیکی واقعی نگاشت کنیم.

■ برنامه‌نویسان ترجیح می‌دهند حافظه را به صورت مجموعه‌ای از قسمت‌هایی (segments) با حافظه متغیر ببینند. برای برنامه‌نویس مهم نیست که "پشته"، "کتابخانه math" و "برنامه main" او در کدام آدرس حافظه قرار دارد. اندازه segmentها متفاوت از هم است و با توجه به هدف برنامه تعیین می‌شود. عناصر هر segment با توجه به offset آنها از شروع segment تعیین می‌شود.

■ تقسیم‌بندی (segmentation) یک مکانیزم مدیریت حافظه است که از دید برنامه‌نویس از حافظه پشتیبانی می‌کند. فضای آدرس منطقی، مجموعه‌ای از segmentها است.

Segmentation: Programmer's View



Segmentation

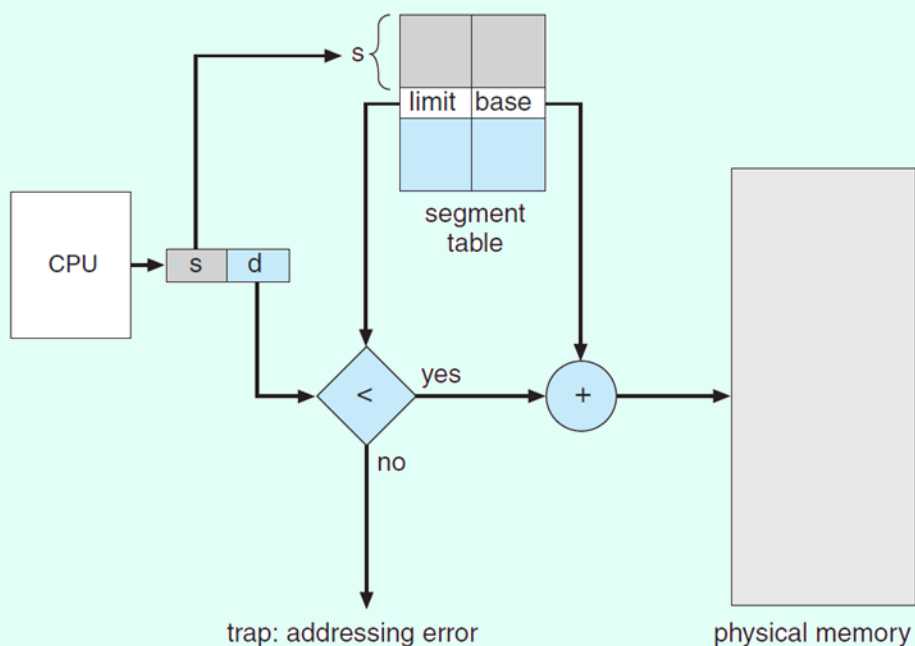
■ هر قسمت یک اسم و یک طول دارد. آدرس‌ها اسم قسمت و offset آن را تعیین می‌کنند. به جای اسم، از یک عدد استفاده می‌کنیم. بنابراین آدرس منطقی شامل دو سطر خواهد بود:
<segment-number, offset>

■ وقتی برنامه‌ای کامپایل می‌شود، کامپایلر به طور خودکار قسمت‌های آن را تولید می‌کند. برای مثال کامپایلر C قسمت‌های زیر را می‌سازد:

1. کد
2. متغیرهای سراسری
3. هرم (heap)
4. پشته‌ها (stacks) که توسط هر نخ استفاده می‌شوند
5. کتابخانه استاندارد C

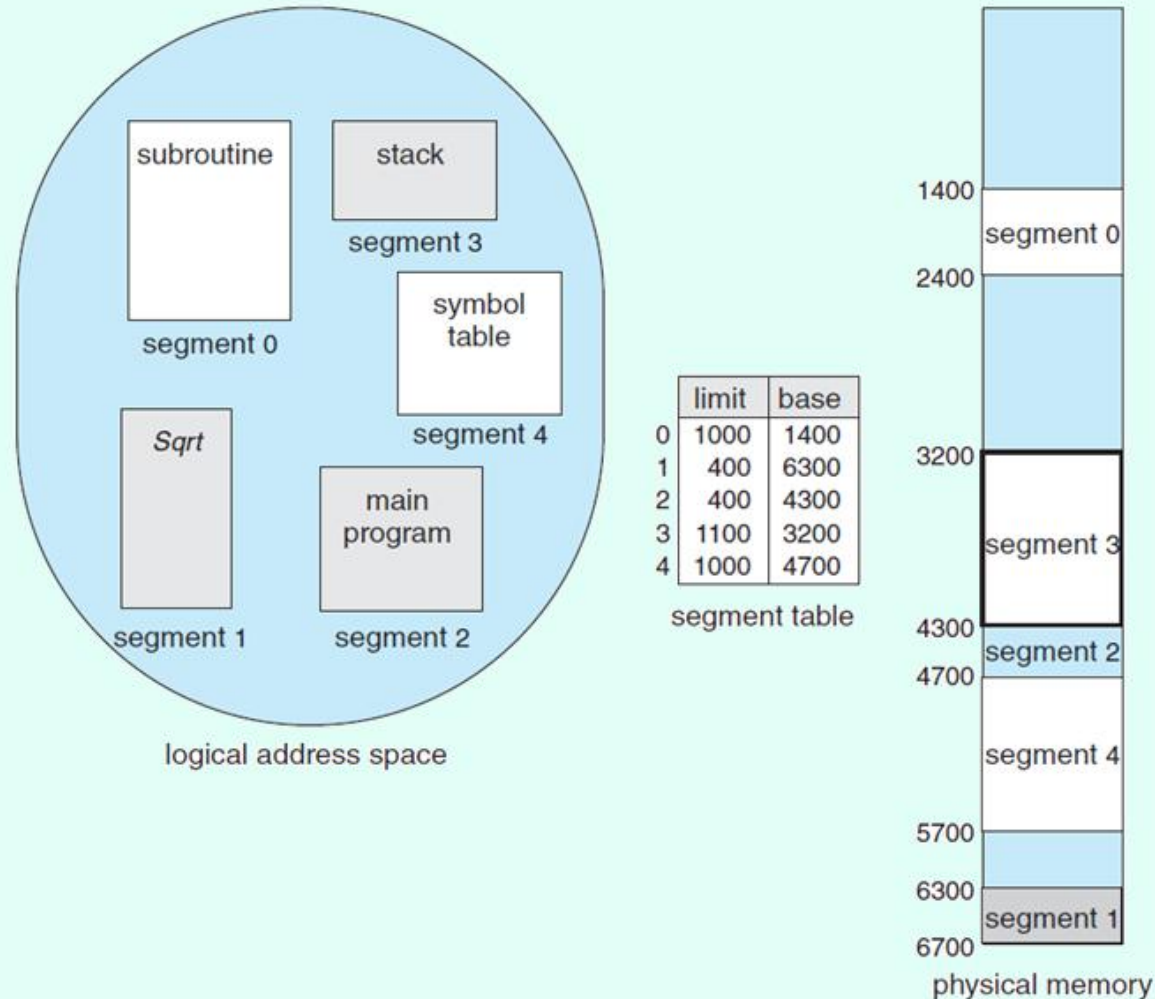
Segmentation Hardware

■ دیدیم که آدرس منطقی دو بعدی است، یک عدد دارد و یک offset. برای نگاشت این آدرس دوبعدی به آدرس فیزیکی یک بعدی، از segment table استفاده می‌کنیم. هر سطر این جدول یک segment base و یک segment limit دارد که segment base شامل آدرس فیزیکی شروع آن segment در حافظه است، و segment limit نشان‌دهنده طول آن segment است.



■ آدرس منطقی شامل شماره (s) و offset (d) است. شماره به عنوان اندیس جدول استفاده می‌شود و offset باید از limit کمتر باشد. به این صورت offset با base در جدول که توسط شماره (s) تعیین شده است جمع می‌شود و آدرس فیزیکی مشخص می‌شود.

Segmentation Hardware: Example



Paging

■ یک روش دیگر مدیریت حافظه غیرپیوسته، مانند segmentation، paging است.

■ روش paging از external fragmentation جلوگیری می‌کند. از دیگر مزایای paging این است که خانه‌های حافظه با اندازه‌های یکسان در انبار پشتیبان نگهداری می‌شوند. مورد دوم از این لحاظ کلیدی است که وقتی می‌خواهیم بخشی از حافظه اصلی را swap out کنیم، در انبار پشتیبان نیز مشکلاتی مانند fragmentation وجود دارد، و از آنجایی که دسترسی در این حافظه کندتر است، compaction غیرممکن است.

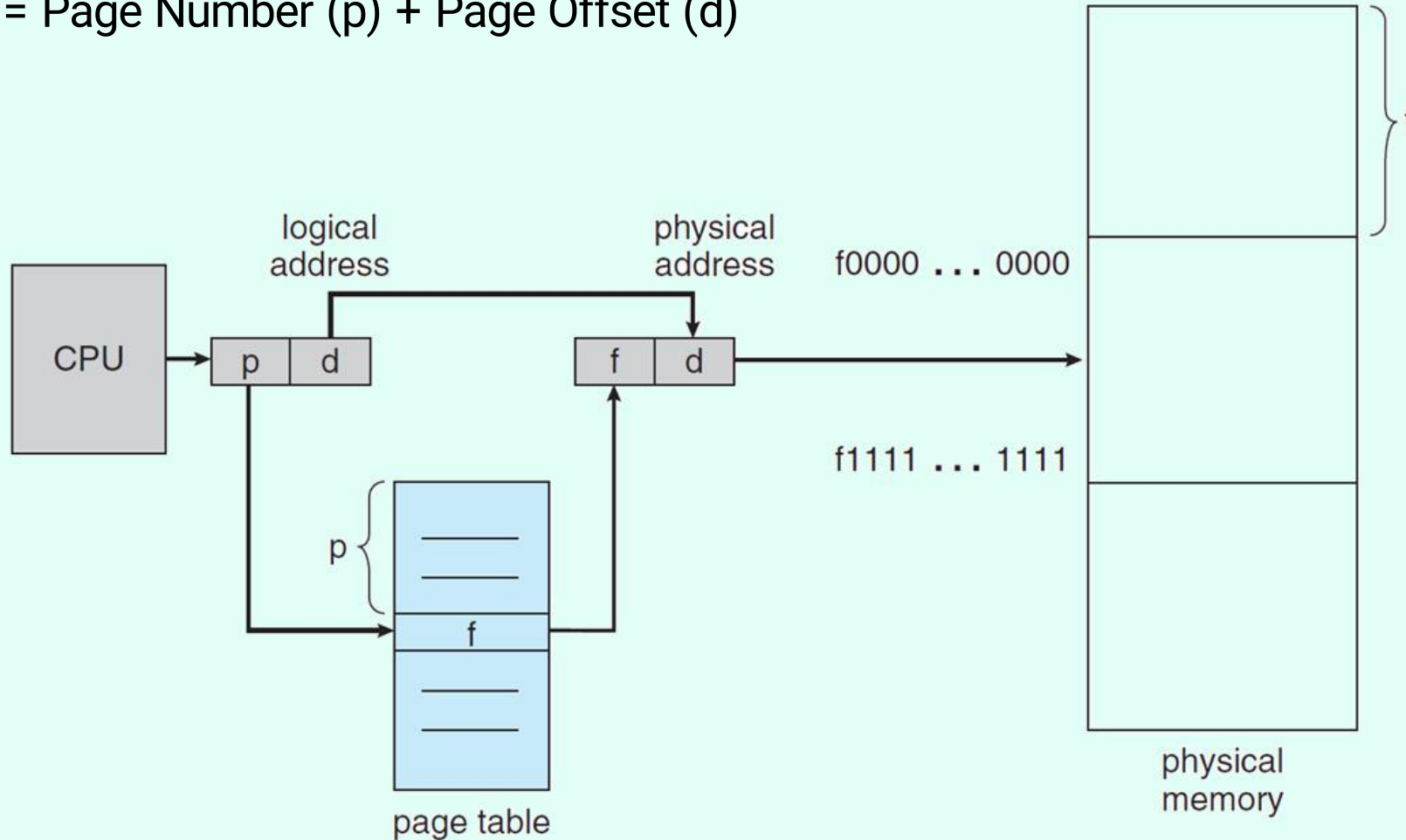
■ به واسطه همکاری سیستم عامل و سخت افزار، paging انجام می‌شود.

Paging: Basic Method

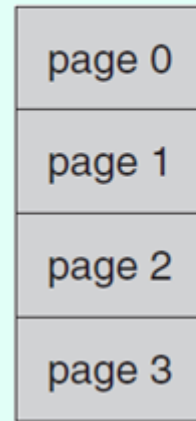
- ایده پیاده‌سازی صفحه‌بندی، تقسیم حافظه فیزیکی به بلوک‌هایی با اندازه ثابت به نام قاب (frame) و تقسیم حافظه منطقی به بلوک‌هایی با همان اندازه به نام (page) است.
- برای اجرای یک فرآیند، صفحات آن در قاب‌های موجود حافظه بارگذاری می‌شود. انبار پشتیبان نیز به بلوک‌هایی با اندازه ثابت و هم‌اندازه با قاب‌های حافظه تقسیم می‌شود
- به این صورت فضای آدرس منطقی و فیزیکی به طور کامل جدا می‌شوند و فرآیندها می‌توانند آدرس‌های 64 بیتی داشته باشند، حتی با وجود اینکه سیستم حافظه فیزیکی بسیار کمتری از 2^{64} بایت داشته باشد.

Paging Hardware

Logical Address = Page Number (p) + Page Offset (d)



Paging Hardware

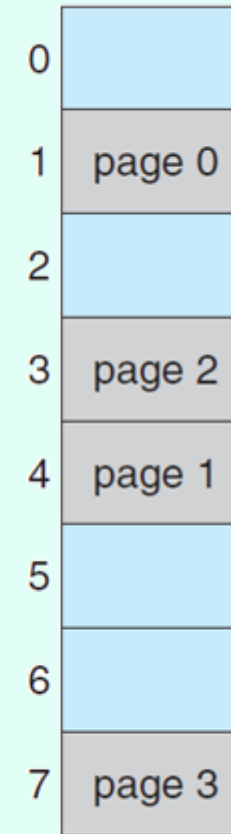


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

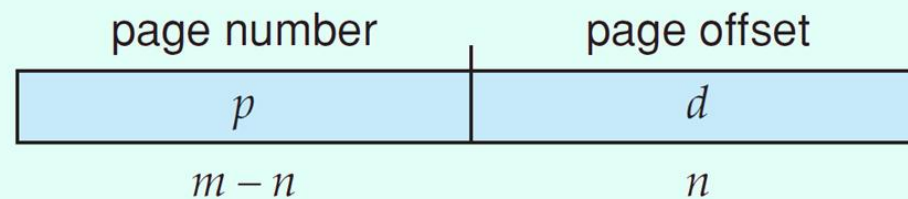


physical
memory

Paging

■ اندازه هر صفحه توانی از 2 است و وابسته به معماری کامپیوتر، بین 512 بایت تا 1 گیگابایت است. انتخاب توان 2 برای اندازه صفحه، ترجمه آدرس منطقی را به page number و page offset ساده تر می کند.

■ اگر اندازه آدرس منطقی 2^m باشد، و اندازه هر صفحه 2^n بایت باشد، $m-n$ بیت برای page number خواهیم داشت و n بیت برای page offset.



0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Paging

مثال) آدرس منطقی با $n=2$ و $m=4$ را در نظر بگیرید.
اندازه صفحه 4 بایت و حافظه منطقی 16 بایت است.

بنابراین $16/4=4$ صفحه خواهیم داشت. باتوجه به
جدول صفحه، هر صفحه به یک قاب در حافظه
فیزیکی قرار می‌گیرد.

Paging: Fragmentation

■ با استفاده از paging دیگر external fragmentation نخواهیم داشت، زیرا می‌توان هر قاب در حافظه فیزیکی را در اختیار یک صفحه یک فرآیند گذاشت.

■ ولی internal fragmentation می‌تواند رخ دهد. برای مثال اگر اندازه هر صفحه 2048 بایت باشد، برای فرآیندی با اندازه 72766 بایت نیاز به 35 صفحه و 1086 بایت داریم که از 36 قاب استفاده خواهیم کرد و مقدار internal fragmentation برابر $1086 - 2048 = 962$ بایت خواهد بود.

■ بدترین سناریو زمانی است که یک فرآیند نیاز به n صفحه و 1 بایت داشته باشد، در این صورت $n + 1$ قاب با بیشینه internal fragmentation خواهیم داشت.

Paging: Fragmentation

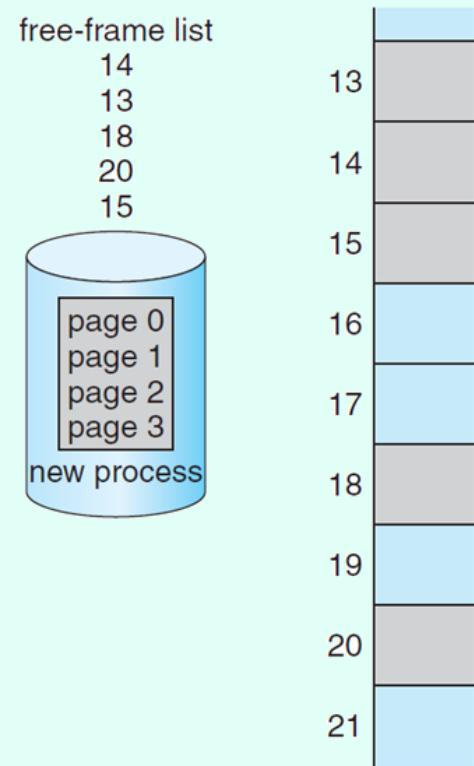
- به طور متوسط مقدار internal fragmentation در paging برابر نصف اندازه یک قاب است. به این ترتیب می‌توان برای کاهش fragmentation، اندازه صفحات را تا حد امکان کم کرد، که البته این کار سربار را افزایش می‌دهد.
- با بزرگ شدن پردازنده‌ها و حافظه اصلی به مرور زمان، اندازه صفحات هم افزایش یافت به طوری که امروزه اندازه صفحه‌ها بین 4 کیلوبایت تا 8 کیلوبایت است.
- برخی سیستم‌ها امروزه از چندین اندازه برای صفحات استفاده می‌کنند، برای مثال Solaris از صفحات 4KB و 4MB استفاده می‌کند.

Paging

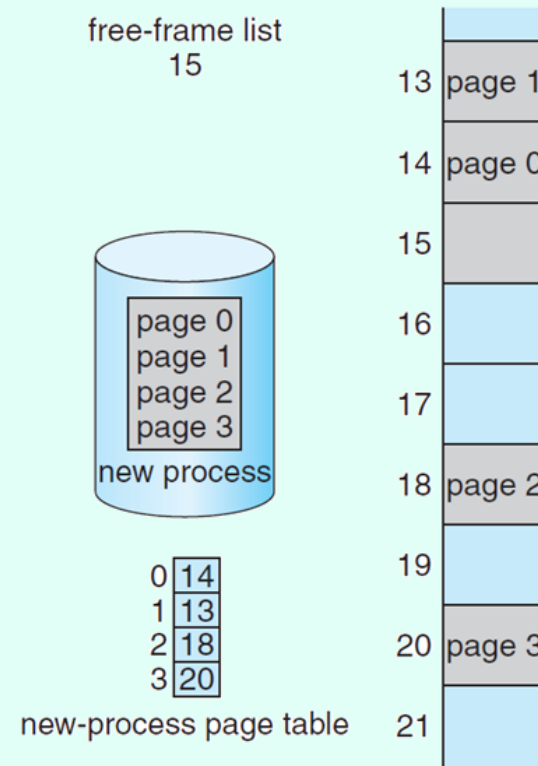
مثال) در یک پردازنده 32 بیتی، هر ردیف جدول صفحه 4 بایت است. هر ردیف می‌تواند به چند قاب اشاره کند؟ و اگر اندازه هر قاب 4 کیلوبایت باشد، حافظه فیزیکی قابل آدرس‌دهی چقدر است؟

Paging

■ وقتی صفحه‌ای در یک قاب بارگذاری می‌شود، شماره آن قاب در جدول صفحه برای آن صفحه نوشته می‌شود:



(a)



(b)

Paging

■ اطلاعات قاب‌ها مانند اینکه کدام قاب‌ها تخصیص داده شده‌اند، کدام قاب‌ها در دسترس هستند، چه تعداد قاب داریم و ... در جدول قاب (frame table) نگهداری می‌شود.

■ جدول قاب برای هر قاب یک ردیف دارد که نشان می‌دهد آیا قاب اختصاص داده شده است یا خیر، و اگر بله به کدام صفحه از کدام فرآیند اختصاص داده شده است.

Paging: TLB

■ هر سیستم عامل روش‌های مخصوص به خود را برای ذخیره جدول‌های صفحه دارد. بعضی سیستم عامل‌ها برای هر فرآیند یک جدول صفحه جداگانه اختصاص می‌دهند. یک اشاره‌گر به جدول صفحه در PCB فرآیند ذخیره می‌شود. برخی دیگر از سیستم عامل‌ها از یک یا نهایت تعداد کمی جدول صفحه استفاده می‌کنند که این کار باعث کاهش سربار هنگام context switch می‌شود.

■ پیاده‌سازی سخت‌افزاری جدول صفحه به روش‌های مختلفی انجام می‌شود. در ساده‌ترین حالت، جدول صفحه به صورت مجموعه‌ای از رجیسترهای اختصاصی پیاده‌سازی می‌شوند. استفاده از رجیستر زمانی مناسب است که جدول صفحه کوچک باشد (برای مثال 256 ردیف). اما بیشتر کامپیوترهای امروزی جدول صفحه بسیار بزرگ دارند (برای مثال 1 میلیون ردیف)، که در این سیستم‌ها استفاده از رجیستر ممکن نیست.

Paging: TLB

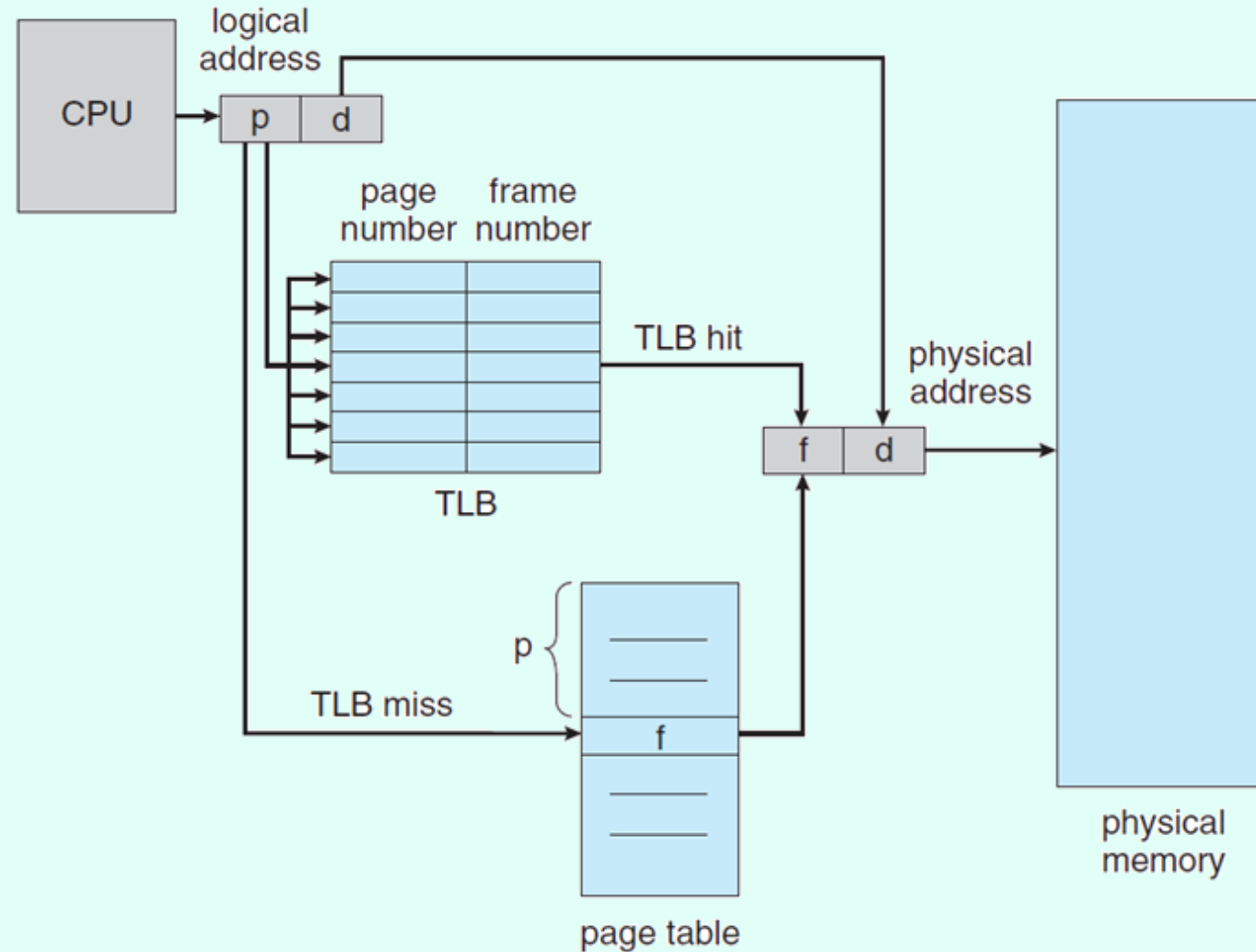
■ در سیستم‌های با جدول صفحه بزرگ، جدول صفحه اصلی نگهداری می‌شود و یک رجیستر پایه جدول صفحه (PTBR) به ابتدای جدول صفحه اشاره می‌کند. برای تغییر جدول، تنها کافی است همین رجیستر را تغییر دهیم که باعث کاهش زمان context switch می‌شود.

■ اما این روش مشکل‌ساز است. اگر بخواهیم به یک مکان در حافظه کاربر دسترسی داشته باشیم، یکبار باید از جدول صفحه آدرس را پیدا کنیم که برای این کار به یک بار دسترسی به حافظه نیاز داریم. و بعد از آن آدرس داده را بخوانیم که خود این نیز دسترسی به حافظه است. یعنی باید دو بار عملیات کند حافظه انجام دهیم.

Paging: TLB

- برای حل این مشکل، از یک حافظه گش کوچک و سریع به نام Translation Look-Aside Buffer یا TLB است. TLB یک حافظه انجمنی (associative) بسیار سریع است. مزیت حافظه انجمنی مقایسه همزمان هر آیتم با تمام کلیدهای آن به طور همزمان است. اگر کلید پیدا شود، فوراً برگردانده می‌شود.
- عملیات جستجوی TLB در خط لوله دستور حافظه انجام می‌شود و هیچ ضرری به کارایی سیستم نمی‌زند. البته برای این امر، باید TLB به حد کافی کوچک باشد، چیزی بین 32 تا 1024 ورودی. برخی پردازنده‌ها از TLBهای جداگانه برای دستورات حافظه و داده استفاده می‌کنند که این امر با حفظ سرعت، ورودی‌ها را دو برابر می‌کند.
- اگر شماره صفحه در TLB پیدا شود، شماره قاب بلافاصله return می‌شود. اگر پیدا نشود (TLB miss)، باید به جدول صفحه در حافظه مراجعه کرد. سپس شماره صفحه و قاب جدید را به TLB اضافه می‌کنیم.

Paging: TLB



Paging: TLB

■ اگر TLB پر باشد و بخواهیم شماره صفحه و قاب جدیدی را اضافه کنیم، باید یکی از ردیف‌ها را حذف کرده و ردیف جدید را اضافه کنیم. این جایگذاری با الگوریتم‌های مختلفی مانند LRU، round robin و تصادفی انجام می‌شود. در برخی TLB‌ها می‌توانیم یک ردیف را wired down کنیم، یعنی هیچگاه حذف نشود.

■ برخی TLB‌ها در هر ردیف شناسه فضای آدرس (ASID) ذخیره می‌کنند. ASID هر فرآیند را به طور یکتا شناسایی می‌کند و برای محافظت فضای آدرس آن به کار می‌رود. وقتی TLB می‌خواهد شماره صفحه مجازی را ترجمه کند، بررسی می‌کند که ASID فرآیند در حال اجرا با ASID صفحه مجازی یکسان باشد. اگر یکسان نباشد، TLB miss است.

■ استفاده از ASID علاوه بر محافظت فضای آدرس، این امکان را می‌دهد که TLB همزمان ردیف‌هایی از چند فرآیند داشته باشد. اگر TLB از ASID استفاده نکند، هر بار که یک جدول صفحه جدید انتخاب می‌شود، باید کل TLB پاک (flush) شود.

Paging: TLB

■ اگر شماره صفحه مد نظر در TLB پیدا شود، به آن **TLB hit** و اگر پیدا نشود به آن **TLB miss** گوئیم. به درصد دفعاتی که شماره صفحه مد نظر در TLB یافت شده است، **نرخ برخورد (hit ratio)** گوئیم.

■ **زمان موثر دسترسی به حافظه (effective memory-access time)**، زمان مورد نیاز CPU برای دسترسی به حافظه است. این زمان میانگین وزن دار دو حالتی است که شماره صفحه در TLB هست یا نیست. اگر زمان دسترسی به حافظه t باشد، زمان موثر به صورت زیر محاسبه می‌شود:

$$effective\ access\ time = hit\ ratio \times t + miss\ ratio \times 2t$$

اگر شماره صفحه در TLB باشد، فقط یک دسترسی به حافظه نیاز داریم، و اگر نباشد به دو دسترسی نیاز داریم.

Paging: TLB

مثال) اگر هر عملیات دسترسی به حافظه 100 نانوثانیه طول بکشد، در سیستمی که TLB نرخ برخورد 80 درصد دارد، زمان موثر دسترسی به حافظه چقدر است؟ اگر نرخ برخورد 99 درصد باشد چطور؟

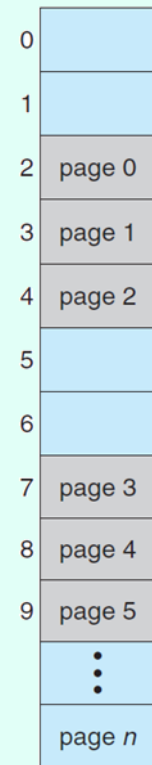
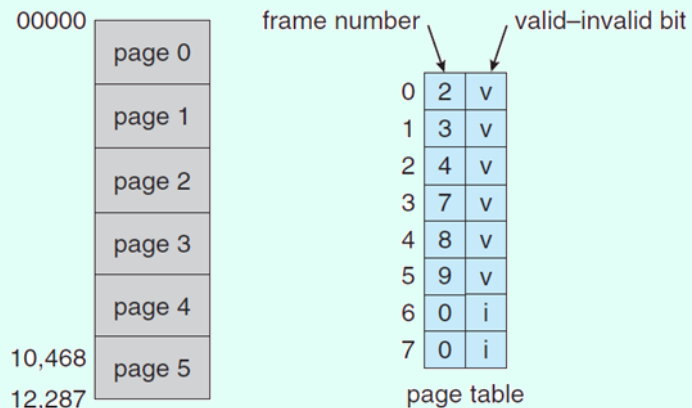
Paging: Protection

■ حفاظت حافظه در یک محیط صفحه‌بندی شده از طریق بیت‌های حفاظتی که به هر قاب مربوط هستند انجام می‌شود. معمولاً این بیت‌ها در جدول صفحه نگهداری می‌شوند. یک بیت می‌تواند مشخص کند که یک صفحه فقط قابل خواندن است (read only)، یا قابل خواندن و نوشتن. با افزایش تعداد بیت‌ها می‌توان انواع دسترسی‌ها را کنترل کرد.

■ یک بیت دیگر در هر ردیف جدول صفحه، بیت معتبر-نامعتبر (valid-invalid) است. اگر این بیت حالت معتبر را برای صفحه مدنظر نشان دهد، دسترسی به آن قانونی است.

Paging: Protection

مثال) در سیستمی با فضای آدرس 14 بیتی (0 تا 16383)، برنامه‌ای داریم که فقط از آدرس‌های 0 تا 10468 استفاده می‌کند و اندازه هر صفحه 2 کیلوبایت است. صفحات 0 تا 5 به طور نرمال نگاشت می‌شوند، ولی صفحات 6 و 7 به دلیل نامعتبر بودن بیت valid-invalid، به حالت تله می‌روند.



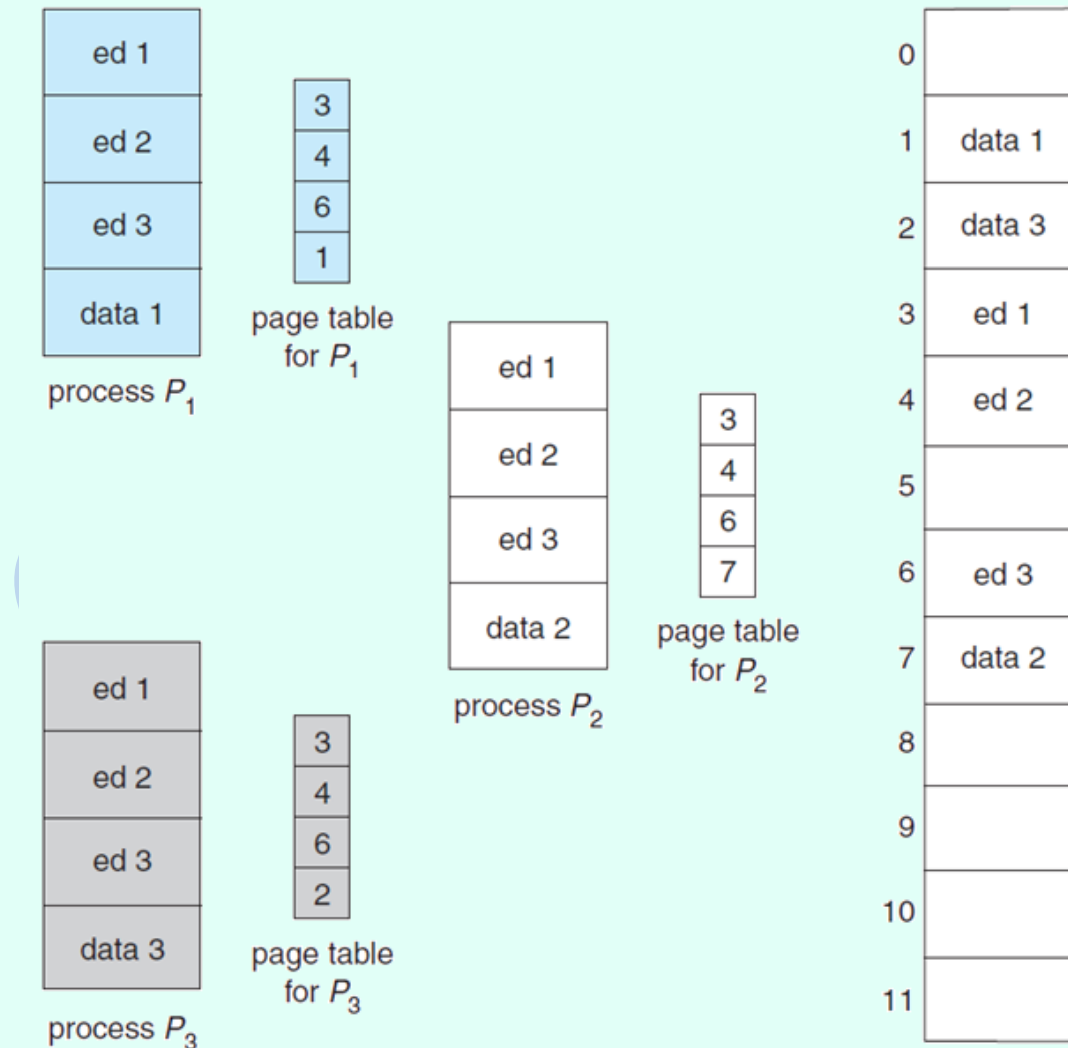
نکته: برنامه تا آدرس 10468 ادامه دارد و هر ارجاعی بعد از آن باید غیرقانونی باشد، اما ارجاع 10468 تا 12287 در صفحه 5 قانونی در نظر گرفته شده است. این مشکل به دلیل سایز 2 کیلوبایتی صفحات است و نمودی از internal fragmentation است.

Paging: Shared Pages

■ یکی از مزایای صفحه‌بندی، امکان اشتراک گذاری کدهای مشترک است. این موضوع در محیط‌های اشتراک زمانی (time sharing) اهمیت زیادی دارد.

■ فرض کنید سیستمی از 40 کاربر پشتیبانی می‌کند و هر کدام یک ویرایشگر متن اجرا می‌کنند. اگر ویرایشگر شامل 150 کیلوبایت کد و 50 کیلوبایت داده باشد، برای پشتیبانی از 40 کاربر به 8000 کیلوبایت حافظه نیاز داریم. اما اگر کد در طول اجرا ثابت باقی بماند (که به آن reentrant code یا pure code گوئیم)، می‌توان آن را به اشتراک گذاشت. در نتیجه به جای 40 نسخه از کد ویرایشگر، یک نسخه نگهداری می‌شود. بنابراین به جای 8000 کیلوبایت، فضای مورد نیازمان $2150 = 150 + 40(50)$ خواهد بود.

Paging: Shared Pages



Structure of Page Table

■ ساختن جدول صفحه به روش‌های مختلفی انجام می‌شود که مورد استفاده‌ترین روش‌های آن عبارت‌اند از:

1. صفحه‌بندی سلسه مراتبی (Hierarchical Paging)

2. جدول صفحه درهم‌سازی شده (Hashed Page Table)

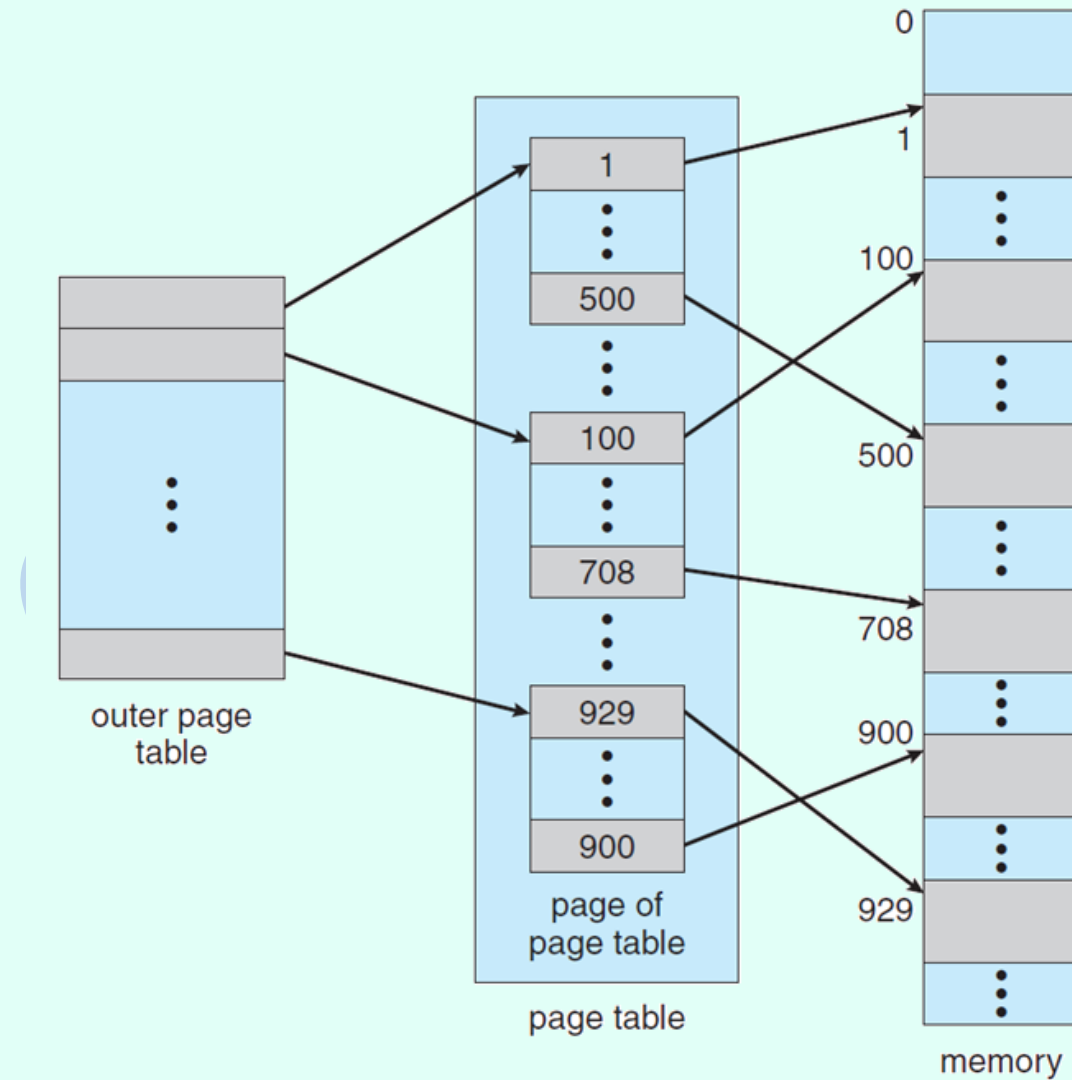
3. جدول صفحه معکوس (Inverted Page Table)

Hierarchical Paging

■ سیستم‌های کامپیوتری مدرن از فضای آدرس منطقی وسیعی پشتیبانی می‌کنند. در این سیستم‌ها جدول صفحه بسیار بزرگ خواهد بود. برای مثال سیستمی با فضای آدرس منطقی 32 بیتی که سایز هر صفحه 4 کیلوبایت است، جدول صفحه 1 میلیون ورودی خواهد داشت. اگر هر ورودی شامل 4 بایت باشد، هر فرآیند به 4 مگابایت فضای آدرس فیزیکی برای جدول صفحه نیاز خواهد داشت.

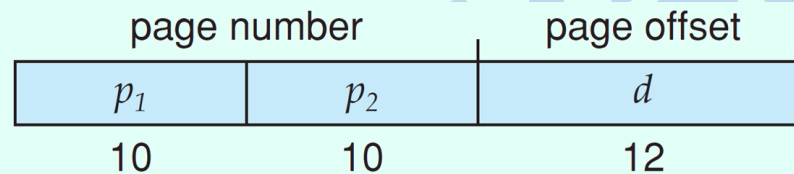
■ یک روش ساده که مجبور نباشیم این فضا را در حافظه اختصاص دهیم، تقسیم جدول صفحه به تکه‌های کوچک‌تر است. یک روش استفاده از یک الگوریتم صفحه‌بندی دو لایه (two-level) است که خود صفحه جدول نیز صفحه‌بندی می‌شود.

Hierarchical Paging

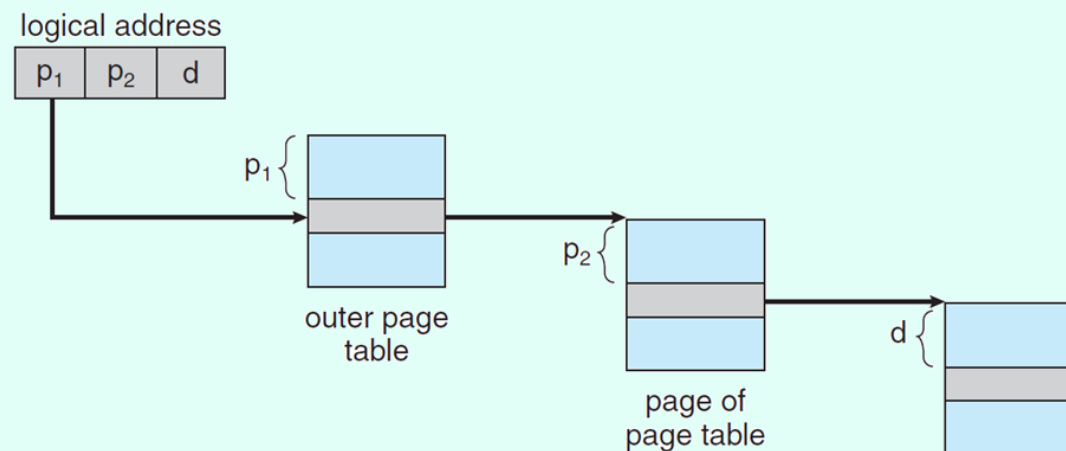


Hierarchical Paging

■ سیستمی با 32 بیت آدرس منطقی را در نظر بگیرید که 12 بیت page offset و 20 بیت page number است. برای دو لایه کردن، 20 بیت را به دو تا 10 بیت میشکنیم:

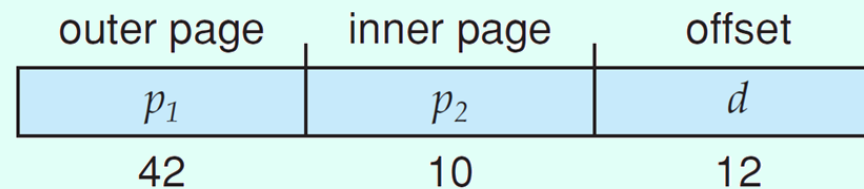


■ در شکل بالا p_1 اندیس یک جدول صفحه کوچک است و p_2 اندیسی برای جدول صفحه لایه دوم است که از لایه اول به دست آوردیم. به این روش forward-mapped page table میگوییم.

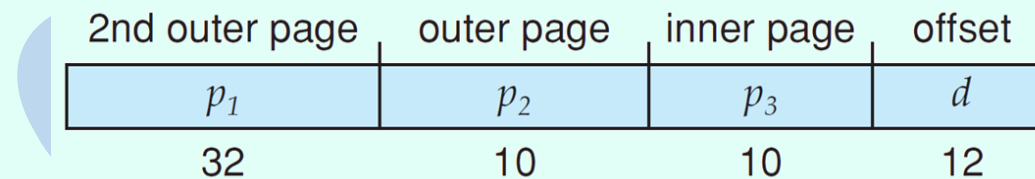


Hierarchical Paging

■ اگر آدرس منطقی 64 بیت شود، دیگر استفاده از صفحه‌بندی دو لایه مناسب نیست:



اندازه جدول بیرونی بسیار زیاد می‌شود. شاید افزایش تعداد لایه‌ها مشکل را حل کند:



همچنان حجم بسیار زیادی دارد. آیا باز هم لایه اضافه کنیم؟ شاید این کار مشکل حجم را حل کند، ولی تعداد دسترسی‌های به حافظه را افزایش می‌دهد و سرعت را کم می‌کند. پس چه کار کنیم؟ خواهیم دید.

Hashed Page Table

■ جدول صفحه درهم‌سازی شده، روشی برای حل مشکل آدرس‌های بزرگتر از 32 بیت است. مقدار هَش از شماره صفحه به دست می‌آید. هر ردیف جدول هَش یک لیست پیوندی از عناصر زیر است:

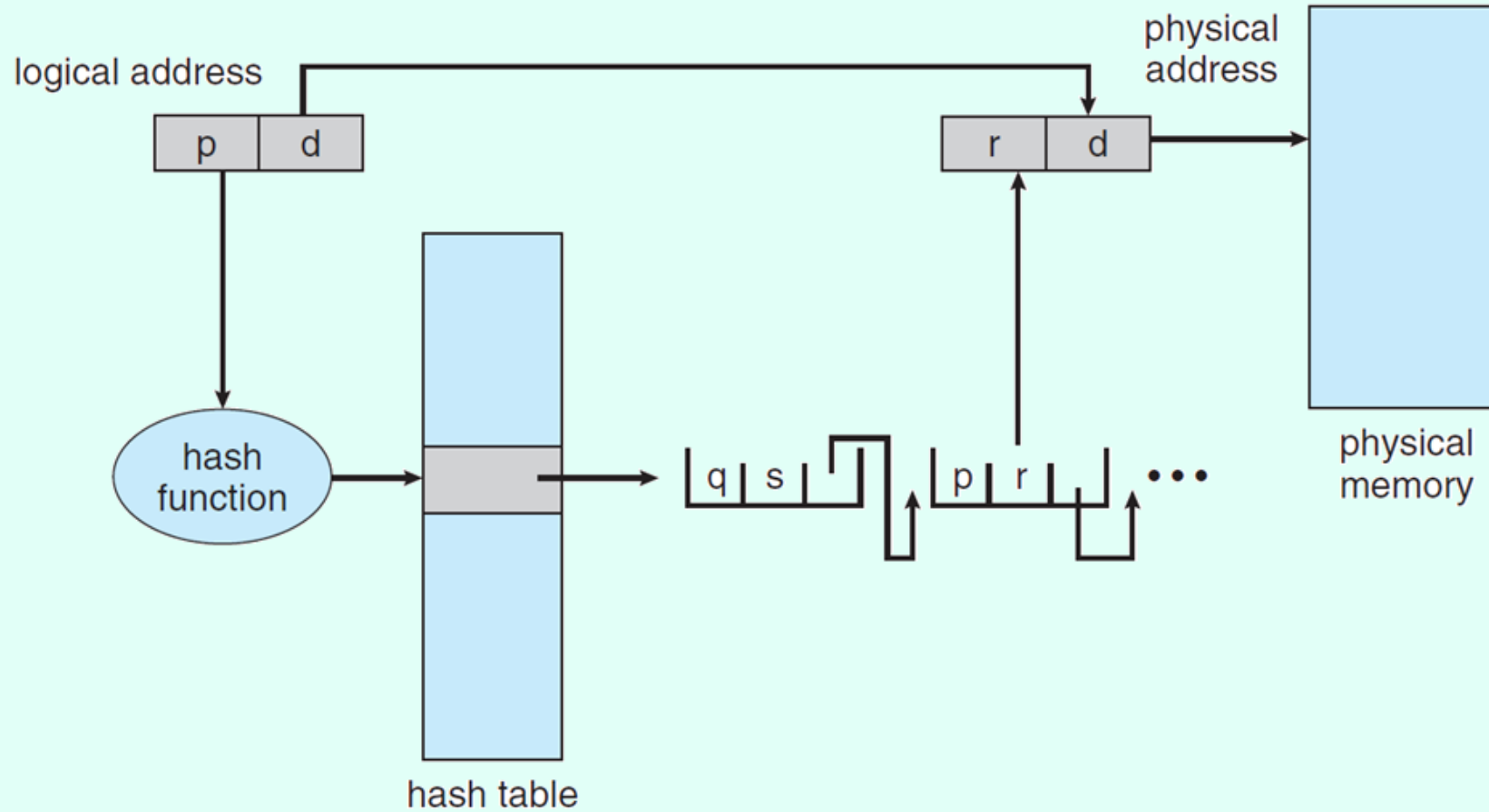
1. شماره صفحه مجازی

2. قاب فیزیکی متناظر

3. اشاره‌گر به عنصر بعدی لیست پیوندی

■ شماره صفحه آدرس منطقی وارد تابع هَش می‌شود. حاصل آن شماره ورودی جدول هَش را تعیین می‌کند. در آن ورودی عنصر اول لیست پیوندی با شماره صفحه مقایسه می‌شود، اگر برابر بودند، عنصر دوم لیست پیوندی به عنوان قاب استفاده می‌شود. اگر برابر نبودند، در عناصر دیگر لیست پیوندی دنبال شماره صفحه می‌گردیم.

Hashed Page Table



Inverted Page Table

■ معمولاً هر فرآیند یک جدول صفحه مخصوص به خود دارد و برای هر صفحه یک ورودی در جدول وجود دارد. از معایب این روش این است که هر جدول صفحه ممکن است شامل میلیون‌ها ورودی باشد و فضای زیادی از حافظه را اشغال کند.

■ برای حل این مشکل از جدول صفحه معکوس (Inverted Page Table) استفاده می‌کنیم. در این جدول برای هر قاب فیزیکی یک ورودی وجود دارد. هر ورودی شامل آدرس مجازی صفحه‌ای است که در حافظه واقعی ذخیره شده است، به همراه اطلاعاتی درباره فرآیند مالک آن صفحه.

Inverted Page Table

